

Regaieg Slim

Lycée ouardanine

Cahier de cours informatique

4 sciences de l'informatiques

TIC



HTML | css | Javascript | Base de données

Pages web dynamiques | SQL | PHP

Nom et Prénom de l'élève :

.....

Année scolaire:2023-2024

Sommaire

Pages 2.....27	<i>Rappel html 5 et css et iframe</i>
Pages 28.... 35	<i>Animation,trasformation et transition avec css3</i>
Pages 36.....84	<i>Javascript cour et TP</i>
Pages 85.....136	<i>Base de données cours et TP</i>
Pages 137...167	<i>PHP cours et tp</i>
Pages 169....209	<i>Annexes html,css,sql,javascript,php et et examens</i>



Rappel HTML5 +css3



Les balises de listes
Mise en forme avec css

4 ème SI -sti

HTML5

C'est quoi HTML5

HTML 5 (*HyperText Markup Language*) est un langage de balisage (dit aussi langage de marquage) qui permet de structurer le contenu des pages web dans différents éléments.

1 C'est un langage interprété par les navigateurs (Internet Explorer, Firefox, Chrome ...), ne pouvant en aucun cas effectuer des calculs, même simples comme une multiplication.

Avantages HTML5

Amélioration de la sémantique des pages Web.

Structure de code plus propre et organisation du site en fonction des contenus.

Déclaration claire et simple de Doctype : `<!DOCTYPE html>`.

Création des animations, diffusion des vidéos ou de la musique.

Compatibilité entre les différents navigateurs et appareils (Pc, Smartphones...).

Ajout d'un Canvas pour l'animation et le développement de jeux. Possibilité de navigation hors ligne.

Prise en charge de la géolocalisation.

Amélioration de stockage.

Fournit des avantages d'optimisation pour les moteurs de recherche.

Structures HTML5

En-tête du document,
Il fournit de multiples
renseignements sur le
document lui-même

Corps du
document.
Le contenu de la
page : texte, blocs,
images

```
<!DOCTYPE html>
<html lang="fr">

<html>
  <head>
    <meta charset="utf-8">
    <title> Titre du document </title>
  </head>
  <body>
    Corps du document.
  </body>
</html>
```

Le Doctype au
début
de page

La balise <HTML>
indique au navigateur
qu'il s'agit d'un
document HTML

Liste des balises en HTML 5

Balises de premier niveau

- `<html>` Balise principale
- `<head>` En-tête de la page
- `<body>` Corps de la page

Balises d'en-tête

- `<link>` Liaison avec une feuille de style
- `<meta>` Métadonnées de la page web (charset, mots-clés, etc.)
- `<script>` Code JavaScript
- `<title>` Titre de la page

Balises de structuration du texte

- `<blockquote>` Citation (longue)
- `<strong>` Mise en valeur forte
- `<em>` Mise en valeur normale
- `<mark>` Mise en valeur visuelle
- `<h1>` Titre de niveau 1
- `<h2>` Titre de niveau 2
- `<h3>` Titre de niveau 3
- `<h4>` Titre de niveau 4
- `<h5>` Titre de niveau 5
- `<h6>` Titre de niveau 6
- `<img>` Image
- `<a>` Lien hypertexte
- `<br>` Retour à la ligne
- `<p>` Paragraphe
- `<hr>` Ligne de séparation horizontale

Balises de liste

- `<ul>` Liste à puces, non numérotée
- `<ol>` Liste numérotée
- `<li>` Élément de la liste à puces
- `<dl>` Liste de définitions
- `<dt>` Terme à définir
- `<dd>` Définition du terme

Balises de tableau

- `<table>` Tableau
- `<caption>` Titre du tableau
- `<tr>` Ligne de tableau
- `<th>` Cellule d'en-tête
- `<td>` Cellule
- `<thead>` Section de l'en-tête du tableau
- `<tbody>` Section du corps du tableau
- `<tfoot>` Section du pied du tableau

Balises de formulaire

- `<form>` Formulaire
- `<fieldset>` Groupe de champs
- `<legend>` Titre d'un groupe de champs
- `<label>` Libellé d'un champ
- `<input>` Champ de formulaire (texte, mot de passe, case à cocher, bouton, etc.)
- `<textarea>` Zone de saisie multiligne
- `<select>` Liste déroulante
- `<option>` Élément d'une liste déroulante
- `<optgroup>` Groupe d'éléments d'une liste déroulante

Balises de formulaire

- `<header>` En-tête
- `<nav>` Liens principaux de navigation
- `<footer>` Pied de page
- `<section>` Section de page
- `<article>` Article (contenu autonome)
- `<aside>` Informations complémentaires

Quelques balises auto fermantes (orphelines):

`<br>`, `<hr>`, `<img>`, `<input>`, `<link>`, `<meta>`

Les balises d'en-tête < head >

Balise < meta >

Les balises META servent à placer des métadonnées (metadata) dans une page HTML. On placera ces informations dans l'élément head, et elles ne seront pas affichées sur la page et qui sont destinées aux

- ❖ Navigateurs web
- ❖ Moteurs de recherche
- ❖ Outils d'indexation

Exemples

- Définissez des mots-clés pour les moteurs de recherche :

```
<meta name="keywords" content="HTML, CSS, JavaScript">
```

- Définissez une description de votre page Web :

```
<meta name="description" content="Cours HTML et CSS">
```

- Définir l'auteur d'une page :

```
<meta name="author" content="Nom Prenom">
```

- Actualiser le document toutes les 30 secondes :

```
<meta http-equiv="refresh" content="30">
```

- Configuration de la fenêtre d'affichage

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

- Encodage des caractères à destination des navigateurs celle-ci doit se placer après le

```
<meta charset="utf-8">
```

<head> et avant le <title>

❖ Balise < title >

La norme HTML a prévu une balise spéciale pour le titre de la page, c'est la balise < title >. Les moteurs de recherche y ont d'ailleurs accordé une grande importance

```
<title>Ici le titre de votre page</title>
```

❖ Balise < link >

Permet de charger les fichiers de code annexes à la page HTML, notamment les fichiers de feuilles de style (.css)

Exemples

```
<link rel="stylesheet"
```

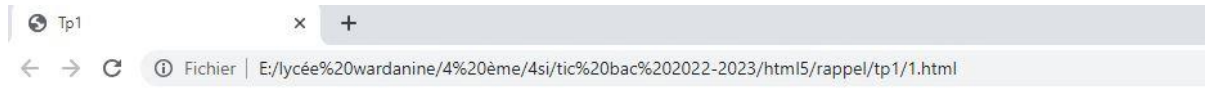
Balise < script >

1 Utilisé pour définir un script en JavaScript encore elle peut pointer vers un fichier de script externe (.js) avec l'attribut src, dans ce cas le contenu de la balise doit rester vide

```
<script src="script.js "> </script>
```

TP1 (HTML5 et CSS3)

- 1) Lancer l'éditeur **visual code**
- 2) Lancer un nouveau fichier
- 3) **Enregistrer** votre travail dans **D:/votre dossier/html5 et css3/ tp1.html**
- 4) **Ecrire** un code **HTML** permettant de réaliser la page web représentée ci-dessous.:



HTML5

HTML5(HyperText Markup Language 5) est la dernière révision majeure du HTML (format de données conçu pour représenter les pages web). Cette version a été finalisée le 28 octobre 2014 .

- 5) **Ajouter** les lignes suivantes à la fin du fichier **tp5** et **visualiser** votre travail.

```
<h3>Évolutions du langage</h3>
<ul><li>1989-1992 : Origine</li></ul>
</body>
```

Évolutions du langage

- 1989-1992 : Origine

🔗 **En déduire** le rôle de chaque balise.

.....

.....

- 6) **Compléter** le reste de la liste sachant qu'elle contient aussi :

- 1993 : HTML 1.0
- 1995-1996 : HTML 2.0
- 1997 : HTML 3.2. et 4.0
- 2000 : XHTML
- De 2007 à nos jours : HTML 5 et abandon du XHTML 2

- 7) **Effectuer** les modifications présentées dans le tableau ci-dessous sur votre code **HTML**.

Modification	Résultat
<code><ul type="square"></code>	
<code><ul type="circle"></code>	

- 8) **Remplacer** la balise la balise **ul** par la balise **ol** et **observer** les résultats obtenus.

🔗 **En déduire** le rôle de chaque balise **ol**.

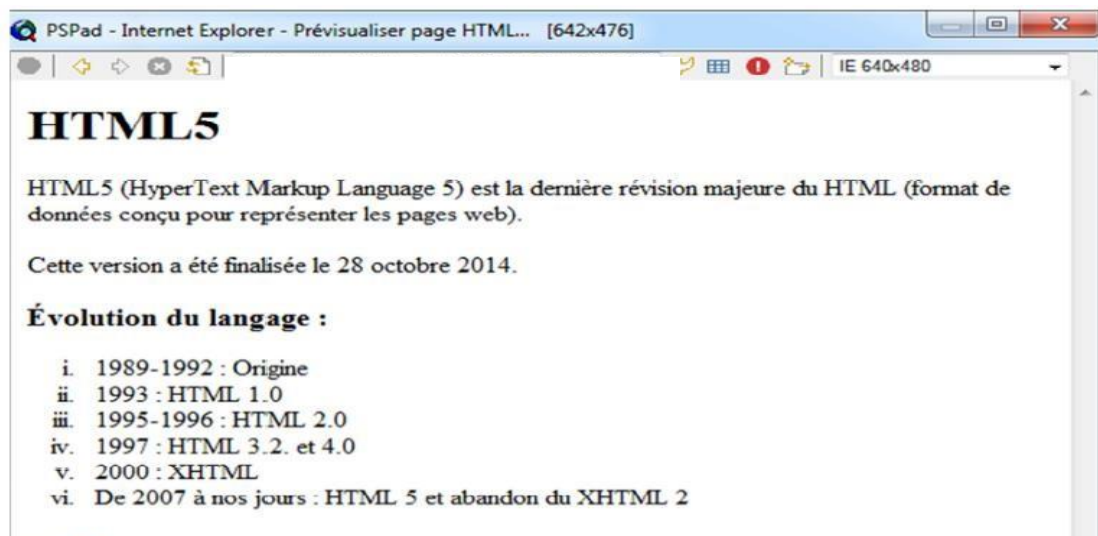
.....

.....

9) Effectuer les modifications présentées dans le tableau ci-dessous sur votre code HTML.

N°	Modification	Résultat
1	<code><ol type="A"></code>	
2	<code><ol type="a"></code>	
3	<code><ol type="I"></code>	
4	<code><ol type="i"></code>	

🔗 Le résultat final de la page est le suivant :



10) créer une feuille de style appelé « **style.css** » qui permet de faire les mises en forme suivantes à la page web « TP1.html » :

- 🔗 **Couleur d'arrière-plan** : rose
- 🔗 **Titre** : centré , souligné , de couleur bleu , taille=40px , police= verdana
- 🔗 **Paragraphe** : taille =20px ,
- 🔗 **La liste numéroté** : Gras , Italique

11) Quelle est la balise qui nous permet de faire la liaison entre la page web et le fichier css ?
Quel est son emplacement dans le code HTML ?

TP2 (HTML5 et CSS3)

- 1) Lancer l'éditeur **visual code**
- 2) Lancer un nouveau fichier
- 3) Enregistrer votre travail dans **D:/votre dossier/html5 et css3/ tp2.html**
- 4) Ecrire un code **HTML** permettant de réaliser la page web représentée ci-dessous.:



- 5) Copier deux images d'extension **jpg** dans votre dossier de travail, et **renommer-les** en **image1.jpg** et **image2.jpg**.

- 6) Ecrire la balise qui permet d'insérer les deux images de la façon suivante :

Image1 : aligné à gauche

Largeur = 200

Hauteur = 150

Info bulle de l'image : « première image »

Image2 : aligné à droite

Bordure = 4

Largeur = 100

Hauteur = 100

Info bulle de l'image : « deuxième image »

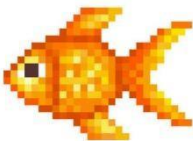
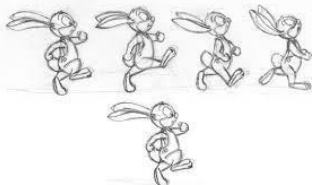
- 7) Comment peut-on mettre une **image** en **arrière-plan** de la page web?

8) ajouter à la fin de la page web le tableau suivant :

Image1	Image2
	

9) Ecrire le code qui construit les tableaux suivants :

📌 Table 1 :

Les types d'images		
Image vectorielle	Image bitmap	Image animée
		

📌 Table 2 :

	Bonjour	Hello
	bonsoir	Hi

📌 Table 3 :

Voici deux colonnes fusionnées		Hello
Bonjour	bonsoir	hi

Balise TD

Attribut	Type de valeur	Description
<u>colspan</u>	entier	Le nombre de colonnes à fusionner
<u>rowspan</u>	entier	Le nombre de lignes à fusionner

TP 3 (HTML5 et CSS3)

-Les liens hypertextes (externes et internes) + iframe

1) Lancer l'éditeur Visual code

2) Lancer un nouveau fichier

3) Enregistrer votre travail dans **C:/votre dossier/html5 et css3/ liens.html**

4) intégrer dans une page HTML le contenu d'une autre page HTML : créer une fenêtre qui nous permet de faire une recherche sur google dans la page web « tp2.html ».

5) Lien hypertexte externe : **Ecrire** un code **HTML** permettant de réaliser la page web **liens.html** qui contient deux liens hypertextes vers les deux pages **tp1.html** et **tp2.html** (Ajouter des liens pour retourner des pages tp1 et tp2 vers la page « liens.html »)

6) Lien hypertexte interne :

- a) Ajouter en haut de la page web tp2.html le mot « descendre vers le bas » qui sera la source d'un lien hypertexte interne vers la fin de page.
- b) Ajouter à la fin de page web tp2.html le mot « Remonter » qui sera la source d'un lien hypertexte interne vers le début de page.

7) Ajouter à chaque balise a les propriétés suivantes, puis interpréter :

Propriété	interpréter
	
	
	

8) En se basant sur le « rappel des mises en forme des liens », appliquer les mises en forme suivantes aux liens de la page web« liens.html »

Mises en forme demandées	Code en CSS
Lien non visité : couleur rouge	
Lien visité : souligné et de couleur bleu	
Lien survolé par le pointeur de la souris : couleur rose	
Lien en cours d'activation : couleur jaune	

Correction du TP 3 (HTML5 et CSS3)

-Les liens hypertextes (externes et internes)

Q4) `<iframe src=" www.google.com " title="description"></iframe>`

Q5)

```
<!DOCTYPE html>
```

```
<html>
```

```
<head>
```

```
    <title></title>
```

```
    <meta charset="utf-8">
```

```
</head>
```

```
<body>
```

```
<p><a href="tp1.html" >TP 1</a></p>
```

```
<p><a href="tp2.html" >TP 2</a></p>
```

```
</body>
```

```
</html>
```

Q6)

↗ lien du bas vers le haut :

```
<a name="top"></a>
```

```
..
```

```
<a href="#top">top</a>
```

↗ Lien du haut vers le bas :

```
<!--source du lien -->
```

```
<p><a href="#down">cliquer ici pour descendre</a></p>
```

```
...
```

```
<!--nom du cible -->
```

```
<h2 id="down">Section plus bas</h2>
```

Q7) Ajouter à chaque balise a les propriétés suivantes , puis interpréter :

Propriété	interpréter
<code></code>	le contexte de navigation actuel. (Par défaut)
<code></code>	le contexte de navigation parent de celui en cours. S'il n'y a pas de parent, il se comporte comme <code>_self</code> .
<code></code>	généralement un nouvel onglet, mais les utilisateurs peuvent configurer les navigateurs pour ouvrir une nouvelle fenêtre à la place.

Q8

Mises en forme demandées	Code en CSS
Lien non visité : couleur rouge	<code>a :link{ color : red ; }</code>
Lien visité : souligné et de couleur bleu	<code>a :visited{ text-decoration : underline; Color : blue; }</code>
Lien survolé par le pointeur de la souris : rose	<code>a :hover{ color : pink ; }</code>
Lien en cours d'activation : couleur jaune	<code>a : active { color : yellow ; }</code>

LES FORMULAIRES

Les pages Web construites en HTML permettent de présenter et de diffuser de l'information en provenance d'un serveur Web vers un navigateur client. Un « dialogue » client-serveur s'instaure lorsque le client peut à son tour envoyer des informations au serveur, notamment par le biais de formulaires :

1. recueil d'informations à l'aide de contrôles dans un formulaire HTML,
2. envoi des informations au serveur,
3. traitement des informations par le serveur (à l'aide d'un langage adapté tels que PHP ou Perl),
4. renvoi éventuel d'informations HTML au client (réponse, accusé de réception, demande de précision...).

Une page peut comprendre un à plusieurs formulaires dans sa section <body>. Tous les contrôles du formulaire doivent être insérés entre les balises <form>...</form> :

Balise	Description	Attributs facultatifs
<form>	formulaire contenant des contrôles	name="nomFormulaire" method="get post" méthode d'envoi des données au serveur (get par défaut). action ("URL") adresse du script auquel les données sont envoyées (pour être traitées). Un à plusieurs événements JavaScript peuvent être spécifiés

Une zone de texte permet de saisir tout type de données. La voici en version « minimale » :

contrôle de formulaire	code HTML
La zone de texte : Nom :	Nom : <input type="text" name="nom" />

L'attribut **name** est obligatoire afin de permettre au serveur, chargé de traiter les valeurs, de récupérer ces valeurs à partir du nom qui leur est attribué (**deux contrôles d'un même formulaire ne peuvent donc pas avoir la même valeur de l'attribut name**).

La version « maximale » :

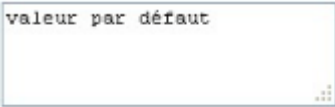
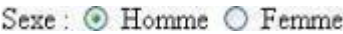
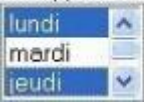
<pre> <label for="idNom">Nom:</label> <input type="text" name="nom" id="idNom" size="20" maxlength="30" placeholder="indiquez votre nom" autofocus required /> </pre>			
↑	↑	↑	↑
zone sélectionnée par défaut (une par page)	contenu requis (validation du formulaire empêchée si la zone est vide)	largeur affichée (en nombre de caractères) de la zone	nombre de caractères maximal saisissable
			message d'aide à la saisie

Le message issu du **placeholder** disparaît lorsque le curseur est placé dans la zone (ce qui est d'emblée le cas ici du fait de l'autofocus) : il apparaît lorsque la zone n'est pas sélectionnée et qu'aucun contenu n'a été saisi. **placeholder**, **autofocus** et **required** sont des attributs HTML5 pas encore reconnus par tous les navigateurs.

L'attribut **default** permet d'indiquer une valeur par défaut dans la zone (que l'arrivée du curseur dans la zone ne supprime pas) : s'il n'y a pas de ressaisie, c'est cette valeur qui sera transmise (alors que placeholder n'indique pas une valeur).

La balise <label> a une portée sémantique (repérer les libellés de formulaire), son attribut **for** se réfère à l'id de la zone de texte (qui peut porter la même valeur que **name**) permet d'envoyer le curseur dans la zone quand on clique sur le libellé.

Présentons les contrôles de formulaire :

contrôle de formulaire	code HTML
La zone de champ caché : elle peut servir à stocker (et transmettre) des valeurs qui n'ont pas besoin d'être affichées à l'écran (mais visibles dans le code source).	<code><input type="hidden" name="c" value="caché" /></code>
Mot de passe :  La zone mot de passe :	Mot de passe : <code><input type="password" name="mp" /></code>
valeur par défaut  La zone de texte long :	<code><textarea type="text" name="lib" rows="3" cols="25">valeur par défaut</textarea></code> L'apparence de la zone de texte multilignes est paramétrable soit à l'aide des attributs <code>rows</code> (lignes) et <code>cols</code> (caractères en elle permet de saisir tout type de données, sur plusieurs lignes colonnes) ou en CSS (propriétés <code>width</code> et <code>height</code>).
Les boutons radio :  ils permettent d'effectuer <u>un seul choix</u> (conditionné par une valeur identique pour l'attribut <code>name</code>) parmi des propositions L'attribut facultatif <code>checked</code> détermine la valeur préselectionnée.	Sexe : <code><input type="radio" name="sexe" value="M" checked /> Homme</code> <code><input type="radio" name="sexe" value="F" /> Femme</code>
Jour(s) de disponibilité : ☐ Mercredi ☐ Samedi ☐ Dimanche Les cases à cocher : elles permettent d'effectuer un à plusieurs choix parmi un nombre limité de propositions ; les choix effectués peuvent être stockés dans un tableau (attribut <code>name</code> ci-contre) ou chaque case peut avoir sa propre valeur de l'attribut <code>name</code>	Jour(s) de disponibilité : <code><input type="checkbox" name="jour[]" value="1" /> Mercredi</code> <code><input type="checkbox" name="jour[]" value="2" /> Samedi</code> <code><input type="checkbox" name="jour[]" value="3" /> Dimanche</code>
Statut :  La liste déroulante : elle permet d'effectuer <u>un seul choix</u> parmi un nombre élevé de propositions	Statut : <code><select name="statut"></code> <code><option value="1">Etudiant</option></code> <code><option value="2">Professeur</option></code> <code></select></code> L'attribut <code>value</code> indique la valeur renvoyée.
Jour(s) de disponibilité :  La zone de liste : elle permet d'afficher d'emblée plusieurs (selon l'attribut <code>size</code>) propositions. L'attribut	Jour(s) de disponibilité : <code><select size="3" multiple name="jour[]"></code> <code><option value="l">lundi</option></code> <code><option value="m">mardi</option></code> <code><option value="j">jeudi</option></code>

facultatif multiple permet le choix multiple (l'attribut name doit alors désigner un tableau)	<pre><option value="v">vendredi</option> </select></pre> L'attribut facultatif selected de la balise <option> permettrait de préselectionner un élément de la liste.
Le fichier joint : PJ: Parcourir... il permet de rechercher un fichier à envoyer en pièce jointe	<pre>PJ: <input type="file" name="pj" value="fichier" /></pre>
Le bouton Annuler : Annuler il réinitialise tous les contrôles du formulaire à vide ou à leur valeur par défaut	<pre><input type="reset" value="Annuler" /></pre>
Envoyer Le bouton Envoyer : il envoie les données du formulaire au serveur pour un traitement par le script spécifié dans l'attribut action de la balise <form>	<pre><input type="submit" value="Envoyer" /></pre>

Les zones de saisie enrichies

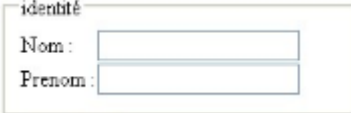
Les zones de type tel, mail et url vérifient que le contenu saisi respecte le format requis et signalent l'erreur si ce n'est pas le cas (empêchant la validation du formulaire) :

type="date"	type="number"	type="range"	type="color"
 il existe également des types time, datetime, week et month	 les attributs min, max permettent de préciser un minimum et un maximum, step un pas d'évolution entre deux nombres <pre><input type="number" min="0" max="9" step="1.5" value="3" /></pre>		 choix parmi 16 millions de couleurs

Regroupement de champ du formulaire par thématique

Liste (déroulante ou non) avec groupements d'options extrait du code HTML	affichage par un navigateur
<pre><select size="9" name="jour"> <optgroup label="semaine"> <option value="1">lundi</option> <option value="2">mardi</option> <option value="3">mercredi</option> <option value="4">jeudi</option> <option value="5" selected>vendredi</option> </optgroup> <optgroup label="week end"> <option value="6">samedi</option> <option value="7">dimanche</option> </optgroup> </select></pre>	 Les libellés de groupes (semaine et week-end) ne sont pas cliquables

Script HTML	Affichage	Liste de suggestions
<pre><input type="text" list="serie"> <datalist id="serie"> <option value="juillet"> <option value="août"> </datalist></pre>	 Rq : si août est saisi il sera ensuite proposée deux fois (une fois par la liste de suggestion, une fois par la mémoire cache du navigateur).	Ce type de valeur est le plus souvent alimenté à partir de valeurs déjà enregistrées dans une base de données.

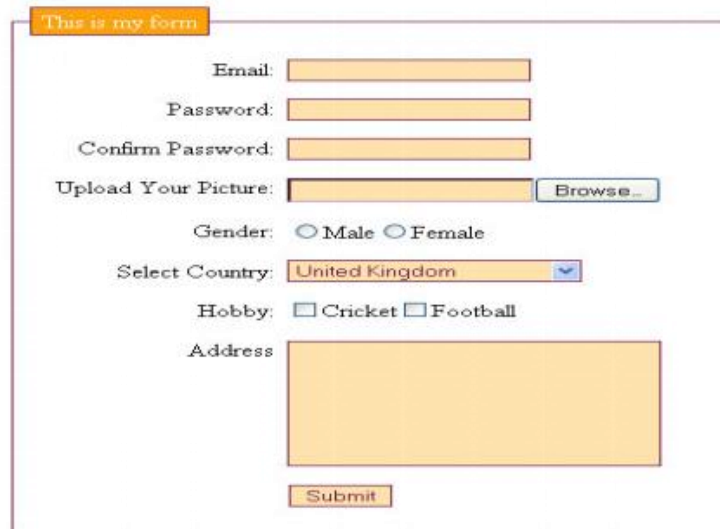
extrait du code HTML	affichage par un navigateur
<pre><fieldset> <legend>identité</legend> <table> <tr><td><label for="idNom">Nom :</label></td> <td><input type="text" name="nom" /></td> </tr> <tr> <td><label for="idPrenom">Prenom : </label></td> <td><input type="text" name="prenom" /></td> </tr> </table> </fieldset></pre>	

Remarque :

Output `<output onload="value = 2 + 2"></output>`

Progress  `<progress id="prog" max=100 value=75> 75%</progress>`

L'apparence des messages de validation est personnalisables à l'aide de pseudo-classes CSS : `:required` | `:optional`, `:valid` | `:invalid` et `:in-range` | `:out-of-range`.

Exercice1 :


Exercise2 :

Step 1: Your details

Name

Email

Phone

Step 2: Delivery address

Address

Post code

Country

Step 3: Card details

Card type
☐  VISA ☐  AmEx ☐  Mastercard

Card number

Security code

Name on card

BUY IT!

Résumé : Les formulaires en HTML5

Les éléments d'un formulaire :

<pre><input type="text" name=" " value=" " ></pre>	Liste déroulante :
<pre><input type="email" name=" " value=" "></pre>	Exemple:
<pre><input type="number" name=" " value=" "></pre>	<pre><select name="profil"></pre>
<pre><input type="tel" name=" " value=" "></pre>	<pre> <option value="parti">Un</pre>
<pre><input type="password" name=" " value=" "></pre>	<pre> particulier</option></pre>
<pre><input type="date" name=" " value=" "></pre>	<pre> <option value="profe" selected="selecte</pre>
<pre><input type="time" name=" " value=" "></pre>	<pre> d">Un professionnel</option></pre>
<pre><input type="range" name=" " value=" "></pre>	<pre> <option value="insti">Un</pre>
<pre><input type="button" name=" " value=" "></pre>	<pre> institutionnel</option></pre>
<pre><input type="submit" name=" " value=" "></pre>	<pre></select></pre>
<pre><input type="reset" name=" " value=" "></pre>	<pre></p></pre>
<pre><input type="checkbox" name=" " value=" "></pre>	Liste déroulante avancée :
Exemple :	Exemple:
<pre><p> Quel est votre sport préféré ?</p></pre>	<pre><input list="nom"></pre>
<pre><input type="checkbox" name="CB"</pre>	<pre> <datalist id="nom"></pre>
<pre> value="foot">Football</pre>	<pre> <option value="....."></pre>
<pre><input type="checkbox" name="CB"</pre>	<pre> <option value="....."></pre>
<pre> value="course">Course</pre>	<pre></datalist></pre>
<pre><input type="checkbox" name="CB"</pre>	<pre><textarea id=" " name=" " rows="5"</pre>
<pre> value="box">Box Américain</pre>	<pre> cols="33"></pre>
<pre><input type="radio"></pre>	<pre></textarea></pre>
Exemple :	Groupe d'éléments d'un formulaire:
<pre><p></pre>	<pre><fieldset></pre>
<pre> <input type="radio" name="civi" value="M</pre>	<pre> <legend>nom du groupe </legend></pre>
<pre> me" /> Madame</pre>	<pre> </pre>
<pre> <input type="radio" name="civi" value="Mll</pre>	<pre></fieldset></pre>
<pre> e" /> Mademoiselle</pre>	
<pre> <input type="radio" name="civi" value="Mr</pre>	
<pre> " /> Monsieur</pre>	
<pre></p></pre>	

Tous les éléments d'un formulaire doivent être dans la balise :

```
<form action=" " method="post">
```

Elements du formulaire

```
</form>
```

TP 4(HTML5 et CSS3)

-Les formulaires

- 1) Lancer l'éditeur **visual code**
- 2) Lancer un nouveau fichier
- 3) **Enregistrer** votre travail dans **D:/votre dossier/html5 et css3/ liens.html**
- 4) **formulaire** : **Ecrire** un code **HTML** permettant de réaliser la page web **formulaire1.html** pour remplir les données suivantes :
 - Créer la liste de sélection en utilisant la balise « SELECT »

The image shows a web form with the following elements:

- Three radio buttons for gender: ☐ Madame, ☐ Mademoiselle, ☐ Monsieur.
- Text input for "Votre prénom :".
- Text input for "Votre nom :".
- Text input for "Votre mot de passe :".
- A dropdown menu labeled "Vous êtes" with the option "Un professionnel" selected. An arrow points to a detailed view of this dropdown.
- Text "Quel type de prestation recherchez vous ?" followed by two checkboxes: ☐ Location de mobilier and ☐ Achat de mobilier.
- A text area for "Votre message :" with the placeholder text "Vous pouvez saisir ici un message."
- Two buttons at the bottom: "Envoyer" and "Annuler".

The detailed view of the "Vous êtes" dropdown menu shows the following options:

- Un professionnel (selected)
- Un particulier
- Un professionnel
- Un institutionnel

- 5) Recréer la liste déroulante en utilisant la balise « datalist »

Correction du TP 4(HTML5 et CSS3)

-Les formulaires

```

4) <form name="mon-formulaire1" action=" " method="get">
<p>
  <input type="radio" name="civi" value="Mme" /> Madame
  <input type="radio" name="civi" value="Mlle" /> Mademoiselle
  <input type="radio" name="civi" value="Mr" /> Monsieur
</p>
<p>
  Votre prénom :<br />
  <input type="text" name="prenom" value="" />
</p>
<p>
  Votre nom :<br />
  <input type="text" name="nom" value="" />
</p>
<p>
  Votre mot de passe :<br />
  <input type="password" name="passe" value="" />
</p>
<p>
  Vous êtes<br />
  <select name="profil">
    <option value="parti">Un particulier</option>
    <option value="profe" selected="selected">Un professionnel</option>
    <option value="insti">Un institutionnel</option>
  </select>
</p>
<p>
  Quel type de prestation recherchez vous ?<br />
  <input type="checkbox" name="interet" value="loc" /> Location de
  mobilier
  <input type="checkbox" name="interet" value="achat" /> Achat de
  mobilier
</p>
<p>
  Votre message :<br />
  <textarea name="le-message" rows="6" cols="40">Vous pouvez saisir ici
  un message.</textarea>
</p>
<p>
  <input type="submit" value="Envoyer" />
  <input type="reset" value="Annuler" />
</p>
</form>

5) <input list="profil" id="myprofil" name="p"/>
<datalist id="profil">
  <option value="Un particulier">
  <option value="Un professionnel">
  <option value="Un institutionnel">
</datalist>

```

Les événements

Les évènements permettent d'effectuer une action (un algorithme) à des **moments** bien précis

1.1 Les événements de la page

Attribut	Description
<u>onload</u>	Se déclenche lorsque la page est complètement chargée. Exemple d'appel : <code><body onload="myFunction()"></code>

1.2 Les événements des éléments du formulaire

Attribut	Description
onsubmit	Se déclenche lorsqu'un formulaire est soumis. <u>Exemple d'appel :</u> <code><form action="..." onsubmit="return myFunction() "></code>
<u>oninput</u>	Se déclenche dès que la valeur d'un élément a changé. <u>Exemples d'appels :</u> <code><input id="..." oninput="myFunction() "></code> <code><textarea id="..." oninput="myFunction() "></code>
onblur	Se déclenche au moment où l'élément perd le focus. <u>Exemple d'appel :</u> <code><input id="..." onblur="myFunction() "></code>
onfocus	Se déclenche au moment où l'élément obtient le focus. <u>Exemple d'appel :</u> <code><input id="..." onfocus="myFunction() "></code>

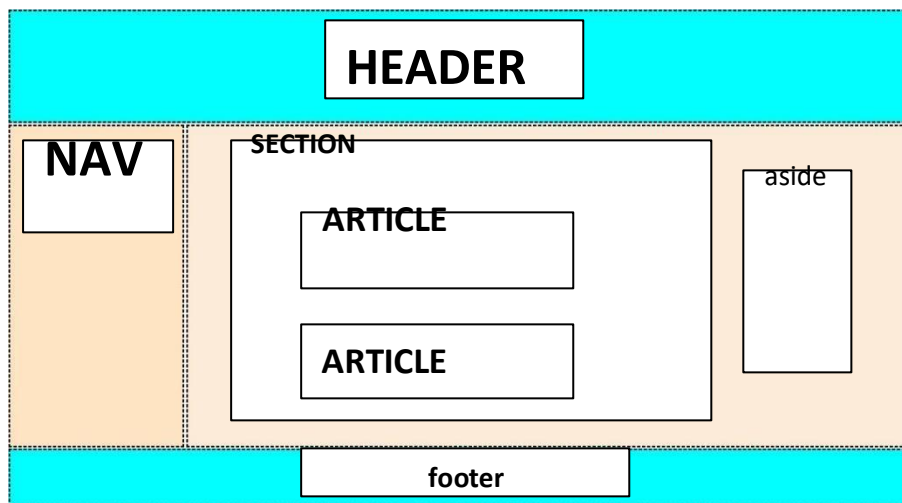
1.3 Les événements de la souris

Attribut	Description
onclick	Se déclenche lors d'un clic sur l'élément. <u>Exemples d'appels :</u> <code><p onclick="myFunction() "></code> <code><input onclick=" myFunction() "></code> <code><div onclick=" myFunction() "></code> <code><h1 onclick="myFunction() "></code>
onmouseover	Se déclenche lorsque le pointeur de la souris survole l'élément. <u>Exemples d'appels :</u> <code><p onmouseover =" myFunction() "></code> <code><input onmouseover =" myFunction() "></code> <code><div onmouseover =" myFunction() "></code> <code><h1 onmouseover =" myFunction() "></code>

TP 5(HTML5 et CSS3)

-Les balises sémantiques + Rappel (image, table, formulaire)

- 1) Lancer l'éditeur **visual code**
- 2) Lancer un nouveau fichier
- 3) **Enregistrer** votre travail dans **D:/votre dossier/html5 et css3/Projet1/ accueil.html**
- 4) **Ecrire** un code **HTML** permettant de réaliser la page web représentée ci-dessous:



- 5) choisir un sujet de votre site web et remplir les différentes parties de cette page web
- 6) Améliorer cette page, en ajoutant un code CSS pour bien organiser ces parties, et faire des mises en forme.
- 7) créer une page qui contient des images, une page qui contient une table et une page qui contient un formulaire
- 8) pour pouvoir naviguer dans votre site web, ajouter des liens hypertexte internes et externes

Correction du TP 5 (HTML5 et CSS3)

-Les balises sémantiques + Rappel (image, table, formulaire)

Q4) `<body>`
 `<header> <h1>bienvenue a mon site web</h1> </header>`
 `<nav>`
 `<p> <a href="page1.html" >page 1</a></p>`
 `<p> <a href="page2.html" >page 2</a></p>`
 `</nav>`
 `<section>`
 `<article>`

 `</article>`
 `<article>`

 `</article>`
 `</section>`
 `<aside> publicite </aside>`
 `<footer>..... </footer>`
`</body>`



Q6)

	Autre méthode qQ6
<pre> header{ border: 1px dashed; top: 0%;background-color: pink; width: 100%; height:20%; } nav{ border:1px dashed; float: left; width: 20%; position: absolute; height: 65%; top: 15%; } section{ border: 1px dashed; margin-left: 21%; width: 60%; height: 65%; position: absolute; top: 15%; } aside{ border: 1px dashed; margin-left: 82%; width: 17%; height: 65%; position: absolute; top: 15%; } footer{ border: 1px dashed; position: absolute; bottom: 0%; background-color: pink; height: 20%; width: 100%; } </pre>	<pre> header { width: auto; height: 200px; top: 0%; border-style: dashed; background-color: aqua; } nav{ display: inline-block; width: 19%; border-style: dashed; height: 580px; background-color: bisque; position: relative; margin-right: 2px; margin-top: 3px; margin-bottom: 2px; } section{ display: inline-block; width: 79%; border-style: dashed; height: 580px; background-color: antiquewhite; position: absolute; margin-top: 3px; margin-left: 2px; margin-bottom: 2px; } footer{ background-color: aqua; width: auto; border-style: dashed; height: 100px; bottom: 0%; } </pre>

<iframe>



Définition :

Un iframe HTML est utilisé pour afficher une page Web dans une page Web.

La balise HTML `<iframe>` spécifie un cadre en ligne.

Un cadre en ligne est utilisé pour incorporer un autre document dans le document HTML actuel.

Syntaxe : `<iframe src="url" title="description"></iframe>`

Conseil : Il est recommandé de toujours inclure un `title` attribut pour le `<iframe>`. Ceci est utilisé par les lecteurs d'écran pour lire le contenu de l'iframe.

Exemple:

a)

```
<iframe src="demo_iframe.htm" height="200" width="300" title="Iframe Example"></iframe>
```

Résultat:



b) Ou vous pouvez ajouter le `style` attribut et utiliser le CSS `height` et les `width` propriétés :

```
<iframe src="demo_iframe.htm" style="height:200px;width:300px;" title="Iframe Example"></iframe>
```

Q1: ajouter un couleur de fond

c) Avec CSS, vous pouvez également modifier la taille, le style et la couleur de la bordure de l'iframe :

```
<iframe src="demo_iframe.htm" style="border:2px solid red;" title="Iframe Example"></iframe>
```

Résultat:



d)Iframe - Cible pour un lien utilisation de **target**

Un iframe peut être utilisé comme cadre cible pour un lien.

L' attribut `target` du lien doit faire référence à l' attribut `name` de l'iframe :

- La balise HTML `<iframe>` spécifie un cadre en ligne
- L' attribut `src` définit l'URL de la page à intégrer
- Toujours inclure un attribut `title` (pour les lecteurs d'écran)
- Les attributs `height` et `width` spécifient la taille de l'iframe
- Utilisez `border:none;` pour supprimer la bordure autour de l'iframe

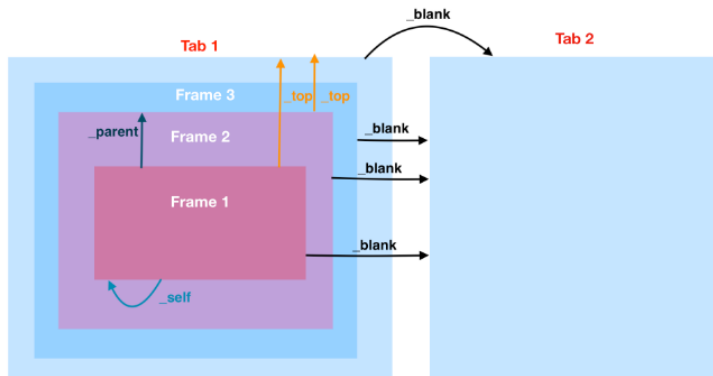
Exemple avec `<a href="" target="">` :

Ajouter un lien pour afficher le site d'inscription :inscription.education.tn dans un cadre (iframe)

Syntaxe :target `<a target= " blank |_self|_parent|_top|nom_du_frame">`

<code>_blank</code>	La page cible s'ouvre dans un nouvel onglet ou dans une nouvelle fenêtre
---------------------	--

_self	Valeur par défaut : la page cible s'ouvre dans le même emplacement (cadre ou « frame ») que là où l'utilisateur a cliqué
_parent	La page cible s'ouvre dans la cadre (frame) de niveau immédiatement supérieur par rapport à l'emplacement du lien
_top	La page cible s'ouvre dans la fenêtre hôte (par dessus le frameset)
nom_du_frame	Ouverture de la page cible dans le cadre portant le nom cité (en valeur de l'attribut name)



```
<!DOCTYPE html>
<html>
  <head>
    <meta http-equiv="content-type" content="text/html; charset=UTF-8">
    <title></title>
  </head>
  <body> .<iframe src="" name="iframe_a" title="Iframe Example"></iframe>
    <p><a href="https://inscription.education.tn/" target="iframe_a">inscription.com</a></p>
    <p><br>
  </p>
</body>
</html>
```

CSS3 (mise en forme des textes)

couleurs	Utilisation de la balise "div"		
color: red; color: #00ff00; color: rgb(34, 12, 64);	<div class="conteneur"><h1>Un titre de niveau 1</h1><p class="p1">Un premier paragraphe</p><p class="p2">Un autre paragraphe</p></div>		
fontes	-----CSS-----		
font-family: arial;	/*On ajoute une bordure autour de l'élément conteneur. *Le div fait 70% de la largeur de son parent (le body)*/ .conteneur{ border: 1px solid black; width: 70%; box-shadow: 5px 5px 0px red; padding: 10px 50px 100px 200px; /* top right bottom left*/ margin: 20px 10px 10px 20px; }		
font-size: 10px;			
font-size: 2.6em;			
font-style: normal; font-style: italic; font-style: oblique;	/*Le texte est centré p/r à l'élément parent (le div conteneur)*/ h1{		
font-weight: 400; /*normal */ font-weight: normal; font-weight: bold;			
Souligner ou surligner un texte	Ombre d'un texte		
h1{ text-decoration: underline; } .p1{ text-decoration: underline overline red; } .p2{ text-decoration: overline dashed; } .p3{ text-decoration: underline wavy; } a{ text-decoration: none; }	/*Ombre noire en bas à droite, nette*/ h1{ text-shadow: 5px 5px; } /*Ombre bleue en bas à gauche, nette*/ .p1{ text-shadow: -5px 2px blue; } /*Ombre rouge centrée sur le texte (décalage horizontal et vertical de 0px), floue*/ .p2{ text-shadow: 0px 0px 5px red; }		
Transformation d'un texte			
h1{ text-transform: uppercase;} .p1{ text-transform: lowercase; } .p2{ text-transform: capitalize;}	NB : Ces mises en forme peuvent être appliquées aux balises de types textes (exemple : p , h1 , a...)		
	text-transform	none capitalize uppercase lowercase	rien 1 ^{ère} lettre des mots en majuscules majuscules minuscules

Les couleurs en CSS

Color	CSS Color Name	Hex Code #RPGGBB	Decimal Code (R,G,B)
	Red	#FF0000	rgb (255,0,0)
	Orange	#FFA500	rgb (255,165,0)
	Yellow	#FFFF00	rgb (255,225,0)
	Green	#008000	rgb (0,128,0)
	Cyan	#00FFFF	rgb (0,255,225)
	Blue	#0000FF	rgb (0,0,225)
	Purple	#800080	rgb (128,0,128)
	Pink	#FFC0CB	rgb (255,192,203)
	Gray	#808080	rgb (128,128,128)
	Brown	#A52A2A	rgb (165,42,42)

Nom des couleurs

Notation hexadécimale

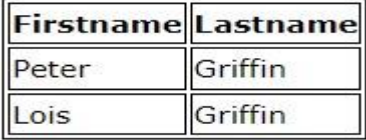
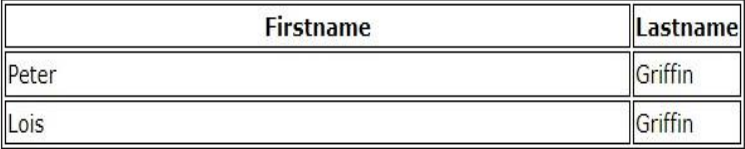
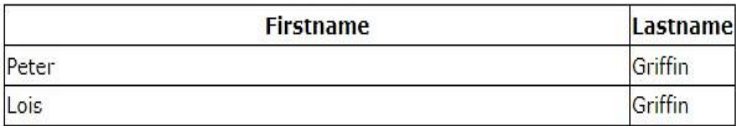
Notation RGB

En anglais, Rouge-Vert-Bleu s'écrit *Red -Green-Blue*

 **rgb** (0..255,0..255,0..255)

 **rgba** (0..255,0..255,0..255,opacité:0..1)

CSS3 (mise en forme d'une table)

code	résultat
<i>bordures d'une table</i>	
<pre>table, th, td { border: 1px solid black; }</pre>	
<pre>table { width: 100%; }</pre>	
<pre>table { border-collapse: collapse; }</pre>	
<pre>table { border: 1px solid black; }</pre>	
<pre>th, td { border-bottom: 1px solid #ddd; }</pre>	
<i>Styles d'une table</i>	
<pre>table { width: 100%; } th { height: 70px; }</pre>	
<pre>table { width: 50%; box-shadow :5px 5px 2px grey; }</pre>	
<i>Alignements d'une table</i>	
<pre>td { text-align: center; }</pre>	
<pre>td { height: 50px; vertical-align: bottom; }</pre>	
<pre>th, td { padding: 15px; text-align: left; }</pre>	

les styles de bordures : **dotted** (pointillé), **dashed** (tirets), **solid** (solide), **double** (double)

La propriété CSS **table-layout** permet de spécifier l'algorithme utilisé pour disposer les cellules, colonnes et lignes d'un tableau.

La propriété de feuille de style **table-layout** CSS peut prendre la valeur de :

fixed : la largeur du tableau et des colonnes est fixé par la largeur des éléments table et col ou par la largeur de la première ligne de cellules.	<table><tr><td>cell</td><td>cell</td><td>A table cell</td><td>A table cell</td></tr><tr><td>cell</td><td>cell</td><td>A table cell</td><td>A table cell</td></tr><tr><td>cell</td><td>cell</td><td>A table cell</td><td>A table cell</td></tr><tr><td>cell</td><td>cell</td><td>A table cell</td><td>A table cell</td></tr></table>	cell	cell	A table cell	A table cell	cell	cell	A table cell	A table cell	cell	cell	A table cell	A table cell	cell	cell	A table cell	A table cell
cell	cell	A table cell	A table cell														
cell	cell	A table cell	A table cell														
cell	cell	A table cell	A table cell														
cell	cell	A table cell	A table cell														
auto : la largeur du tableau et de ces cellules dépendent du contenu.	<table><tr><td>cell</td><td>cell</td><td>A table cell</td><td>A table cell</td></tr><tr><td>cell</td><td>cell</td><td>A table cell</td><td>A table cell</td></tr><tr><td>cell</td><td>cell</td><td>A table cell</td><td>A table cell</td></tr><tr><td>cell</td><td>cell</td><td>A table cell</td><td>A table cell</td></tr></table>	cell	cell	A table cell	A table cell	cell	cell	A table cell	A table cell	cell	cell	A table cell	A table cell	cell	cell	A table cell	A table cell
cell	cell	A table cell	A table cell														
cell	cell	A table cell	A table cell														
cell	cell	A table cell	A table cell														
cell	cell	A table cell	A table cell														

• [Boder-collapse](#)

border-collapse: collapse;

Country:	Capital:
Japan	Tokyo
China	Beijing

border-collapse: separate;

Country:	Capital:
Japan	Tokyo
China	Beijing

no border-spacing <table><tr><td>foo</td><td>one</td></tr><tr><td>bar</td><td>two</td></tr><tr><td>baz</td><td>three</td></tr></table>	foo	one	bar	two	baz	three	border-spacing: 0px <table><tr><td>foo</td><td>one</td></tr><tr><td>bar</td><td>two</td></tr><tr><td>baz</td><td>three</td></tr></table>	foo	one	bar	two	baz	three	border-spacing: 1px <table><tr><td>foo</td><td>one</td></tr><tr><td>bar</td><td>two</td></tr><tr><td>baz</td><td>three</td></tr></table>	foo	one	bar	two	baz	three
foo	one																			
bar	two																			
baz	three																			
foo	one																			
bar	two																			
baz	three																			
foo	one																			
bar	two																			
baz	three																			
border-spacing: 1em <table><tr><td>foo</td><td>one</td></tr><tr><td>bar</td><td>two</td></tr><tr><td>baz</td><td>three</td></tr></table>	foo	one	bar	two	baz	three	border-spacing: 1em 0em <table><tr><td>foo</td><td>one</td></tr><tr><td>bar</td><td>two</td></tr><tr><td>baz</td><td>three</td></tr></table>	foo	one	bar	two	baz	three	border-spacing: 0em 1em <table><tr><td>foo</td><td>one</td></tr><tr><td>bar</td><td>two</td></tr><tr><td>baz</td><td>three</td></tr></table>	foo	one	bar	two	baz	three
foo	one																			
bar	two																			
baz	three																			
foo	one																			
bar	two																			
baz	three																			
foo	one																			
bar	two																			
baz	three																			

CSS3 (*mise en forme d'un lien hypertexte*)

(*Quelques exemples*)

```
a:link {
    color: yellow;
}

/* visited link */
a:visited {
    color: green;
}

/* mouse over link */
a:hover {
    color: hotpink;
    text-decoration: underline;
}

/* selected link */
a:active {
    color: blue;
    text-decoration: none;
}
```

-----A link styled as a button:-----

```
a:link, a:visited {
    background-color: #f44336;
    color: white;
    padding: 14px 25px;
```

❖ **Les sélecteurs avancés**

Les sélecteurs avancés sont l'une des grandes forces du CSS. En effet, grâce à eux, nous allons pouvoir cibler du contenu très précisément et assez simplement. Il existe de très nombreux sélecteurs avancés en CSS on présentera les plus utiles et les plus utilisés dans le tableau suivant :

Sélecteur	Sert à ...
*	Sélectionner tous les éléments (sélecteur universel)
A, B	Sélectionner deux éléments A et B
A B	Sélectionner un élément B contenu dans un élément A
A + B	Sélectionner le premier élément B suivant un élément A
A[nom de l'attribut]	Sélectionner tous les éléments A possédant un attribut particulier
A[nom de l'attribut* = « valeur »]	Sélectionner tous les éléments A possédant un attribut particulier avec une valeur
A[nom de l'attribut = « valeur »]	Sélectionner tous les éléments A possédant un attribut particulier avec une valeur précise

font-weight	normal bold	gras
text-align	left right center justify	aligné à gauche aligné à droite centré justifié
text-decoration	none underline overline line-through blink	rien souligné surligné rayé clignotant
text-transform	none capitalize uppercase lowercase	rien 1 ^{ère} lettre des mots en majuscules majuscules minuscules

Le modèle des boîtes

Dans une mise en page réalisée en CSS, tous les éléments sont considérés comme des boîtes. Chacune de ces boîtes est constituée d'un contenu, d'un espacement intérieur, d'une bordure, et d'une marge externe.

Voici les propriétés CSS qui permettent de déterminer les dimensions, la couleur, le style de chacun de ses constituants :

Propriété CSS	Ce qui est concerné :
width et height	largeur et hauteur du contenu (texte, image, etc.)
padding	espacement intérieur , entre le contenu et la bordure
border	bordure (ou encadrement)
margin	marge externe , espace (transparent) entourant le tout



Pour créer une animation CSS on doit :

- créer et nommer l'animation (avec **@keyframes**)
- attacher cette animation à un élément (avec **animation-name**)

1. Notre première animation :

On commence par créer une animation :

```
@keyframes taille {
  from {width: 0px;}
  to {width: 200px;}
}
```

On utilise la règle **@keyframes** et on nomme l'animation **taille**.

Rque : Il existe quelques règles avec le signe **@** (nommées **at-rules**) qui permettent d'encapsuler plusieurs règles pour le processeur CSS du navigateur, par exemple **@media**, **@page**...

Ici, on change la largeur (**width**) de (**from**) 0px à (**to**) 200px.

On attache ensuite cette animation à un élément :

```
div {
  animation-name: taille;
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-duration: 4s;
}
```

On utilise :

- **animation-name** : pour désigner l'animation utilisée. (*obligatoire*)
- **animation-duration** : pour définir la durée de l'animation (par défaut elle est de 0 seconde). (*obligatoire*)

2. Plusieurs étapes :

L'intérêt des animations est de pouvoir gérer des étapes intermédiaires. Voici une nouvelle version de l'animation :

```
@keyframes taille {
  0% {width: 0px;}
  50% {width: 300px;}
  100% {width: 200px;}
}
```

On a 3 étapes :

- le départ (0%) avec une largeur de 0px
- la moitié (50%) avec une largeur de 300px

- l'arrivée (100%) avec une largeur de 200px

3. Plusieurs propriétés :

On peut aussi changer plusieurs propriétés :

```
@keyframes taille {
  0% {width: 0px; background-color: red;}
  25% {width: 50px; background-color: yellow;}
  50% {width: 100px; background-color: green;}
  75% {width: 150px; background-color: pink;}
  /* 100% {width: 200px; background-color: blue;} devient facultatif car
  l'état 100% est déjà l'état du div "final".*/
}
```

4. Démarrage différé :

Dans les exemples ci-dessus, l'animation commence dès le chargement, ce qui n'est pas toujours souhaitable. On dispose de la propriété **animation-delay** pour différer le départ de la durée voulue.

En poursuivant notre exemple ça pourrait donner ce code :

```
div {
  animation-name: taille;
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-duration: 4s;
  animation-delay: 2s;
}
```

5. Nombre d'exécutions :

Par défaut, une animation est exécutée une seule fois, si on en veut plus, il faut utiliser la propriété **animation-iteration-count**. Toujours avec notre exemple :

```
div {
  animation-name: taille;
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-duration: 4s;
  animation-iteration-count: 2;
}
```

Rq :

Si on veut un nombre d'exécutions infini, au lieu de mettre une valeur on utilisera **infinite**.

6. Sens de l'animation :

Par défaut une animation va dans le sens normal mais on peut aussi l'obliger à aller dans l'autre sens en utilisant la propriété **animation-direction**. On dispose de deux valeurs :

- **reverse** : on va dans le sens inverse.
- **alternate** : on alterne d'un sens à l'autre (dans ce cas évidemment il faut prévoir au moins deux exécutions).

```
div {
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-name: taille;
  animation-duration: 4s;
  animation-iteration-count: infinite;
  animation-direction: alternate;
}
```

7. Progression de l'animation :

On peut modifier l'animation avec la propriété **animation-timing-function** et ces valeurs :

- **ease** : début rapide, puis ça accélère, puis ça ralentit à la fin (c'est l'effet par défaut).
- **linear** : même vitesse du début à la fin.
- **ease-in** : début lent.
- **ease-out** : fin lente.
- **ease-in-out** : début et fin lents.

8. Le style avant et après l'animation :

Le style avant et après l'animation est défini par défaut par les règles de l'élément. On a la possibilité de changer ce comportement en utilisant la propriété **animation-fill-mode**. On dispose de ces valeurs :

- **none** : comportement par défaut (en gros ça sert à rien)
- **forwards** : on garde les valeurs calculées lors de la dernière étape
- **backwards** : on garde les valeurs calculées lors de la première étape

L'interprétation peut être délicate selon les répétitions et sens des animations.

Par exemple avec ce code :

```
@keyframes taille {
  0% {width: 0px ;}
  100% {width: 200px ;}
}
div {
  height: 100px;
  width: 200px;
  background-color: blue;
  animation-name: taille;
  animation-duration: 4s;
  animation-delay: 2s;
  animation-fill-mode: backwards;
}
```

Au départ et pendant le délai de démarrage, la largeur sera de *0px* qui correspond à la première étape (à 0%) parce qu'on a utilisé la valeur **backwards** pour **animation-fill-mode**.

9. Action sur une animation :

Pour l'instant nous n'avons pas vu de possibilité d'intervention sur une animation. On dispose de la propriété **animation-play-state** qui permet une interaction avec ces valeurs :

- **running** : valeur par défaut de l'animation en action.
- **paused** : animation en pause.

Voici le code d'un exemple où on met l'animation en pause en survolant l'élément :


```
@keyframes taille {
  0% {width: 50px ;}
  100% {width: 200px ;}
}
div {
  height: 100px;
  background-color: blue;
  animation-name: taille;
  animation-duration: 4s;
  animation-fill-mode: forwards;
  animation-iteration-count: infinite;
  animation-direction: alternate;
  animation-timing-function: linear;
}
div:hover {
  animation-play-state: paused;
}
```

On peut faire exactement l'inverse et mettre l'animation en action seulement au survol :

```
@keyframes taille {
  0% {width: 50px}
  100% {width: 200px}
}
div {
  height: 100px;
  background-color: blue;
  animation-name: taille;
  animation-duration: 4s;
  animation-fill-mode: forwards;
  animation-iteration-count: infinite;
  animation-direction: alternate;
  animation-timing-function: linear;
  animation-play-state: paused;
}
div:hover {
  animation-play-state: running;
}
```

TP6 : TRANSFORMATION ET TRANSITION D'ÉLÉMENT AVEC CSS3

A l'aide du logiciel « visual code » rédiger le code de la page web nommée « **Tp2css.html** » qui doit avoir l'apparence ci-dessous :

- Créer un titre de premier niveau : « Transformation et Transition ».
- Créer 5 balises « div » à chaque balise associée un ID.
- Dans chaque balise écrire respectivement : « paragraphe 1 » « paragraphe 2 » « paragraphe 3 » « paragraphe 4 » « paragraphe 4 »..
- Dans la balise div 2 : créer une liste de 4 puces : Accueil, à propos, liens, contact.
- Créer une page CSS nommée « **Tp2css.css** » et lier ce fichier avec la page « **Tp2css.html** »

Pour la transition on a les propriétés suivantes :

- ✓ La propriété **transition-duration** permet de spécifier la durée de la transition (la durée que mettra l'animation pour aller de l'état 1 à l'état 2). Elle est exprimée en secondes.
- ✓ La propriété **transition-property** : permet de désigner quels sont les propriétés qui devront subir la transition et quels sont celles qui ne s'animeront pas.
- ✓ La propriété **transition-delay** : permet de retarder le début de la transition lorsque l'événement qui la déclenche est détecté.

L'exemple suivant montre un élément <div> rouge de 100px * 100px. L'élément <div> a également spécifié un effet de transition pour la propriété width, d'une durée de 2 secondes :

```
#d1 {
  width: 100px;
  height: 100px;
  background: red;
  transition: width 2s;
}
```

L'effet de transition commencera lorsque la propriété CSS spécifiée (largeur) changera de valeur.

Maintenant, spécifions une nouvelle valeur pour la propriété width lorsqu'un utilisateur survole l'élément <div> :

Exemple :

```
#d1:hover {
  width: 300px;
}
```

L'exemple suivant ajoute un effet de transition pour les propriétés width et height, avec une durée de 2 secondes pour la largeur et de 4 secondes pour la hauteur :

```
#d1 {
  transition: width 2s, height 4s;
}
```

Dans ce cas, l'animation fera une demi seconde. Comme pour la propriété **transition-property** il faut prévoir le préfixe vendeur à **transition-duration** :

```
#d1{
  border:solid 1px blue;
  background-color:orange;
}
#d1:hover{
  border:dotted 1px green;
  background-color:yellow;
```

```
#d1:hover {
  width: 300px;
  height: 200px;}
```

```
transition-property:background;  
transition-duration:0.5s;
```

```
}
```

Dans l'exemple suivant, on opère une transition CSS sur la taille de police du titre de quatre secondes après deux secondes écoulées lorsque l'utilisateur passe la souris sur l'élément :

```
h1{  
  font-size: 14px;  
  transition-property: font-size;  
  transition-duration: 4s;  
  transition-delay: 2s;  
}
```

```
h1:hover{  
  font-size: 36px; }
```

On utilise parfois CSS pour mettre en avant les éléments d'un menu lorsque l'utilisateur les survole avec sa souris. On peut facilement utiliser les transitions CSS pour améliorer l'effet obtenu. On construit l'apparence du menu :

```
a {  
  color: #fff;  
  background-color: #333;  
  transition: all 1s;  
}  
a:hover,a:focus {  
  color: #333;  
  background-color: #fff;  
  border-radius: 10px; }
```

Appliquer une transition sur plusieurs propriétés CSS :

Cette boîte utilisera des transitions pour width, height, background-color, avec une rotation de 180 degré :

```
#d3{  
  border-style: solid;  
  border-width: 1px;  
  display: block;  
  width: 150px;  
  height: 150px;  
  background-color: #0000FF;  
  transition: width 2s, height 2s, background-color 2s, transform 2s;  
}  
#d3:hover {  
  background-color: #FFCCCC;  
  width: 200px;  
  height: 200px;  
  transform: rotate(180deg); }
```

Un autre exemple :

```
#d4{  
  background-color: red;  
  width:150px;  
  height: 150px;  
  transition-property: opacity, left, top, height;;  
  transition-duration: 3s, 5s, 3s, 5s;  
}
```

```
#d4:hover{
    background-color: red;
    height: 200px;
    opacity: 0.5;
}
```

La propriété `transform` va nous permettre de définir un ou plusieurs effets de transformation à appliquer à un élément : inclinaison, rotation, déformation, etc.

Nous appliquons un premier effet de transformation qui va être une translation, c'est-à-dire un déplacement selon une direction. Ici, on va donc déplacer l'élément de 100px à partir de leur point d'origine vers la droite.

```
#d2{
    background-color: pink;
    width:150px;
    height: 150px;
    transform-origin: 0 0;
    transform:translate(100px);
    text-transform: uppercase;
}
```

Le point d'origine de la transformation est le centre . Comme la transformation n'est ici qu'un déplacement horizontal, modifier le point d'origine ne change pas le résultat de la transformation.

```
#d5{
    background-color: green;
    width:150px;
    height: 150px;
    transform-origin: 50% 50%;
    transform:rotate(45deg);
}
```

```
#d5:hover{
    border:dotted 1px green;
    background-color:orange;
    transition-property:background;
    transition-duration:0.5s; }
```

Exemple de transformation 2D :

La fonction `scale()` permet de modifier la taille ou plus exactement l'échelle de l'élément. Nous allons pouvoir lui passer deux nombres qui vont correspondre au pourcentage d'agrandissement en largeur et en hauteur de l'élément.

Par exemple, en écrivant `transform : scale(2, 0.5)`, l'élément va doubler en largeur et être diminué de moitié en hauteur.

Déformer un élément avec `skewX()` et `skewY()`

A la place, il faudra plutôt utiliser les fonctions `skewX()` et `skewY()` qui vont nous permettre de déformer un élément selon son axe horizontal ou vertical.

```
#d1{
    transform: skewX(30deg); }
#d2{
    transform: skewY(30deg); }
#d3{
    transform: skewX(30deg) skewY(30deg);}
```



INTRODUCTION :

JavaScript est un langage de scripts, permet d'améliorer la présentation et l'interactivité des pages Web (tel que la communication avec le navigateur).

Il permet de :

- ✓ Contrôler la saisie dans un formulaire
- ✓ Permet d'accéder aux objets du navigateur
- ✓ Exécuter des commandes du côté client (date système, gestion de la fenêtre, gestion de navigateur) etc...

Avantage :

- ✓ Rapide en exécution
- ✓ Faible coût, exécuté par le navigateur : (il ne nécessite pas une installation particulière pour pouvoir être exécuté)

Inconvénients :

- ☞ Code source visible et peut être copié par tout le monde (view source).

I.FORMALISME DE BASE DU JAVASCRIPT

1. Pour ajouter un script java, il faut utiliser **<script>** et pour le terminer **</script>**.
2. Tout ce qui est écrit entre le symbole **"//"** et la fin de la ligne, représente un commentaire et il sera ignoré pendant l'exécution.
3. Il est possible d'inclure des commentaires sur plusieurs lignes avec le code :

/ commentaire sur
Plusieurs lignes*/*

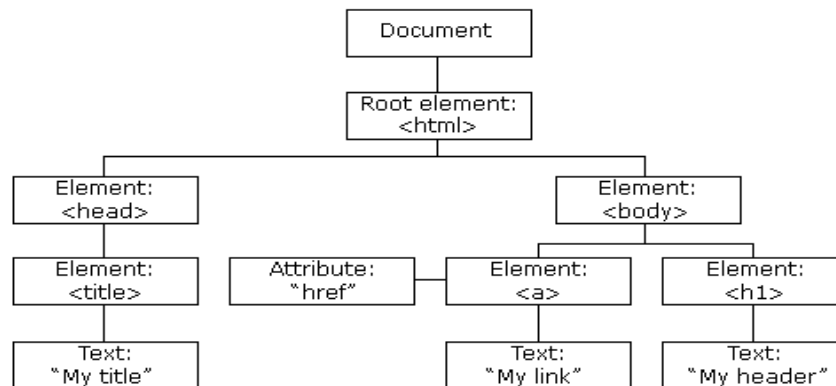
II. DOM JAVASCRIPT HTML

Avec le **DOM HTML**, JavaScript peut accéder et modifier tous les éléments d'un document HTML. Lorsqu'une page Web est chargée, le navigateur crée un **D**ocument **O**bjets **M**odèle de la page. Avec le modèle objet, JavaScript obtient toute la puissance dont il a besoin pour créer du HTML dynamique :

- JavaScript peut modifier tous les éléments HTML de la page
- JavaScript peut modifier tous les attributs HTML de la page
- JavaScript peut changer tous les styles CSS de la page
- JavaScript peut supprimer les éléments et attributs HTML existants
- JavaScript peut ajouter de nouveaux éléments et attributs HTML
- JavaScript peut réagir à tous les événements HTML existants dans la page
- JavaScript peut créer de nouveaux événements HTML dans la page

Le modèle HTML DOM est construit comme un arbre d'Objets :

L'arbre d'objets HTML DOM



IV. Différents emplacements du code JavaScript

a. Avec un script intégré dans le code HTML :

```
<script>
  // Code JavaScript
</script>
```

b. Avec un attribut d'évènement

```
<p onClick='alert("4SI1")'> Bonjour </p>
```

c. Avec un script externe (recommandé) :

Tout d'abord il faut créer un document que l'on appellera **Controles.js** (par exemple) dans lequel on écrira le code que l'on souhaite intégrer. Ensuite dans la page Web il suffira d'intégrer :

```
<script src="Controles.js"> </script>
```

Dans ce cas, la balise doit être vide

III. Les Structures de données en JavaScript :

Il existe 3 façons de déclarer une variable JavaScript :

- À l'aide de **var**
- À l'aide de **let**
- À l'aide de **const**



	var	let	const
Stockée en global	✓	✗	✗
Portée de la fonction	✓	✓	✓
Portée du block	✗	✓	✓
Ré-assignation	✓	✓	✗
Re-déclaration	✓	✗	✗
Hoisting	✓	✗	✗

- ☞ Lorsque vous déclarez une nouvelle variable, pensez à privilégier d'abord **const**, puis **let** et enfin **var** en dernier recours.
- ☞ Attention néanmoins, **let** et **const** ne sont supportés que par les navigateurs qui sont compatibles avec **l'ES6**

JavaScript est un langage à typage dynamique. Cela signifie que vous n'avez pas besoin de spécifier le type de données d'une variable lorsque vous la déclarez. Les types de données sont convertis automatiquement durant l'exécution du script.

Type	Exemples de valeurs typées / Notes
Nombres (number)	42, 3.14159
Booléens (boolean)	true / false
Chaînes de caractères (string)	"Bac Informatique"
null	Un mot-clé spécifique pour désigner une valeur informatique nulle (et pas mathématique).
undefined	Une variable pour laquelle aucune valeur n'a été assignée sera de type undefined.

IV. OPERATEURS PREDEFINIS

1. LES OPERATEURS ARITHMETIQUES : SOIT Y = 5

Opérateur	Description	Exemple	Y	X
+	Addition	$x = y + 2$	$y = 5$	$x = 7$
-	Soustraction	$x = y - 2$	$y = 5$	$x = 3$
*	Multiplication	$x = y * 2$	$y = 5$	$x = 10$
/	Division	$x = y / 2$	$y = 5$	$x = 2.5$
%	Modulo	$x = y \% 2$	$y = 5$	$x = 1$
++	Pré-Incrémentation	$x = ++y$	$y = 6$	$x = 6$
++	Post-Incrémentation	$x = y++$	$y = 6$	$x = 5$
--	Pré-Décrémentation	$x = --y$	$y = 4$	$x = 4$
--	Post-Décrémentation	$x = y--$	$y = 4$	$x = 5$

2. LES OPERATEURS D'AFFECTATION : SOIENT X = 10 ET Y = 5

Opérateur	Exemple	Identique à	X
=	$x = y$	$x = y$	$x = 5$
+=	$x += y$	$x = x + y$	$x = 15$
-=	$x -= y$	$x = x - y$	$x = 5$
*=	$x *= y$	$x = x * y$	$x = 50$
/=	$x /= y$	$x = x / y$	$x = 2$
%=	$x \% = y$	$x = x \% y$	$x = 0$

3. LES OPERATEURS RELATIONNELS :

JavaScript	Description
------------	-------------

==	égalité
>	supérieur
>=	Supérieur ou égale
<	Inferieur
<=	inferieur ou égale
!=	différent

4. LES OPERATEURS LOGIQUES :

JavaScript	Description
&&	Et Logique
	Ou logique
!	NON logique

V. ENTREES / SORTIES EN JAVASCRIPT

→ L'opération d'entrée : On peut utiliser soit :

- ☉ la méthode **prompt()** de l'objet **window**,
- ☉ à l'aide d'objets graphiques du **formulaire** HTML.

→ L'opération de sortie : on peut utiliser soit :

- ☉ les méthodes **alert** ou **confirm** de l'objet **window**.
- ☉ les méthode **write** ou **innerHTML** de l'objet document.
- ☉ à l'aide d'objets graphiques du **formulaire** HTML.



les entrées / sorties à l'aide des objets graphiques des formulaires seront traitées ultérieurement. (Les formulaires JavaScript)

VI. STRUCTURES DE CONTROLE :

Structure	Exemple	Description/Note
<pre>if (condition) { instructions1 } else { instructions2 }</pre>	<pre>if (Moy >= 10) { alert("Succès"); } else { alert("Echec"); }</pre>	Les parenthèses sont obligatoires. Les accolades sont facultatives si le traitement comporte une seule instruction.
<pre>while (condition) { instructions }</pre>	<pre>var i = x = 0; while (i < 10) { i++; x += i; }</pre>	Permet de calculer la somme des entiers de 1 à 10.
<pre>do { instructions } while (condition);</pre>	<pre>do { N = prompt("N: ") } while (N <= 0);</pre>	Permet de saisir un entier strictement positif
<pre>for ([initialisation]; [condition]; [expression_finale]){ insructions }</pre>		

initialisation

Une expression (pouvant être une expression d'affectation) ou une déclaration de variable. On utilise généralement une variable qui agit comme un compteur. Cette expression peut éventuellement déclarer de nouvelles variables avec le mot-clé **var** ou **let**.

condition

Une expression qui est évaluée avant chaque itération de la boucle. Si cette expression est vérifiée, les instructions sont exécutées.

expression finale

Une expression qui est évaluée à la fin de chaque itération. Cela se produit avant l'évaluation de l'expression condition. Cette expression est généralement utilisée pour mettre à jour ou incrémenter le compteur.

```
var f = 1; // Un script qui calcule la factorielle de 10
```

```
for (var i = 1; i <= 10; i++) { f *= i; }
```

```
switch (expression) {
```

```
case val1:
```

```
instructions1
```

```
[break;]
```

```
case val2:
```

```
instructions2
```

```
[break;]
```

```
... default:
```

```
val_par_defaut
```

```
}
```

```
switch (val){
```

```
case 1: case 2: alert("1 ou 2");
```

```
break;
```

```
case 3: case 4: alert("3 ou 4");
```

```
break;
```

```
default: alert("Valeur par défaut"); }
```

VII. FONCTIONS EN JAVASCRIPT

```
function nom_fonction(arguments)
```

```
{Instructions;
```

```
return nom_variable ;}
```

Arguments : représentent les paramètres en entrée pour la fonction : les paramètres ne sont pas obligatoires mais les parenthèses oui.

Return nom_variable : permet de retourner le résultat de la fonction vers l'objet appelant.

- La définition de la fonction est faite dans la partie entête alors que son appel est fait dans la partie corps.
- En JavaScript, il existe deux types de fonctions :
 1. Les fonctions prédéfinies en JavaScript
 2. Les fonctions définies par le programmeur selon les besoins de l'application.

VIII. GESTION DES EVENEMENTS EN JAVASCRIPT

Événement	Description
<u>CLICK</u>	Lorsque l'utilisateur clique sur un bouton, un lien ou tout autre élément.
<u>FOCUS</u>	Lorsqu'un élément du formulaire a le focus c à d devient la zone d'entrée active.
<u>CHANGE</u>	Lorsque la valeur d'un champ de formulaire est modifiée.
<u>BLUR</u>	Se produit lorsque l'élément perd le focus, c'est-à-dire que l'utilisateur clique hors de cet élément, celui-ci n'est alors plus sélectionné comme étant l'élément actif.

<u>LOAD</u>	Se produit lorsque le navigateur de l'utilisateur charge la page en cours
<u>UNLOAD</u>	Se produit lorsque le navigateur de l'utilisateur quitte la page en cours
<u>SUBMIT</u>	Se produit lorsque l'utilisateur clique sur le bouton de soumission d'un formulaire (le bouton qui permet d'envoyer le formulaire)

IX. Les fonctions globales prédéfinies du JavaScript :

1. L'objet NUMBER

Fonction	Param	Rôle	Exemples
Number()	-	Convertie le paramètre en un nombre	<code>Number("10");</code> //retourne 10
String()	-	Convertie le paramètre en une chaîne	<code>String(50);</code> //retourne "50"
isNaN()	-	Vérifie si le paramètre n'est pas un nombre. <code>isNaN</code> essaie de convertir le paramètre fourni en un nombre. Si cette conversion n'est pas possible, la fonction renvoie true . Dans les autres cas, elle renvoie false .	<code>isNaN("Ali");</code> // renvoie true <code>isNaN("-12.2e+3");</code> //false! <code>isNaN(12);</code> // renvoie false <code>isNaN("");</code> // renvoie false ! <code>isNaN(" ");</code> // renvoie false ! <code>isNaN(null);</code> // renvoie false !
parseInt()	Chaîne	Analyse la chaîne et renvoie un entier. Si le premier caractère ne peut pas être convertie en un entier, elle renvoie NaN . Les caractères d'espacement au début et à la fin de la chaîne sont ignorés. Seule la première partie qui peut être convertie sera retournée.	<code>parseInt("10")</code> // 10 <code>parseInt(" 10 ")</code> // 10 <code>parseInt("10 25")</code> // 10 <code>parseInt("4 SI")</code> // 4 //retourne <code>parseInt("Bac 2021")</code> // NaN
parseFloat()	Chaîne	Analyse la chaîne et renvoie un réel.	<code>parseFloat("10.25")</code> //10.25 <code>parseFloat("Bac 2021")</code> // NaN

2. Les méthodes & les propriétés de l'objet « window » :

Méthodes	Rôle	Exemples
<code>alert()</code>	Affiche un dialogue d'alerte contenant le texte spécifié.	<code>alert("Bac SI");</code>
<code>prompt()</code>	Pour la saisie d'une entrée textuelle.	<code>nom = prompt("Votre nom:");</code>
<code>confirm()</code>	Renvoie un booléen	<code>ok = confirm("Quitter la page?");</code>

3. Les méthodes & les propriétés de l'objet « String » :

Méthodes	Rôle	Exemples
<code>ch.length</code> (Propriété)	Retourne la longueur de ch	<code>var ch = "4SI"; var x = ch.length; //x=3</code> <code>var y = "4SI-Bac2021".length; //y=11</code>
<code>ch.indexOf(ch2)</code> [<code>ch.lastIndexOf(ch2)</code>]	Retourne la première [dernière] position de ch2 dans ch si elle existe, ou -1 dans le cas contraire.	<code>var ch = "Javascript" ; var p1 = ch.indexOf('a'); //p1=1</code> <code>var p2 = ch.lastIndexOf('a'); //p2=3</code> <code>var p3 = '4SI'.indexOf("@"); //p3= -1</code>
<code>ch.substr(p, n)</code>	Renvoie une sous-chaîne de n caractères à partir de p . Si n est omis, substr renvoie le reste des caractères jusqu'à la fin de ch .	<code>ch = "Bac2021";</code> <code>sch1 = ch.substr(0,3); //sch1="Bac"</code> <code>sch2 = ch.substr(3); //sch2="2021"</code>
<code>ch.substring(a, b)</code>	Renvoie la sous-chaîne entre les indices a et b , b exclu. Si b est omis, substring effectuera l'extraction des caractères jusqu'à la fin de la chaîne.	<code>ch = "Bac2021";</code> <code>sch1 = ch.substring(3,5); //sch1="20"</code> <code>sch2 = ch.substring(3); //sch2="2021"</code>
<code>ch.toLowerCase()</code>	Convertie une chaîne en minuscule	<code>ch = "Bac".toLowerCase()</code> <code>//ch="bac"</code>
<code>ch.toUpperCase()</code>	Convertie une chaîne en majuscule	<code>ch = "Bac".toUpperCase()</code> <code>//ch="BAC"</code>
<code>ch.replace(sch, nouv_sch)</code>	Renvoie une copie de ch en remplaçant la première occurrence de sch par nouv_sch .	<code>ch = "Baccalauréat".replace("a", "A")</code> // <code>ch = "BACcalauréat"</code>

<code>ch.split(sep)</code>	La méthode split permet de diviser une chaîne de caractères à partir d'un séparateur pour fournir un tableau de sous-chaînes. (Utiliser join pour joindre les éléments d'un tableau dans une chaîne)	<pre>var ch = "14 Janvier 2011" var T = ch.split(" ") // T = ["14", "Janvier", "2011"] Ch2 = T.join("*") // ch2 = "14*Janvier*2011"</pre>
----------------------------	--	--

X. Autres instructions

`window.status='message'` ; ➔ Ecrire un message dans la barre d'état du navigateur
`window.close();` ➔ Fermer la page Web

XI. FORMULAIRE EN JAVASCRIPT

1. Zone de texte, Mot de passe, Zone de saisie :

```
<form name="f1">
Nom Prénom:<input type="text" name="nom" id="nom">
```

Nom & Prénom :

➔ Pour lire le contenu d'un champ de texte *nom* on écrit, par exemple :

```
var Nom = document.f1.nom.value;
//Ou bien
let Nom=document.getElementById("nom").value ;
alert("votre Nom est:"+Nom);
```

➔ Pour affecter une valeur à un champ de texte *nom* on écrit, par exemple :

```
document.f1.nom.value= "Ali";
//Ou bien
document.getElementById("nom").value="Ali" ;
```

➔ Pour mettre le curseur dans la zone de texte :

```
document.f1.nom.focus();
//Ou bien
document.getElementById("nom").focus() ;
```

2. Boutons Radio

```
<form name="f1">
<label>Civilité:</label>
<input type="radio" name="R" id="C"
value="Celibataire"><label>Célibataire</label>
<input type="radio" name="R" id="M" value="marie"><label>Marié(e)</label>
```

Civilité : ☒ Célibataire ☐ Marié(e)

→ Pour tester si un bouton radio est coché ou non utiliser la propriété : **checked**

```
if (document.f1.R[0].checked) {alert('Bouton radio 1 coché.')}
else alert('Bouton radio 2 coché.');
```

```
//Ou bien
let liste= document.getElementsByName("R");

if(liste[0].checked){alert('Bouton radio 1 coché.')}
else alert('Bouton radio 2 coché.');
```

→ Pour déterminer le nombre de boutons radios ayant un même nom utiliser la propriété : **length**

```
var nbr= document.f1.R.length;
alert("le nombre des boutons est"+nbr)
//Ou bien
let liste= document.getElementsByName("R");
alert("le nombre des boutons est"+liste.length)
```

→ Pour déterminer la valeur d'un bouton radio utiliser la propriété : **value**

```
alert(document.f1.R[0].value); // renvoie la valeur associée au premier bouton
radio
//Ou bien
let liste= document.getElementsByName("R") ;
alert(liste[0].value) ;
```

3. Cases à cocher

```
4. <form name="f1">
5. <label>Votre langue préférée:</label>
6. <input type="checkbox" name="A" id="A"
   value="arabe"><label>Arabe</label>
7. <input type="checkbox" name="F"
   id="F" value="français"><label>Français</label>
8. <input type="checkbox" name="An" id="An"
   value="anglais"><label>Anglais</label>
9. <input type="checkbox" name="E" id="E"
   value="espanol"><label>Espagnol</label>
10. <input type="button" value="ok" onclick="affichage()>
```

Votre langue préférée : ☐ Arabe ☐ Français ☐ Anglais ☐ Espagnol

→ Pour tester si une case à cocher est cochée utiliser la propriété : **checked**

```
if (document.f1.A.checked) {
    alert("L'Arabe est la langue du Coran.");
}
//Ou bien
if(document.getElementById("A").checked ){
    alert("L'Arabe est la langue du Coran.");
```

}

→ Pour déterminer la valeur associée à une case à cocher utiliser la propriété : *value*

```
if (document.f1.A.checked) {
    alert("votre langue est: "+document.f1.A.value);
}
//Ou bien
if(document.getElementById("A").checked ){
    alert("Votre langue est:"+document.getElementById("A").value);
}
```

Liste de Choix :

```
<form name="f1">
<label>Votre Classe:</label>
<select name="C1" id="C1">
    <option value="4SI">4ème Sciences de l'Informatique </option>
    <option value="4Tec">4ème Technique </option>
</select>
```

Votre classe :

4ème Sciences de l'Informatique ▼

→ Pour déterminer l'indice de l'élément sélectionné utiliser : *selectedIndex* (0 pour le 1^{er} et -1 pour aucun)

```
alert("Vous avez sélectionné l'option num : "+document.f1.C1.selectedIndex);
// Ou bien :
alert("Vous avez sélectionné l'option num : "+document.getElementById("C1").selectedIndex);
```

→ Pour déterminer la valeur ou le texte associés à un élément d'une liste de choix utiliser : *value* et *text*

```
var valeur = document.f1.C1.options[index].value; // index : indice de
l'élément
//Ou bien
let valeur= document.getElementById("C1").options[index].value

//Pour déterminer le texte d'une option dans une liste de choix utiliser :
text
var txt = document.f5.c1.options[index].text; // index : indice de l'élément
//Ou bien
let txt= document.getElementById("C1").options[index].text
```

→ Pour ajouter un élément à la fin de la liste :

```
Taille = document.f1.C1.options.length;
VariableAux = new Option('4 SCientifique', '4Sc') ;
document.f1.C1.options[Taille]=VariableAux ;
//Ou bien
liste= document.getElementById("C1") ;
liste.options[liste.length]= new Option('4 SCientifique', '4Sc') ;
```

→Pour supprimer un élément N° i de la liste :

```
document.f1.C1.options[indice]= null ;
//Ou bien
document.getElementById("C1").options[indice]= null ;
```

Javascript (serie1)

Exercice :1

- a)Afficher un message de bien venu 4 SI avec **alert()**
 b)Ecrire dans une page html en utilisant **document.write** le même message de bienvenu

Exercice :2

Modifier le contenu d'un paragraphe en affichant un message de bienvenu

Exercice :3

- a)Afficher la somme de deux entier en utilisant prompt dans la console
 b)afficher la somme via une alert
 c)afficher la somme dans une paragraphe

Exercice 4: (if et else)

créer une page html et un programme javascript (fonction absolue) qui permet de calculer la valeur absolue d'un entier saisie (**prompt**) et afficher le résultat en utilisant une alert.

Cette page indique

donner un nombre

OK

Annuler

Exercice 5: Manipulation de quelques méthodes prédéfinies **isNaN,Number,toString,indexOf**

Saisir une valeur au clavier et afficher « **conversion impossible** » si elle n'est pas numérique,si non,afficher cette valeur augmenté de 1 et dire si c'est un entier ou réel.Enregistrer la page sous le nom **conversion.html**.

Exemple 1

saisir une valeur

5.3

OK

Annuler

la valeur de b augmenté de 1 = 6.3 c'est un réel

Exemple 2

saisir une valeur

8

OK

Annuler

la valeur de b augmenté de 1 = 9 c'est un entier

Rappel :

```
Sch="script";
p=Ch.indexOf(Sch,0);
```

Cette page indique

4

Javascript (serie1)correction

Exercice 4 (if else valeur absolu)

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
  <script lang="javascript" src="abs.js">

  </script>
</head>
<body onload="absolue()">

</body>
</html>
```

Javascript :abs.js

```
function absolue(){
  A=prompt("donner un nombre");
  if(A<0)
  {alert("la valeur absolue est" + -A)}
  else
    alert("la valeur absolue est" + A)
}
```

Exercice5 :html

```
<!DOCTYPE html>
<html lang="fr">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>Document</title>
  <script lang="javascript" src="ex3s4.js"></script>
</head>
<body onload="réel_entier()">
  <p id="res">&nbsp;</p>
</body>
</html>
```

Javascript :ex3s4.js

```
1 function réel_entier(){
2   x=prompt("donner un nombre");
3   if(isNaN(x)){
4     alert("conversion impossible!");
5   }
6   else//il est numérique
7   {
8     x=Number(x);
9     x=x+1;
10    a=x.toString();
11    if(a.indexOf(".")!=-1)
12    {
13      document.getElementById("res").innerHTML=a+" c'est un nombre";
14    }
15    else
16    {
17      document.getElementById("res").innerHTML=a+" c'est un réel";
18    }
19  }
20 }
21 }
22 }
23 }
```

Serie2 (javascript)

Exercice 1:

Ecrire un script javascript qui permet de résoudre l'équation de premier degré $ax+b=0$ dans l'ensemble des réels distinguer les différents cas a et b sont deux données.

Exemple 1 :

Si l'utilisateur saisi : $a=0$ $b=5$

Le script affichera : **pas de solution**

Exemple 2 :

Si l'utilisateur saisi : $a=0$ $b=0$

Le script affichera : **La solution est IR**

Si ($a=0$) alors
 Si ($b=0$) alors
 Ecrire ("Tout réel est solution de l'équation")
 Fin Si
 Si ($b \neq 0$) alors
 Ecrire ("L'équation n'admet pas de solution")

U/MATH

L'algorithme

Ecrire un algorithme qui permet de résoudre l'équation $ax+b=0$

Algorithme : Solution-équation

Variables : a, b, s Réel

Début

Ecrire ("Veuillez donner deux réels")

Lire (a) 5 $\neq 0$

Lire (b) 3

Si ($a \neq 0$) alors

$s \leftarrow -b / a$

Ecrire ("La solution de l'équation est", s)

$$\begin{aligned} 5x+3 &= 0 \\ x &= -\frac{3}{5} \end{aligned}$$

Si (condition) alors

instruction

Fin Si

Exercice 2:(for et if else)

Écrire un script en JavaScript qui permet de calculer S1 et S2 avec :

$$S1 = 1 + 3 + 5 + \dots + N \quad (\text{ou } (N-1))$$

$$S2 = 2 + 4 + 6 + \dots + (N-1) \quad (\text{ou } N)$$

Exercice 3:

Créer un formulaire et Ecrire un script qui permet de calculer le factoriel d'un nombre saisi dans une zone de texte et l'afficher .

Rappel :

var f = 1; // Un script qui calcule la factorielle de 10

for (var i = 1; i <= 10; i++) { f *= i; }

Factorielle d'un nombre

Nombre

24

b)ajouter un contrôle de saisie pour n'accepter que les valeurs positif (do while)
rappel:

do { instructions }
while (condition);

```
do {
  N = prompt("N: ")
} while (N <= 0);
```

Permet de saisir un entier strictement positif

Correction serie 2

Exercice 1 :	Exercice :2
Function equation() <pre> { var a=prompt("Saisir a:") var b=prompt("Saisir b:") if (a==0){ if (b==0){ alert('IR') } else { alert("Pas de Solution") } } else { alert("x= " + -b/a) } } </pre>	function ex2() <pre> { var N=prompt("N? ") var S1=S2=0; for (i=1;i<=N;i=i+1){ if (i%2==1){ S1=S1+i; }else S2=S2+i; } alert("S1="+S1+" S2="+S2) } </pre>

Exercice :3

```

<!DOCTYPE html>
<html lang="fr">

<head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script lang="javascript" src="factoriel.js">

        </script>
</head>

<body>

    <h1 style="text-align: center;"> Factorielle d'un nombre </h1>
    <hr>
    <br>
    <br>
    <form action="" onsubmit="return false;">

        Nombre <input type="text" name="" id="v"> <br><br>

        <input type="submit" value="calculer factoriel" onclick="factoriel()">
        <p id="resultat">

            </p>
    </form>

</body>
</html>

```

```
function factoriel() {
    val = document.getElementById('v').value
    var f = 1;      // Un script qui calcule la factorielle de 10
    for (var i = 1; i <= val; i++) {    f *= i; }

    document.getElementById('resultat').innerHTML = f
}
```

b)function factoriel()

```
{
    do {
        N=prompt("Saisir un entier positif:")
    } while (N<0);

    var F=1;
    for (i=2; i<=N; i++){
        F=F*i;
    }
    alert(N+"! = "+F)
}
```

Serie 3 (javascript)

Exercice1 : (switch)

Écrire un script en JavaScript qui permet de lire l'année A, et le numéro du mois M puis affiche le nombre de jours correspondant à ce mois. On rappelle que si l'année est bissextile (*elle est divisible par 4: $\rightarrow A \text{ MOD } 4 = 0$*), le mois de février comprend 29 jours.

Exercice 2 :

a) Écrire un script qui permet de n'accepter que des messages composés uniquement de lettres majuscules.

Sur les touches (2, 3, 4, 5, 6, 7, 8 et 9) du clavier d'un téléphone portable, sont inscrites des lettres pour écrire des messages en plus des chiffres.

Par exemples, sur la touche **5** sont inscrites les lettres **J, K et L**.

- Pour taper la lettre **J** on appuie **une seule fois**
- Pour taper la lettre **K** on appuie **deux fois**
- Pour taper la lettre **L** on appuie **trois fois**

1	2 ABC	3 DEF
4 GHI	5 JKL	6 MNO
7 PQRS	8 TUV	9 WXYZ

Écrire un code en JavaScript permettant de déterminer et afficher le nombre total d'appuis sur les touches du clavier d'un téléphone portable pour saisir un mot **M** donné, qui est composé uniquement par des lettres majuscules.

Indication : La figure ci-contre donne la répartition des lettres sur les touches du clavier d'un téléphone portable.

EXEMPLE : Si l'utilisateur saisit **M= "BO"**

Le programme affichera : **5**

(En effet, pour écrire "**B**" on appuie 2 fois et pour écrire "**O**" on appuie 3 fois : $2 + 3 = 5$).

Exercice 3

On se propose de créer un site web permettant de traiter deux problèmes sur les nombres entiers à savoir la somme des diviseurs et les nombres amis.

a. Créer la page **Somme_div.Html** qui comporte le formulaire suivant :

Le clic sur le bouton "**Calculer**" fait appel à une fonction JavaScript permettant de :

- Vérifier la saisie de la valeur de **N** ($N > 1$). Si la condition n'est pas vérifiée, le message "**Veillez saisir un entier strictement supérieur à 1**" sera affiché.
- Calculer la somme des diviseurs de l'entier **N** saisi et l'afficher dans la zone appropriée.

b. Créer la page **Amis.Html** qui comporte le formulaire suivant :

Liste des couples de nombres amis

B_inf =

B_sup =

Le clic sur le bouton "Afficher" fait appel à une fonction JavaScript permettant de :

- Vérifier la saisie des valeurs de "B_inf" et "B_sup" ($200 \leq B_inf < B_sup \leq 300$). Si la condition n'est pas vérifiée, le message "Saisie non valide" sera affiché.
- Afficher les couples de nombres amis dans l'intervalle $[B_inf, B_sup]$, sachant que deux entiers m et n sont dits amis si la somme des diviseurs de m , sauf lui-même, est égale à n et la somme des diviseurs de n , sauf lui-même, est égale à m .
- Modifier le code pour que l'affichage du résultat soit dans un paragraphe.

Syntaxe	Rôle
.....	recupérer le contenu d'une zone de texte
.....	affecter une valeur à une zone de texte

Exercice 4:

Le sujet consiste à développer un site Web contenant un formulaire permettant à un élève de répondre à un test pour évaluer ses connaissances en algorithmique.

- Créer une page web intitulée **Exam.html** contenant le formulaire suivant :
- L'élève répond au test puis valide en cliquant sur le bouton **Envoyer**. L'appui sur le bouton **Envoyer** fait appel à une fonction JavaScript intitulée **calculer()** qui permet de vérifier les champs CIN, Nom et Prénom (CIN doit être composé de 8 chiffres, Nom et Prénom doivent être non vide) ensuite de calculer et d'afficher la note obtenue par l'élève.

N.B : La note de l'élève est la somme des points obtenus dans le test, sachant que les réponses correctes sont :

- Exercice 1 : "+, -, *, /"
- Exercice 2 : "MOD"
- Exercice 3 : Somme $\leftarrow a+b$ Lire(a,b) Ecrire(Somme)

Identification

CIN Nom Prénom

T E S T

Exercice 1 (4 points)

Quels sont les opérateurs qui peuvent être utilisés avec les réels ?

AND, OR, NOT ▼
 AND, OR, NOT
 +, -, *, /
 DIV, MOD

Exercice 2 (4 points)

Quel est l'opérateur qui peut être utilisé dans l'expression suivante :

19 3 = 1

☐ DIV ☐ AND ☐ MOD

Exercice 3 (12 points : 4+4+4)

Numéroter les étapes de l'algorithme suivant :

Somme <-- a + b

Lire (a,b)

4 Fin calcul

Ecrire (Somme)

0 Début calcul

Envoyer



Retenons 2

Syntaxe	Rôle
.....	Retourne l'état d'un élément de la liste (sélectionné ou non)
.....	Retourne la valeur de l'élément sélectionné
.....	Retourne l'indice de l'élément sélectionné
.....	Retourne le nombre d'éléments de la liste



Retenons 3

Syntaxe	Rôle
.....	Retourne l'état d'un bouton radio (coché ou non)
.....	Retourne la valeur d'un bouton radio

Exercice 5:

Créer la page "**evaluation.html**" permettant à un testeur d'évaluer un modèle de voiture en attribuant une note à chaque critère d'évaluation, via le formulaire suivant :

Evaluation d'un modèle :

N° Permis : Modèle testé : Choisir un modèle ▼
 WALLIS IRIS
 WALLIS 619
 WALLIS 216

Notes attribuées :

Sécurité : Conduite : Confort :

Je ne suis pas un robot : ☐

Le clic sur le bouton "Valider" fait appel à une fonction JavaScript intitulée "verif" permettant de s'assurer de la validité des champs du formulaire tout en respectant les contrôles ci-dessous :

Champ	Contrôle
N° Permis	Une chaîne de 8 caractères respectant le format suivant : xx/xxxxxx (où chaque x représente un chiffre).
Modèle testé	La sélection d'un modèle est obligatoire.
Sécurité	Un entier entre 1 et 5.
Conduite	Un entier entre 1 et 5.
Confort	Un entier entre 1 et 5.
Je ne suis pas un robot	La sélection de la case à cocher est obligatoire.

Retenons 4

Syntaxe	Rôle
.....	Retourne l'état d'une case à cocher (cochée ou non)
.....	Rétourne la valeur d'une case à cocher

Exercice

Créer la page "**enregistrement.html**" permettant d'ajouter un testeur à la base de données via le formulaire suivant :

Enregistrement d'un testeur :

N° Permis :

Nom :

Prénom :

Genre : ☐ Féminin ☐ Masculin

Sachant que le clic sur le bouton "**Ajouter**" fait appel à une fonction JavaScript intitulée "**verif1**" permettant de s'assurer de la validité des champs du formulaire tout en respectant les contrôles suivants :

Champ	Contrôle
N° Permis	Une chaîne de 8 caractères respectant le format suivant : xx/xxxxxx (où chaque x représente un chiffre).
Nom	Une chaîne alphabétique ayant une longueur comprise entre 3 et 20.
Prénom	Une chaîne alphabétique ayant une longueur comprise entre 3 et 20.
Genre	La sélection d'un genre est obligatoire.

Exercice

Développer une fonction Javascript **Espaces(ch)** permettant de retourner une chaîne qui résulte de l'application des traitements ci-dessous sur la chaîne **ch** saisie dans une zone de texte et passée en paramètre :

- Supprimer les espaces de début et de fin,
- Supprimer les espaces superflus afin de garder un seul espace entre deux mots consécutifs,
- Remplacer la première lettre de chaque mot par son équivalent majuscule.

Enregistrer votre page sous le nom **espaces.html**

Correction serie 3

Exercice 1

```
function ex1()
{
    var M=Number(prompt("Mois? "))
    switch (M) {
        case 1:case 3:case 5:
        case 7:case 8:case 10:
        case 12: alert("Le mois numéro "+M+" comprend 31 jours")
                break;

        case 4:case 6:
        case 9:case 11: alert("Le mois numéro "+M+" comprend 30 jours")
                break;

        case 2: A=prompt("Année ?")
                if (A%4==0) alert("Février en "+A+"comprend 29 jours")
                else alert("Février en "+A+" comprend 28 jours");
                break;
        default: alert("Numéro du mois non valide")
    }
}
```

Exercice2

a) <pre>function Ex7(){ do{ msg=prompt("Message? :"); }while (!LettresMajus(msg)); function LettresMajus(Ch){ var Valide=true for (var i=0; i<Ch.length; i++){ if(Ch.charAt(i)<'A' Ch.charAt(i)>'Z'){ Valide=false; } } return Valide } }</pre>	<pre>function Ex7(){ do{ msg=prompt("Message? :"); }while (!LettresMajus(msg)); var a=0; for (i=0; i<msg.length; i++){ switch(msg.charAt(i)){ case "A":case "D":case "G": case "J":case "M":case "P": case "T":case "W": a=a+1; break; case "B":case "E":case "H": case "K":case "N":case "Q": case "U":case "X": a=a+2; break; case "C":case "F":case "I": case "L":case "O":case "R": case "V":case "Y": a=a+3; break; case "S":case "Z": a=a+4; break; } } alert(a) }</pre>	<pre>function LettresMajus(Ch){ var Valide=true for (var i=0; i<Ch.length; i++){ if(Ch.charAt(i)<'A' Ch.charAt(i)>'Z'){ Valide=false; } } return Valide }</pre>
--	--	--

Exercice 3 :

.....

.....

.....

.....

.....

[illegible]

Exercice 4 :

.....

.....

.....

.....

A series of horizontal dotted lines for writing.

Exercice 5 :.....

[illegible]

A series of horizontal dotted lines spanning the width of the page, intended as a guide for handwriting practice.

Serie 4

Date (javascript)

C'est quoi La gestion du temps? }

Il s'agit en fait, en Javascript, du nombre de millisecondes écoulées depuis le 1er janvier 1970 à minuit. Cette manière d'enregistrer la date est très fortement inspirée du système d'horodatage utilisé par les systèmes Unix.

La seule différence entre le système **Unix** et le système **du Javascript**, c'est que ce dernier stocke le nombre de millisecondes, tandis que le premier stocke le nombre de secondes. Dans les deux cas, ce nombre s'appelle un **timestamp**. }

Ce nombre ne nous sert vraiment qu'à peu de choses à nous, développeurs, car nous allons utiliser l'objet Date qui va s'occuper de faire tous les calculs nécessaires pour obtenir la date ou l'heure à partir de n'importe quel timestamp

L'objet Date ?

Les méthodes et les propriétés de l'objet date :

L'objet Date nous fournit un grand nombre de méthodes pour lire ou écrire une date. } Le constructeur : Commençons par le constructeur ! Ce dernier prend en paramètre de nombreux arguments et s'utilise de différentes manières. Voici les quatre manières de l'utiliser :

<code>new Date();</code>	Instancie un objet Date dont la date est fixée à celle de l'instant même de l'instanciation.
<code>new Date(timestamp);</code>	Permet de spécifier le timestamp à utiliser pour notre instanciation.
<code>new Date(dateString);</code>	Prend une chaîne de caractères qui doit être interprétable par la méthode <code>parse()</code> .
<code>new Date(année, mois, jour [, heure, minutes, secondes, millisecondes]);</code>	Permet de spécifier manuellement chaque composant qui constitue une date.

..Pour créer un nouvel objet de type Date

Exemples :

<pre>t=new Date(); alert(t);</pre> <i>contient la date et l'heure courantes.</i>	Cette page indique Mon Nov 14 2022 13:52:27 GMT+0100 (heure normale d'Europe centrale) OK
---	--

<pre>const t = new Date(2023,04,04,18,18,54,0); //aa,mm,jj,h,m,s,ms alert(t);</pre>	Thu May 04 2023 18:18:54 GMT+0100 (UTC+01:00)
---	--

Les méthodes de l'objet Date: Étant donné que l'objet Date ne possède aucune propriété standard, nous allons directement nous intéresser à ses méthodes qui sont très nombreuses

<code>getFullYear();</code>	Renvoie l'année sur 4 chiffres ;
<code>getMonth();</code>	Renvoie le mois (0 à 11) ;
<code>getDate();</code>	Renvoie le jour du mois (1 à 31) ;
<code>getDay();</code>	Renvoie le jour de la semaine (0 à 6, la semaine commence le dimanche) ;
<code>getHours();</code>	Renvoie l'heure (0 à 23) ;
<code>getMinutes();</code>	Renvoie les minutes (0 à 59) ;
<code>getSeconds();</code>	Renvoie les secondes (0 à 59) ;
<code>getMilliseconds();</code>	Renvoie les millisecondes (0 à 999).
<code>getTime();</code>	Renvoie le timestamp de la date de votre objet ;
<code>setTime();</code>	Permet de modifier la date de votre objet en lui passant en unique paramètre un timestamp.

Exemples :

Méthodes	Rôle	Exemples
<code>getDate()</code>	Renvoie le numéro du jour du mois (1- 31)	<code>var n = t.getDate(); //n=4</code>
<code>getDay()</code>	Renvoie le numéro du jour de la semaine sachant que zero correspond au dimanche et six au samedi.	<code>var n1 = t.getDay(); //n1=4</code>
<code>getFullYear()</code>	Renvoie l'année (4 chiffres)	<code>var y = t.getFullYear(); //y=2023</code>
<code>getHours()</code>	Renvoie l'heure courante (0-23)	<code>var h = t.getHours(); //h=18</code>
<code>getMinutes()</code>	Renvoie le nombre de minutes de l'heure courante. (0-59)	<code>var m=t.getMinutes(); //m=18</code>
<code>getMonth()</code>	Renvoie le numéro du mois courant sachant que 0 correspond à jan et 11 à déc.	<code>var mo=t.getMonth(); //mo=4</code>
<code>getSeconds()</code>	Renvoie le nombre de secondes de l'heure courante. (0-59)	<code>var s=t.getSeconds() //s=54</code>
<code>toLocaleDateString()</code>	Renvoie une chaîne qui contient la partie date en utilisant les conventions locales.	<code>var lo= t.toLocaleDateString(); //lo="4/4/2023"</code>
<code>toLocaleTimeString()</code>	Renvoie une chaîne qui contient l'heure locale.	<code>n = t.toLocaleTimeString() //n—"18: 18:54"</code>

Exemple 1

```
<script>
//Afficher l'heure actuelle du système
var d = new Date();
var heure = d.getHours() + ":" + d.getMinutes() + ":" + d.getSeconds();
alert(heure);
</script>
```

Exemple 2

```
<script>
// Exemple d'affichage de gestion d'un timestamp
var myDate = new Date('Sat, 04 May 1991 20:00:00 GMT+02:00');
//Creation d'un objet date avec timestamp comme argument
alert(myDate.getMonth()); // Affiche : 4
// Faites attention les mois commencent par 0
alert(myDate.getDate()); // Affiche le mois : 4
alert(myDate.getDay()); // Affiche le jour : 6
</script>
```

Exercice 1 :

a) créer un script pour afficher la date d'aujourd'hui dans un paragraphe comme indiqué dans la figure.

bonjour nous sommes le 9/11/2022

Exercice 2 : Créer une horloge numérique

04:30:26

Exercice 3

3) Créer la page "Spectacle.html" contenant le formulaire suivant :

Ajout d'un spectacle :

Pièce : Choisir une pièce : Choisir une pièce Le trésor Le péché du succès Le gardien Les soldats de la lune L'éternel retour	Salle : Choisir une salle : Choisir une salle Théâtre municipal Quatrième Art L'Etoile du Nord Espace Artisto	Date du spectacle : / / (JJ MM AAAA)
Annuler		Ajouter

N.B. :

- Le clic sur le bouton **Annuler** permet d'initialiser les champs du formulaire.
- Toutes les fonctions JavaScript devront être développées dans un fichier intitulé "Controle.js".
- Le champ relatif à l'année de présentation du spectacle sera rempli automatiquement par l'année courante **dès le chargement** de la page à travers l'instruction suivante :

new Date().getFullYear()

Le clic sur le bouton **Ajouter** fait appel à :

- a) Une fonction JavaScript permettant de contrôler le remplissage du formulaire en respectant les contraintes suivantes :
- ❖ la sélection d'une pièce est obligatoire,
 - ❖ la sélection d'une salle est obligatoire,
 - ❖ la date du spectacle doit être valide tout en respectant le format indiqué dans le formulaire. Sachant qu'une date est dite valide, si elle vérifie les contraintes décrites ci-dessous.

N° du mois	Valeurs autorisées des jours
1, 3, 5, 7, 8, 10, 12	▪ De 1 à 31
4, 6, 9, 11	▪ De 1 à 30
2	▪ De 1 à 29 dans le cas où l'année est divisible par 4 ▪ De 1 à 28 dans le cas contraire

Serie 4 javascript 'date) correction

Exercice 1 :date

Créer un fichier html qui permet d'afficher la date du jour avec un code en JavaScript au chargement de la page dans une paragraphe sous le format " JJ /MM/AAAA " (utilisez l'évènement onload)

Exemple : **bonjour nous sommes le 17/5/2023**

```
<html lang="fr">
  <head>
    <meta charset="UTF-8">
    <meta http-equiv="X-UA-Compatible" content="IE=edge">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script lang="javascript" src="date auj.js">
  </script>
  </head>
  <body onload="dateauj()">
    <p id="resultat" >bonjour</p>
  </body>
</html>
```

date auj.js

```
function dateauj()
{
  dt=new Date();
  dj="bonjour nous sommes le "+dt.getDate()+"/"
  +(dt.getMonth()+1)+"/"+dt.getFullYear();
  document.getElementById('resultat').innerHTML=dj;
}
```

Exercice 2 :Créer une horloge numérique



```
<html>
  <head>
    <script type="text/javascript">
      function refresh(){
```

```

    var t = 1000; // rafraîchissement en millisecondes

    setTimeout('showDate()',t)
}

function showDate() {
    var date = new Date()
    var h = date.getHours();
    var m = date.getMinutes();
    var s = date.getSeconds();
    if( h < 10 ){ h = '0' + h; }
    if( m < 10 ){ m = '0' + m; }
    if( s < 10 ){ s = '0' + s; }
    var time = h + ':' + m + ':' + s

    document.getElementById('horloge').innerHTML = time;
    refresh();
}

</script>
</head>
<body onload=showDate();>

    <span id='horloge' style="background-color:#1C1C1C;color:silver;font-size:40px;"></span>

</body>
</html>

```

Exercice 3 :

.....

.....

.....

.....

.....

.....

.....

Serie 5

Chaine de caractères

Activité1 : chaine de caractères

<pre> <script lang="javascript"> a="informatique"; x=a.length; alert(x); alert(a.charAt(2)); Ch="javascript"; Sch="script"; p=Ch.indexOf(Sch,0); alert(p); p1=Ch.lastIndexOf("a"); alert(p1); p2=Ch.lastIndexOf("q"); alert(p2); Sch1=Ch.substr(3,3); alert(Sch1); Sch2=Ch.substr(1); alert(Sch2); R=Ch.toUpperCase(); alert(R); X=Ch.charCodeAt(1); alert(X); s=Ch.substring(6); alert(s); </script> </pre>	Cette page indique 12 Cette page indique f Cette page indique 4 Cette page indique 3 Cette page indique -1 Cette page indique asc Cette page indique avascript Cette page indique JAVASCRIPT Cette page indique 97 Cette page indique ript
--	--

Activité2:

ch.replace (ch1, ch2) recherche dans la chaine ch la première occurrence de ch1 et la remplace par ch2 et renvoie une nouvelle chaine. Pour remplacer toutes les occurrences : ch.replace(/ch1/g,"ch2") ;	<pre> ch="good very good"; ch1=ch.replace("good","lucky") ; ch2=ch.replace(/good/g,"lucky"); </pre>
--	---

Exercice 1: substring

Créer une page qui affiche une zone de saisie pour saisir une date sous forme jj/mm/aaaa

puis de Calculer l'Age à partir de cette date.

This page says saisir la date de naissance 26/04/2010 OK Cancel	This page says votre age est 11 OK
---	--

Exercice 2 :onkeyup,onkeypress,substr,checked(bouton radio),fonction avec paramètre

Créer la page suivante contenant un formulaire

nom
 prenom
 mdp
 sexe
 masculin ☐ féminin ☐

Exécution :

nom REGAIEG prenom SLIM mdp ***** sexe masculin ☒ féminin ☐ crer code abandonner	This page says RESL6m OK
---	---

1-permetant de changer le lettres saisie dans la zone du nom en majuscule.

2-de colorer la zone de mot de passe mdp en rouge lors du saisie du mot de passe.

3-de générer un nom d'utilisateur composé

a-des deux première lettres du nom en majuscule,les deux premières lettres du prénom,la longueur du mot de passe mdp+le genre(m ou f)
et de l'afficher via une alert.

Exercice 4 :(charAt() et boucle for ()){ ou do while(}{)}

Saisir un texte au clavier et afficher s'il est composé de lettre alphabétique entièrement (des minuscules ou des majuscules) pour cela créer une fonction alpha() grace à laquelle on saisie ce text et on affiche via des alert s'il est valide ou non.

Exemple :

Cette page indique donner 11m OK Annuler	Cette page indique donner hello OK Annuler
Cette page indique champ invalide OK	Cette page indique champ valide OK

Exercice 5 :Créer ce formulaire :utilisation de la boucle for et indexof(verification d'un formulaire)

Saisir le formulaire suivant et effectuer les vérifications suivantes :

- Le pseudo doit être alphabétique ainsi que l'email.
- Le mot de passe doit contenir au moins une lettre majuscule et des valeurs numériques
- L'email doit être de la forme [ch1@ch2.ch3](#) (ch3 comporte au maximum 3 caractère)
- Tester si l'utilisateur a choisi le genre (masculin ou féminin)

veuillez remplir ce formulaire pour s'inscrire

pseudo
 mail
 mot de passe
 retapper mot de passe
 masculin ☐ féminin ☐

Exercice: conjugaison d'un verbe(substr,length)

a) Créer une page html contenant un formulaire puis écrire un programme javascript qui permet conjuguer un verbe du premier groupe qui sera saisie dans une zone de texte au présent de l'indicatif, en cliquant sur le bouton conjuguer.

Conjugaison verbe du premier groupe	
Verbe conjuguer	Verbe jouer conjuguer je joue tu joues il joue nous jouons vous jouez ils jouent

Fonctions de l'objet String

Méthode

Chaine.length

Chaine.substr(position1, longueur)

Description

Retourne le nombre de caractères de la chaîne

La méthode retourne une sous-chaîne commençant à l'index dont la position est donnée en argument et de la longueur donnée en paramètre.

Serie 5 (corrigé)

Exercice 1 : correction :

```

1 <!DOCTYPE html>
2
3 <head>
4
5 <title>act1</title>
6 </head>
7
8 <body>
9     <p id="dn">je suis né le 02/07/1983</p>
10
11 <script>
12     D=prompt("saisir la date de naissance") ;
13     a=D.substring(6);
14     an=Number(a);
15     ag=2021-an;
16     alert('votre age est'+ag);
17     console.log(ag);
18 </script>
19 </body>
20 </html>

```

Exercice 2 :

Correction :

Html :

```

1 <!DOCTYPE html>
2 <head>
3 <script src="form.js">
4 </script>
5 <title>act3</title>
6 </head>
7 <body>
8 <form method="post" action="" name="f1">
9 <table>
10 <tr><td>nom</td><td><input type="text" id="T1" onkeyup="majus('T1')"></td></tr>
11 <tr><td>prenom</td><td><input type="text" id="T2" onkeyup="majus('T2')"></td></tr>
12 <tr><td>mdp</td><td><input type="password" id="mdp" onkeypress="colorap()"></td></tr>
13 <tr><td>sexe</td><td></td></tr>
14 <tr><td>masculin<input type="radio" name="r1" id="m" value="m"></td><td>feminin<input type="radio" name="r1" id="f" value="f"></td></tr>
15 <tr><td><input type="submit" onClick="codes()" value="crer code" name="s1"></td><td><input type="reset" value="abandonner" name="s2"></td></tr>
16 </table>
17 </form>
18 </body>
19 </html>

```

javascript :form.js

```

function codes()
{
    x=document.getElementById("T1").value;
    var y=document.getElementById("T2").value;
    var z=document.getElementById("mdp").value.length;
    var d="";
    if(document.getElementById("m").checked)
    {d=document.getElementById("m").value ;}
    else
    d=document.getElementById("f").value;
    code=x.substr(0,2)+y.substr(0,2)+z+d;
    alert(code);}
function colorap()
{document.getElementById("mdp").style.backgroundColor = "red";}

function majus(T) {
f=document.getElementById(T).value;
document.getElementById(T).value=f.toUpperCase();
}

```

Question : Modifier le code pour que la zone de mot de passe se colorise en rouge si la longueur est inférieure à 6. Sinon en vert.

Exercice 4 :html

```

1
2 <!DOCTYPE html>
3 <html Lang="fr">
4 <head>
5     <meta charset="UTF-8">
6     <meta http-equiv="X-UA-Compatible" content="IE=edge">
7     <meta name="viewport" content="width=device-width, initial-scale=1.0">
8     <title>Document</title>
9     <script Lang="javascript" src="ex4v1s4.js">
10
11     </script>
12 </head>
13 <body onLoad="alpha()">
14
15 </body>
16 </html>

```

Javascript :ex4v1s4.js

```

1  function alpha()
2  {
3      ch=prompt("donner");
4      let x=true;
5      //var ch=document.getElementById("1").Value;
6      alert(ch);
7      for (i = 0; i < ch.length; i++)
8      {
9          if(ch.charAt(i).toUpperCase()<="A" || ch.charAt(i).toUpperCase()>="Z")
10         {
11             x=false;
12         }
13     }
14 }
15
16 if (x==false)
17 {
18     alert("champ invalide") ;return false;
19 }
20 else
21     alert("champ valide") ;return true;
22 }

```

Exercice 5 :

Exercice :conjugaison

Correction:

```

1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5     <meta charset="UTF-8">
6     <meta http-equiv="X-UA-Compatible" content="IE=edge">
7     <meta name="viewport" content="width=device-width, initial-scale=1.0">
8     <title>Document</title>
9 </head>
10
11 <body>
12
13     <h1 style="text-align: center;"> Conjugaison verbe du premier groupe </h1>
14     <hr>
15     <br>
16     <br>
17     <form action="" onsubmit="return false;">
18
19         Verbe <input type="text" name="" id="v"> <br><br>
20
21         <input type="submit" value="conjuguer" onclick="conjuger()">
22         <p id="resultat">
23
24         </p>
25     </form>
26
27     <script>
28         function conjuger() {
29             verbe = document.getElementById('v').value
30             longueur_radical = verbe.length - 2
31             radical = verbe.substr(0, longueur_radical)
32             resultat = 'je '+radical+'e'+ '<br>'+
33             'tu '+radical+'es'+ '<br>'+
34             'il '+radical+'e'+ '<br>'+
35             'nous '+radical+'ons'+ '<br>'+
36             'vous '+radical+'ez'+ '<br>'+
37             'ils '+radical+'ent'+ '<br>';
38             document.getElementById('resultat').innerHTML = resultat
39         }
40
41     </script>
42
43 </body>
44
45 </html>

```

Application

Créer un dossier « TP02 » dans lequel vous enregistrer :

- } La page « TpJs.html » contenant les formulaires ci-dessous
- } Le fichier « Controle.js » qui doit contenir toutes les fonctions *JavaScript*
- } le fichier « Style.css » qui contient les mises en forme nécessaires

La page « TpJs.html » contient les formulaires suivant :

1.

Inscription

Pseudo: De 5 à 10 caractères

Email:

Mot de passe: Contient des lettres maj

Confirmer mot de passe: Confirmer le mot de pas

Le clic sur le bouton envoyer fait appel à la fonction JavaScript `verif1()` qui permet d'effectuer les contrôle de saisie suivant

Contrôle de saisie	
✓	Le champ pseudo doit contenir entre 5 et 10 caractères
✓	Le champ email est de type email, il est obligatoire
✓	Le champ mot de passe est de type <i>password</i> , il doit contenir au moins une lettre majuscule, une lettre minuscule et un chiffre
✓	Le quatrième champ doit être identique au troisième

2.

Abonnement salle de Sport

Activité sportive:

Age: ☐ Enfant ☐ Adult

Abonnement: ☐ 1 Mois ☐ 3 Mois

Net à payer =

Le clic sur le bouton *calculer* fait appel à la fonction *calculer()* qui permet de calculer le net à payer selon le tableau suivant :

	Enfant		Adulte	
	1 Mois	3 Mois	1 Mois	3 Mois
Gymnastique	40	100	50	140
karaté	30	80	40	100

3.

Calcul IMC

Calculer votre Indice de Masse Corporelle

Taille :

Poids :

Résultat :

Le clic sur le bouton *calculer* fait appel à la fonction **Imc()** qui permet de calculer l'indice de masse corporelle et afficher l'interprétation correspondante dans la zone résultat

IMC = Poids / Taille²

Interprétation :

- Moins de 18.5 : Maigreux
- Entre 18.5 et 25 : Normal
- Entre 25 et 30 : Obèse
- Supérieur à 30 : Obésité sévère

Le fichier « style.css » contient les mises en formes suivantes

Le fichier « style.css » contient les mises en formes suivantes

« Style.css »	
<pre>main{ } label{ font-family: Georgia; font-style: oblique; color: rgb(194, 40, 117); } #res{ color: green; font-family: Verdana; font-style: italic;}</pre>	<pre>/* division contenant un formulaire */ .q1{ width: 40%; height: auto; background-color: azure; border-radius: 10px; border: 2px solid blue; padding: 30px 30px; margin-top: 30px; }</pre>

Le fichier « contrôle Js » contient les fonctions JavaScripts suivantes :

```
function verif1()
{
    ps=document.getElementById('pseudo').value;
    if ( (ps.length<5) || ps.length>10 )
    {
        alert('pseudo invalide')
        return false;
    }
    em=document.getElementById('adr1').value;
    if(em==''){
        alert('Remplir le champ Email!');
        return false;
    }
    pw1=document.getElementById('pwd1').value;
    let chiffre=true;
    let maj=true;
    let min=true;
    for(let i=0;i<pw1.length;i++){
        if( (pw1.charAt(i) >='0') && (pw1.charAt(i)<='9')) {
            chiffre=false;
```

```

}
else
{
if( (pw1.charAt(i) >='A') && (pw1.charAt(i)<='Z' ) ) {
maj=false;
}
else{
if( (pw1.charAt(i) >='a') && (pw1.charAt(i)<='z' ) ){
min=false;
}
}
}
}
if (chiffre){
alert('le mot de passe doit contenir au moins un chiffre');
return false;
}
if(maj){
alert('le mot de passe doit contenir au moins une lettre majuscule');
return false;
}
if(min){
alert('le mot de passe doit contenir au moins une lettre miniscule');
return false;
}
pw2=document.getElementById('pwd2').value;
if(pw1!=pw2){
alert('mot de passe différents');
return false;
}
}
}

```

```

function calculer(){
var net =0;
var s=document.getElementById('sport').value;
if(s=="Gymnastique"){if(document.getElementById('en').checked){
if(document.getElementById('m1').checked){ net=40; }
else{ net=100; }
}
if(document.getElementById('ad').checked){
if(document.getElementById('m1').checked){ net=50; }
else{ net=140; }
}
else{
if(document.getElementById('en').checked){
if(document.getElementById('m1').checked){ net=30; }
else{ net=80; }
}
if(document.getElementById('ad').checked){

```



```
if(document.getElementById('m1').checked){ net=40; }
else{ net=100; }
}
}
}
document.getElementById('prix').value=net;
}
function imc(){
  var p=document.getElementById('poids').value;
  var t=document.getElementById('taille').value;
  var i= p/(t*t);
  alert(i);
  let res="";
  if(i<18.5){
    res="Maigneur";
  }
  else{
    if(i<25){
      res="Normal";
    }
    else{
      if(i<30){ res="Obèse"; }
      else
      {
        res="Obésité sévère";
      }
    }
  }
  document.getElementById('res').innerHTML=res;
}
```

Exemples de vérification en Javascript

Soit le formulaire suivant

```

<form name="f" action="..." onsubmit="return verif()">
  Veuillez saisir une valeur <br>
  <input type="text" name="T1" id="T1" > <br>
  <br>
  choisir une couleur <br>
  <input type="radio" name="c" id="c1" value="red">
Rouge <br>
  <input type="radio" name="c" id="c2" value="green">
Vert <br>
  <input type="radio" name="c" id="c3"
value="yellow"> Jaune <br>
  <br>
  Choisir un billet<br>
  <input type="radio" name="b" id="b1" value="10"> 10
dinars <br>
  <input type="radio" name="b" id="b2" value="20"> 20
dinars <br>
  <br>
  Choisir vos centres d'intérêt<br>
  <input type="checkbox" name="mu" id="mu"
value="Musique"> Musique <br>
  <input type="checkbox" name="th" id="th"
value="Théâtre"> Théâtre <br>
  <input type="checkbox" name="des" id="des"
value="Dessin"> Dessin <br>
  <br>
  Choisir une matière<br>
  <select name="matiere" id="mat">
  <option value=""></option>
  <option value="m">Math</option>
  <option value="ph">Physique</option>
  <option value="an">Anglais</option>
  </select> <br>
  <br>
  <input type="submit" value="Valider">
</form>
<script>
function verif() {
  // A terminer Les vérifications
  // .....
  return true;
}
</script>

```

Veuillez saisir une valeur

choisir une couleur

☐ Rouge

☒ Vert

☐ Jaune

Choisir un billet

☐ 10 dinars

☒ 20 dinars

Choisir vos centres d'intérêt

☒ Musique

☒ Théâtre

☐ Dessin

Choisir une matière

Math ▼

Valider

Quelques traitements sur les zones textes (T1)

```
// La zone texte « T1 » ne doit
pas être vide
ch =
document.getElementById('T1').value
if (ch == "") {
    alert("le champ ne doit pas être
vide");
    return false
}
```

```
// La zone texte « T1 » doit contenir 3 caractères
ch = document.getElementById('T1').value
if (ch.length != 3) {
    alert("le champ doit contenir 3 caractères ");
    return false
}
```

```
// La zone texte « T1 » doit
contenir au moins 3 caractères
ch =
document.getElementById('T1').value
if ( ch.length<3) {
    alert ("le champ doit contenir au
moins 3 caractères ");
    return false
}
```

```
// La zone texte « T1 » doit être numérique
ch = document.getElementById('T1').value
if (isNaN(ch)==true) {
    alert("le champ doit être numériques");
    return false
}
```

```
// La zone texte « T1 » doit
contenir exactement 8 chiffres
ch =
document.getElementById('T1').value
if ( !( ch.length==8 &&
isNaN(ch)==false ) ) {
    alert("le champ doit contenir 8
chiffres")
    return false
}
```

```
// La zone texte « T1 » doit être numérique et >=0
ch = document.getElementById('T1').value
if ( !( isNaN(ch)==false && Number(ch)>0 ) ) {
    alert("le champ doit être numériques supérieur à 0");
    return false
}
```

```
// La zone texte « T1 » doit être
formée que par des lettres
alphabétiques
ch =
document.getElementById('T1').value
for(i=0; i < ch.length ;i++) {
    if (
        !(
            ch.charAt(i).toUpperCase() >= "A"
&&
            ch.charAt(i).toUpperCase() <= "Z"
        )
    )
```

```
// La zone texte « T1 » doit être formée que par des
lettres
alphabétiques ou espace
ch = document.getElementById('T1').value
for(i=0; i < ch.length ;i++) {
    if (
        !(
            ch.charAt(i).toUpperCase() >= "A" &&
            ch.charAt(i).toUpperCase() <= "Z"
            || ch.charAt(i) == " "
        )
    ) {
        alert("le champ doit contenir des lettres "+
```

<pre>{ alert("le champ doit être alphabétiques"); return false }</pre>	<pre>" alphabétiques ou espace "); return false }</pre>
<pre>// La zone texte « T1 » doit contenir le caractère @ ch = document.getElementById('T1').valu e if (ch.indexOf("@")== -1) { alert("le champ doit contenir le caractère @"); return false }</pre>	
Traitement sur le premier caractère de la zone texte « T1 » :	
<pre>// Le premier caractère de la zone texte « //T1 » doit être une //lettre majuscule (A .. Z) ch = document.getElementById('T1').valu e if ((ch.charAt(0) <"A") (ch.charAt(0) >"Z")) { alert("Le premier caractère doit être une lettre majuscule") return false }</pre>	<pre>// Le premier caractère de la zone texte « //T1 » doit être une Lettre //minuscule (a..z) ch = document.getElementById('T1').value if ((ch.charAt(0) <"a") (ch.charAt(0) >"z")) { alert("Le premier caractère doit être une lettre minuscule"); return false }</pre>
<pre>// Le premier caractère de la zone texte « T1 » doit être « + » ou « - » ch = document.getElementById('T1').valu e if (!(ch.charAt(0) == "+" ch.charAt(0) == "-")) { alert("Le premier caractère doit être A ou B "); return false }</pre>	<pre>// Le premier caractère de la zone texte « T1 » doit être un chiffre ch = document.getElementById('T1').value if (isNaN(ch.charAt(0))==true) { alert("Le premier caractère doit être un chiffre"); return false }</pre>

<pre>// Le premier caractère de la zone texte « T1 » doit être une lettre voyelle ch = document.getElementById('T1').value premier_carac = ch.charAt(0). toUpperCase () voyelles = "AEIOUY"; if (voyelles.indexOf(premier_carac) == -1) { alert("Le premier caractère doit être une voyelle"); return false; }</pre>	<pre>// Le premier caractère de la zone texte « T1 » doit être une lettre alphabétique ch = document.getElementById('T1').value if (!(ch.charAt(0).toUpperCase() >= "A" && ch.charAt(0).toUpperCase() <= "Z")) { alert("le premier caractère doit être alphabétiques"); return false }</pre>
Quelques Traitements sur les zones Radio couleur choisir une couleur ☐ Rouge ☒ Vert ☐ Jaune	
<pre>// Couleur obligatoire c1 = document.getElementById('c1') c2 = document.getElementById('c2') c3 = document.getElementById('c3') if (c1.checked == false && c2.checked == false && c3.checked == false) { alert("Il faut choisir une couleur") return false }</pre>	<pre>// Afficher la valeur de la couleur choisie c1 = document.getElementById('c1') c2 = document.getElementById('c2') c3 = document.getElementById('c3') if (c1.checked){ alert("la valeur de la couleur choisie est "+c1.value) } if (c2.checked){ alert("la valeur de la couleur choisie est "+c2.value) } if (c3.checked){ alert("la valeur de la couleur choisie est "+c3.value) }</pre>
Quelques Traitements sur les zones case à cocher centres d'intérêt : Choisir vos centres d'intérêt ☐ Musique ☒ Théâtre ☒ Dessin	
<pre>// Cocher au moins un centre d'intérêt mu = document.getElementById('mu') th = document.getElementById('th') des = document.getElementById('des') if (mu.checked == false &&</pre>	<pre>// Afficher la Liste des valeurs des centres d'intérêt cochés mu = document.getElementById('mu') th = document.getElementById('th') des = document.getElementById('des') msg = '' if (mu.checked){</pre>

```

th.checked == false &&
des.checked == false
) {
  alert("Il faut choisir un centre
d'interet")
  return false
}

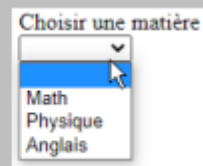
```

```

msg = msg + mu.value + " "
}
if ( th.checked){
  msg = msg + th.value + " "
}
if ( des.checked){
  msg = msg + des.value + " "
}
if(msg != ""){
  alert("les valeurs des centres d'intérêt cochés sont
: " + msg)
}
else{
  alert("Aucun centre d'intérêt coché")
}

```

Quelques Traitements sur les zones listes déroulantes matière :



Soit l'id de la liste est matière

```

// matière obligatoire
liste =
document.getElementById('matiere')
if(liste.options.selectedIndex==0)
{
  alert("la matière est
obligatoire");
  return false
}
Ou bien
liste =
document.getElementById('matiere')
if(liste.value==''){
  alert("la matière est
obligatoire");
  return false
}

```

```

/Afficher une matière
liste = document.getElementById('matiere')
indice_selection = liste.options.selectedIndex
if (indice_selection == 0){
  alert("il faut selectionner une matière");
}
else{
  alert(
    "la matière selectionner est " +
    liste.options[indice_selection].text +
    ". Sa valeur est "+
    liste.options[indice_selection].value +
    ". Son indice est "+indice_selection
  ) ;
}

```



GESTION DES DONNEES : LES BASES DE DONNEES RELATIONNELLES



I. Introduction à la gestion des données :

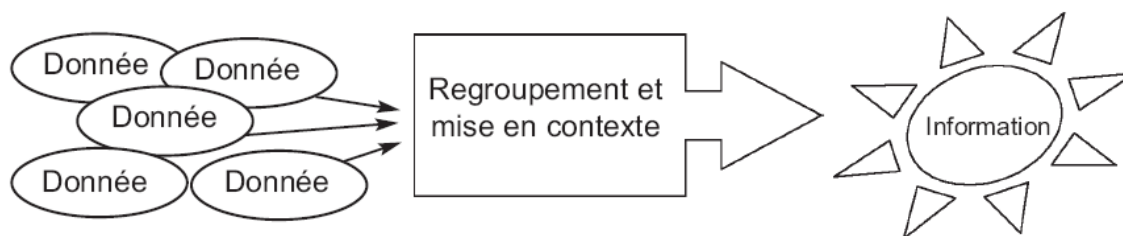
1. Notion de donnée et d'information :

a. Définition :

Les textes d'un traitement de texte, les chiffres d'un tableur, des noms, prénoms et des adresses, etc constituent des **données**.

Une donnée est un élément fondamental pour effectuer une recherche, une étude ou un raisonnement.

L'ensemble de données, rattaché à un contexte et éventuellement transformé donne naissance à une **information**.



b. Eléments Constituant une information :

Les données sont généralement regroupées par entité. Une entité est un ensemble d'attributs.

2. La Persistance :

La façon de mémoriser une donnée à une influence sur la rapidité de la retrouver lorsqu'on a besoin d'elle. cette capacité de mémoriser et de pouvoir retrouver une donnée est appelé : **persistance**.

Persistance = mémorisation + disponibilité

Une donnée persistante est celle qui est mémorisée sur un support quelconque et disponible

La persistance des données peut être donc assurée grâce aux organisations suivantes :

- Organisation papier** : si la gestion des données n'est pas informatisée.
- Organisation en fichiers** : si la gestion des données est informatisée.

c. Organisation Papier :

Les données sont stockées sur supports papiers, tels que des fiches, des registres, des cahiers...etc pour assurer la **persistance**.

Cette organisation présente plusieurs **inconvénients** :



- Le classement se fait sous une seule référence
- La consultation nécessite un certain délai



- Les contraintes de volume et de taille

L'organisation des données en papier consiste à utiliser différents supports papiers pour assurer la persistance. Ces supports peuvent être des fiches, des registres, des cahiers, etc. Cette organisation peut conduire à plusieurs problèmes parmi lesquels :

- **Classement** : un document ne peut être classé que sous une seule référence, limitant ainsi les possibilités de recherche.
- **Consultation** : le document est difficile d'accès, le délai de mise à disposition peut atteindre plusieurs jours ; en outre, pendant sa consultation par une personne, le document n'est plus disponible à d'autres personnes.
- **Contraintes** : de volume et de taille des documents.
- **Délai** : de recherche et de restitution mais aussi de conservation.
- **Fiabilité** du classement et du reclassement en cas d'utilisation d'une archive.
- **Sécurité** : destruction ou détérioration (volontaire ou non), vol.
- **Qualité de la restitution** : elle peut être médiocre (photocopie, fax...)
-

d. Organisation Fichier :

L'organisation des données en fichiers consiste à utiliser des supports informatiques pour assurer la persistance. Ces supports peuvent être des disques durs, des disquettes, des CD, des flashs disques, etc.



Un **fichier** (en anglais: **file**) est un ensemble de données structurées mémorisées sur un support de stockage permanent.

Cette organisation en fichiers possède les **inconvénients** suivants :



- **Lourdeur d'accès aux données**. En pratique, pour chaque accès, même le plus simple, il faudrait écrire un programme.
- **Manque de sécurité**. Si tout programmeur peut accéder directement aux fichiers, il est impossible de garantir la sécurité et l'intégrité des données de ce fichier.
- **Redondance de données**. Etant donné que les fichiers sont généralement conçus par des équipes différentes, il y a un risque qu'un même ensemble de données figurent dans deux ou plusieurs fichiers : c'est la **redondance**. En plus du gaspillage de l'espace disque, il y a un risque d'incohérence dans le cas où la mise à jour n'a pas touché la totalité des copies.



Il faut alors structurer les données sous une forme qui ne présente pas ces insuffisances.

II. Bases données :



1. Définition :

Une base de données (BD) est un ensemble de données structuré relatif à un ou plusieurs domaines du monde réel.

Exemple 1 :

Une Base de Données « **Étudiants** » regroupe **toutes les données** concernant les étudiants (num_cin, nom, prénom, adresse, modules auxquels est inscrit l'étudiant, notes, etc.) et servira à toutes les applications.

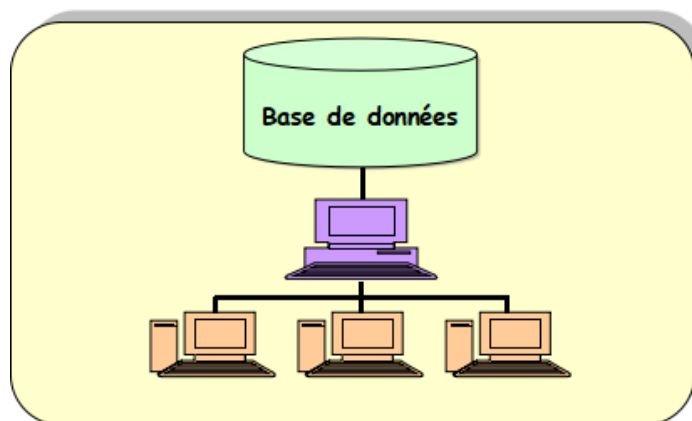
Exemple2 : Une base de données « **Bibliothèque** » regroupe les données concernant les ouvrages, les adhérents qui empruntent les ouvrages, etc. De plus, il y a **des règles de fonctionnement** comme :

- Un adhérent ne peut emprunter en même temps plus de 5 ouvrages.
- La durée maximale d'emprunt est limitée à 10 jour

Lexique :

Français	Anglais	Arabe	Synonymes
Base de données	Data base	قاعدة بيانات	BD

Dans une architecture client/serveur, une base de données est considérée comme une ressource partagée par un ensemble d'applications situées sur les postes clients. La machine qui gère cette base de données est appelée « Serveur de données » ou bien « Serveur » tout simplement.



2. Intérêts de l'utilisation des bases de données :

a. Centralisation :

Les données peuvent être utilisées par plusieurs programmes et plusieurs utilisateurs.

b. Indépendance entre données et programmes :

Dans une BD les données sont décrites indépendamment des programmes. ce qui n'est pas le cas avec les fichiers.

c. Intégration des liaisons entre les données.

Ces relations font partie de la BD et non des programmes. Pas besoin d'un programme pour retrouver les liens entre les données.

d. Intégrité des données

Ce sont des règles assurant la cohérence des données : (unicité, référence et valeur)

- **Unicité des enregistrements.**
- **Interdiction de la suppression des données utilisées par d'autres utilisateurs.**

e. Partage des données (concurrence d'accès) :

différents utilisateurs accèdent en même temps aux mêmes données (Accès multiple simultané)

3. Les Modèles des bases de données :

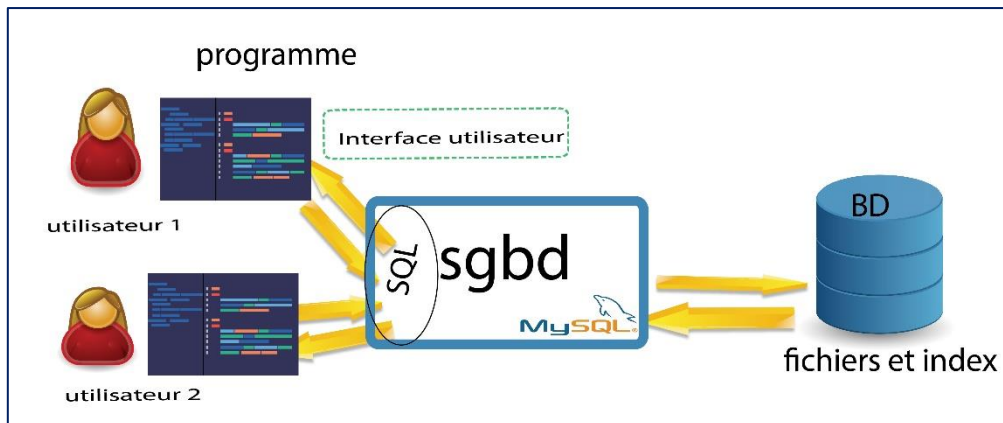
Les modèles des BD se distinguent par la façon selon laquelle les liens entre les données sont représentés on trouve :

- ➔ ***Le modèle Hiérarchique***
- ➔ ***Le modèle Réseau***
- ➔ ***Le modèle Relationnel***
- ➔ ***Le modèle orienté objet***

4. Définition d'un SGBD

Le logiciel qui permet d'interagir avec une base de données s'appelle un système de gestion de base de données (S.G.B.D).

Un SGBD est constitué de deux composantes principales : **un moteur** et **une interface**. Le moteur qui constitue la composante principale d'un SGBD et l'interface qui permet un accès facile aux données.



a. Les fonctions d'un SGBD

Un **SGBD** doit permettre de:

- Décrire les données qui seront stockées,
- **Manipuler ces données** (ajouter, modifier, supprimer des informations),
- Obtenir **des renseignements** à partir de ces données (sélectionner, trier, calculer, etc.),
- Définir **des contraintes d'intégrité sur les données** (contraintes de domaines, d'existence,
- Définir des protections d'accès (mots de passe, autorisations, etc.),
- Résoudre les problèmes d'accès multiples aux données (blocages, interblocages),
- Prévoir des procédures de reprise en cas d'incident (sauvegardes, journaux, etc.).

b. Les principaux SGBD :



c. Types d'utilisateurs d'une BD :

- L'administrateur
- Le programmeur d'application
- L'utilisateur final

III. Structure d'une BD relationnelle :

INFORMATION		
Entité(s)	Attribut(s)	Valeur(s)
C'est la personne, l'objet, le fait pris en considération.  De qui ou de quoi s'agit-il?	Il est nécessaire pour décrire l'entité.  Quelles sont les caractéristiques proposées pour décrire l'entité ?	C'est le contenu ou la valeur de l'attribut.  Quelle est la valeur de chacune des attributs (caractéristiques ou propriété) ?

1. Notion de table

Définition

Une **table** est un ensemble de données relatives à une même **entité**, structurée sous forme d'un tableau (**liste**). une table est composée horizontalement d'un ensemble de **lignes** et verticalement d'un ensemble de **colonnes** : les colonnes décrivent les propriétés relatives au sujet représenté par la table et les lignes correspondent aux occurrences du sujet.

Une table peut être appelée aussi une "**Entité**" ou "**Relation**".

Exemple :

La table Article regroupe les données relatives aux articles commercialisés dans un magasin. Chaque article est décrit par :

- **Code article** : C'est un code attribué de façon unique à chaque article.
- **Désignation article** : C'est le nom courant de l'article.
- **Prix unitaire** : C'est le prix de vente de l'article.
- **Quantité stock** : C'est la quantité actuellement disponible pour un article.

A un moment donné, la table Article peut être représentée comme suit :

Code article	Désignation article	Prix unitaire	Quantité en stock
V10	Vis 50x3	40	2500
V20	Vis 20x2	20	1300
B100	Boulon 90x15	450	100
C60	Crau 60x2	5	5000

Exemple2 : Cas d'une bibliothèque

LIVRES

Code livre	Titre	Auteur	Année	Nbre page

PRETS

Numéro prêt	Date	Durée	Code abonné	Code livre

--	--	--	--	--

Remarques :

- Les données d'une table peuvent être stockées sur un ou plusieurs fichiers.
- Une table peut être considérée comme un ensemble mathématique. Ainsi on pourra faire l'union ou l'intersection de deux tables

2. Notion de colonne :

Définition

Une **colonne** (*champ*) représente une **propriété** élémentaire de l'entité décrite par cette table.

Caractéristiques d'un champ :

- **Un nom** : C'est le nom de la colonne. Il est sous forme de code et il est généralement soumis aux mêmes règles de nommage des variables dans les langages de programmation.
- **Un type de données** : C'est le type de données prises par cette colonne. Les types de données les plus connus sont : numérique, chaîne de caractères (ou texte), date et booléen.
- **Une taille éventuelle** : Pour certains types de données tel que le type numérique ou chaînes de caractères, la taille indique la longueur maximale que peut prendre la colonne.
- **Un indicateur de présence obligatoire** : indique si cette colonne doit être toujours renseignée ou peut être vide dans certains cas. Lorsque la colonne n'est pas renseignée, on dit qu'elle contient une valeur nulle. Il est à noter que la valeur nulle est différente de zéro pour les colonnes de type numérique et de la chaîne vide pour les chaînes de caractères.
- **Une valeur par défaut éventuelle** : Permet d'attribuer une valeur par défaut lorsqu'aucune valeur n'a été attribuée à cette colonne.
- **Une règle éventuelle indiquant les valeurs autorisées** : Dans certains cas, une colonne peut être soumise à certaines règles tel que : les valeurs attribuées à cette colonne doivent être inférieures à une certaine valeur, supérieures à une certaine valeur ou bien comprises entre deux valeurs.

Exemple :

- Nous avons vu dans l'exemple précédent que la table Article regroupe les quatre colonnes suivantes : **code article, désignation article, prix unitaire et quantité en stock.**

Nous allons décrire de façon détaillée chacune de ces colonnes à travers le tableau suivant :

Nom de la table : Article						
Description : Détail des articles commercialisés						
Nom colonne	Description	Type de données	Taille	Obligatoire	Valeur par défaut	Valeurs autorisées
Code_art	Code de l'article	Chaîne de caractères	20	Oui		
Des_art	Désignation de l'article	Chaîne de caractères	50	Oui		
PU	Prix unitaire de l'article	Numérique	8,3	Non		> 0
Qte_stock	Quantité en stock	Numérique	4	Non	0	>= 0

3. Notion de ligne

Définition :

Une **ligne** (*enregistrement, n_uplet*) représente une **occurrence** du sujet représenté par la table

Exemple

Code livre	Titre	Auteur	Année	Nbre-page
.....				
L1005	Base de données	Dan Brown	2004	224

4. Notion de clé primaire

Définition :

La **clé primaire** d'une table **est une colonne**(champ)ou **un ensemble de colonnes** (champs) qui permet d'identifier d'une manière **unique** chaque enregistrement de la table. De ce fait, elle doit être **unique** et **non nul**. Autrement dit, la connaissance de la valeur de la clé primaire, permet de connaître **sans aucune ambiguïté les valeurs des autres colonnes de la table**.

Lexique :

Français	Anglais	Arabe	Synonymes
Clé primaire	Primary key	مفتاح أساسي	Identifiant

Remarque : Chaque table doit comporter une et une seule clé primaire. Pour distinguer une colonne qui fait partie de la clé primaire des autres colonnes, on la souligne.

5. Liens entre les tables

a) Les tables d'une BD sont en **relation** par des **liens** (*association*).

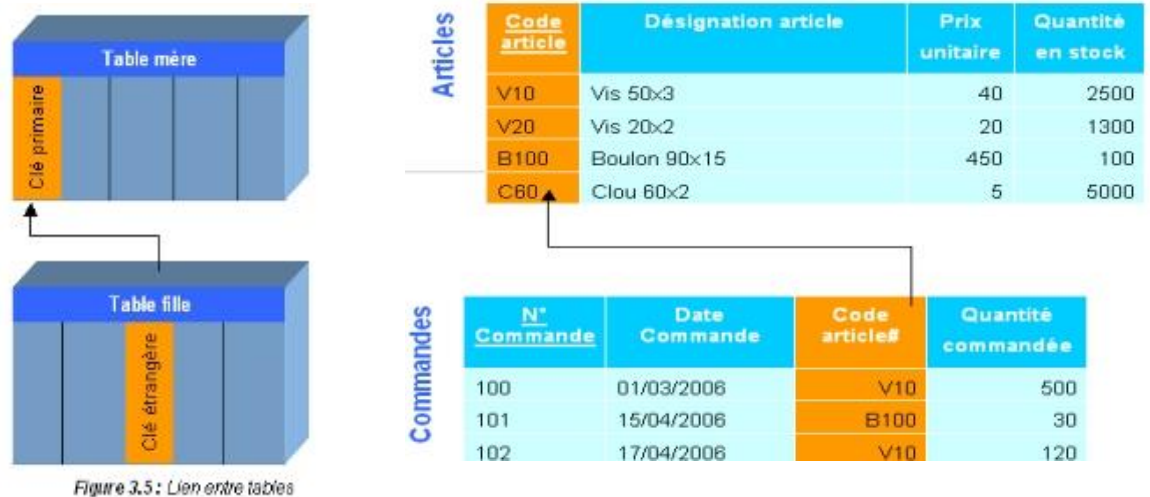
Exemple : pour la BD commande_article, on a :

- Un article article peut être commandé plusieurs fois
- Une commande concerne un et un seul article

Définition :

- ➔ Un **lien** entre 2 tables **A** et **B** se traduit par l'ajout dans la table B d'un nouveau champ correspondant à la clé primaire de la table A. ce champ est appelé **clé étrangère**
- ➔ Dans ce cas A est **une table mère** et B est une **table fille**

Exemple :



Nous avons ajouté au niveau de la table **commande** la clé primaire de la table **Articles**

La table article est la **table mère** et la table commande est la **table fille**.

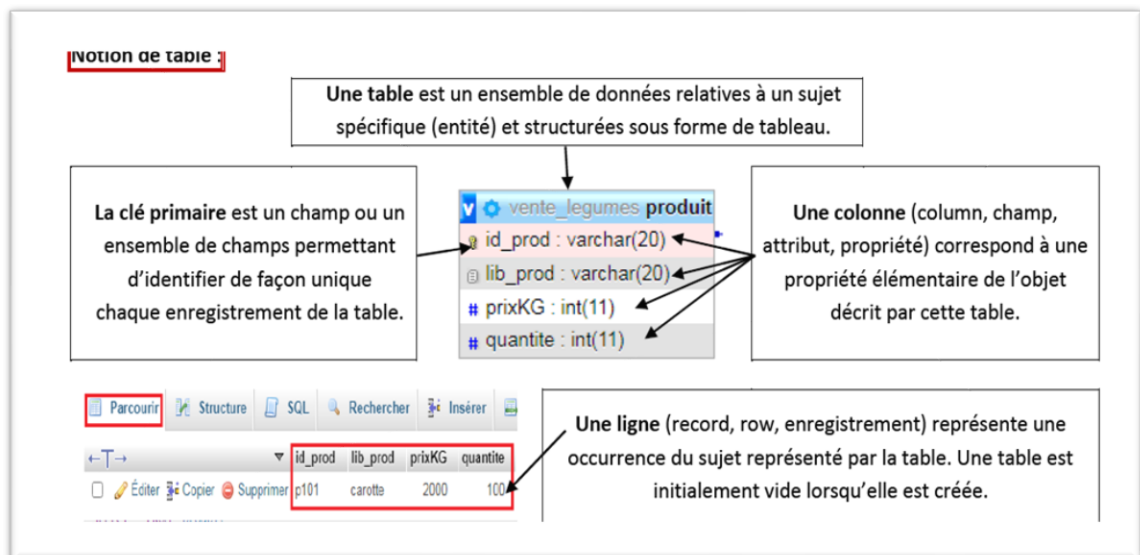
Remarques !!!:

- Il est fortement recommandé que le nom de la colonne qui est une clé étrangère soit identique au nom de la colonne clé primaire à laquelle elle se réfère.
- Pour distinguer une colonne qui fait partie d'une clé étrangère des autres colonnes, on la fait suivre d'un dièse (#).

• Lexique :

Français	Anglais	Arabe	Synonymes
Clé étrangère	Foreign key	مفتاح خارجي	Référence, contrainte d'intégrité référentielle

Retenons :



6. Notion de contrainte d'intégrité :

Définition :

Une **contrainte d'intégrité** est une règle appliquée à **une colonne**(champ)ou à **une table** et qui doit être toujours vérifiée.

→ **Les contraintes de domaine** : ce sont des contraintes appliquées à des colonnes (valide si)

Exemple une moyenne n'est valide que si elle **est comprise entre 0 et 20**

→ **Les contraintes d'intégrité de tables** : permettent d'assurer que chaque table a une clé primaire

Exemple : La table Élève doit avoir une clé primaire, le numéro de carte d'identité par exemple.

→ **Les contraintes d'intégrité référentielle** :

- ⊙ Permettent de s'assurer que les valeurs introduites dans une colonne figurent dans une autre colonne en tant que clé primaire
- ⊙ La suppression d'un enregistrement d'une table mère A utilisé par une table B fille est interdit

Lexique :

Français	Anglais	Arabe	Synonymes
Contrainte d'intégrité	Integrity constraint	قييد	

Exemple : On n'accepte pas le Code article saisi dans une **Commande** qui n'existe pas dans la colonne Code article de la table **Article**.

7. Représentation de la structure d'une BD

La représentation des différentes structures d'une BD est appelée schéma ou modèle.

Cette représentation peut être effectuée selon deux formalismes :

→ La représentation textuelle

La représentation textuelle consiste à décrire les tables, les colonnes et les liens entre les tables en utilisant du texte.

Soient les deux tables A et B composées des attributs a1, a2, a3 et a4 pour la première et b1, b2 et b3 pour la deuxième et dont les clés primaires respectives sont a1 et b1. En supposant que B se réfère à A, la représentation textuelle de ces deux tables se fait de la façon suivante :

A (a1, a2, a3, a4)

B (b1, b2, b3, a1#)

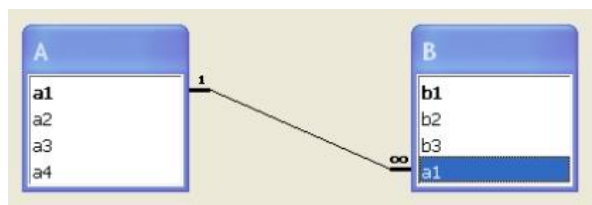
Exemple :

Livre (Code livre, Titre, Auteur, Année, Nbre de page)

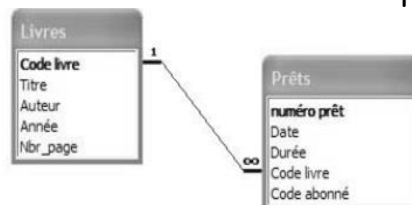
Prêts (Numéro prêt, Date, Durée, Code-abonné, **Code livre #**)

→ La représentation graphique

La représentation graphique consiste à décrire les tables, les colonnes et les liens entre les tables en utilisant des symboles graphiques.



Exemple : Les deux tables décrites ci-dessus seront représentées comme suit :



Les clés primaires sont représentées en **gras** et les clés étrangères à l'aide d'un lien entre les deux tables : le symbole (∞) est placé du côté de la **clé étrangère** et le symbole (**1**) du côté de la **clé primaire référencée**.

La **relation** entre les deux tables est dite de type « **un à plusieurs** » car à une ligne de A peut correspondre plusieurs lignes de B alors qu'à une ligne de B ne peut correspondre qu'une seule ligne de A.

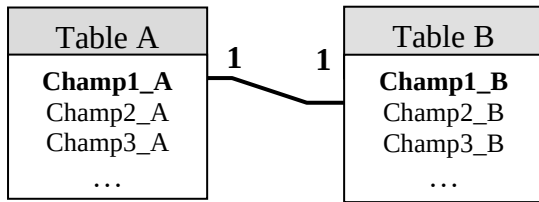
b) Différents types de relations (les cardinalités):

Soit deux tables A et B. On distingue trois types de relations :

✱ **Relation un-à-un (1-1):**



Chaque enregistrement de la table A ne peut correspondre qu'à un enregistrement de la table B et inversement.



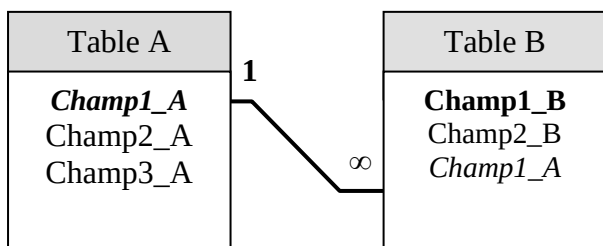
Dans une relation 1-1 les deux champs Liés sont deux clés primaires.

Exemples :

- Un directeur dirige un seul club
- Un club est dirigé par un seul directeur

* Relation un- à- plusieurs (1-N):

Dans ce type chaque enregistrement de la table contenant la clé primaire peut être associé à plusieurs enregistrements de la deuxième table, mais chaque enregistrement de la deuxième table n'est associé qu'à un seul enregistrement de la première table.



- Champ1_A de la table A est une clé primaire
- Champ1_A de la table B est appelé **clé étrangère**

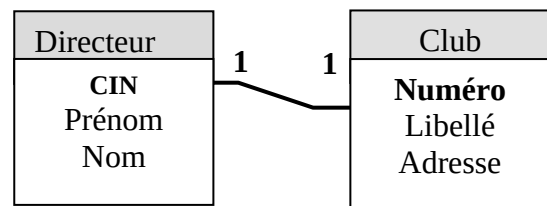
Exemples :

Un club lui adèrent un ou plusieurs abonnés

Un abonné est adère à un seul club

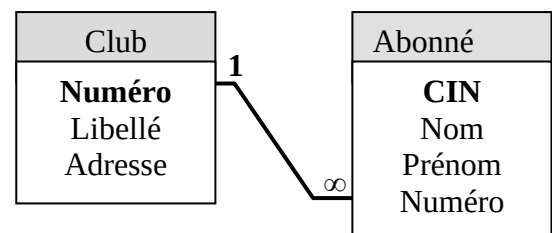
- Club(Numéro, Libellé, Adresse)
- Abonnés(CIN, Nom, Prénom, Numéro)

Exemple: gestion d'un groupe de club



Une personne possède une seule CIN
Une CIN est possédée par une seule personne

Exemple:



Une personne possède plusieurs voitures
Une voiture est possédée par une seule personne

Établir les conditions préalables:

- Il existe une clé primaire unique dans la table du côté 1.
- Le champ commun dans la table associée (côté plusieurs ∞) est de **même type** et de **même taille** que la **clé primaire**.
- Une **clé étrangère** est un champ externe par rapport à la table en cours, importé d'une autre table.
- une **clé primaire** est la donnée qui permet d'identifier de manière unique un enregistrement dans une table. Une clé primaire peut être composée d'une ou de plusieurs colonnes de la table

✱ Relation plusieurs-à-plusieurs (N-M):

Plusieurs enregistrements de la table A peut être mis en correspondance avec plusieurs enregistrements de

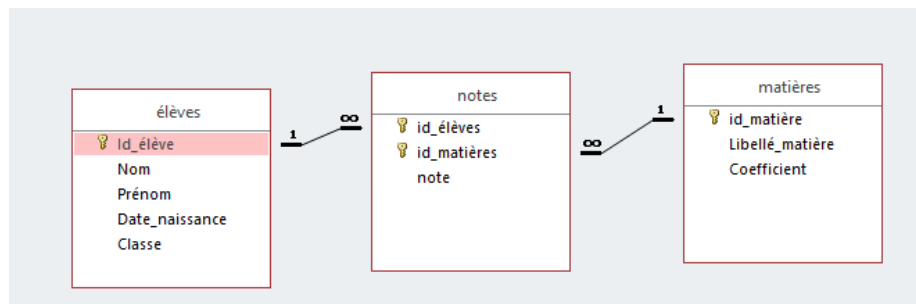
la table B et inversement. Cette relation est traduire en Access par deux relations **1-N**

Activité: Déterminer les relations entre les tables de la BD Gestion de notes et indiquer leurs cardinalités.

C Comment identifier une relation :

Pour savoir le type de relation entre deux tables, il faut poser la question suivante pour les deux tables :

Pour 1 enregistrement de cette table, combien peut-on avoir de correspondance dans l'autre table ?



On ne peut définir ce type qu'après définition d'une troisième table A_B (table de **jonction**) dont la clé primaire est formée de deux clés des tables A et B.

→ Dans une relation N-M on trouve deux relations 1-N avec une troisième table.

Ajouter une table supplémentaire, contenant uniquement les champs associés, pour servir de **jointure** entre les deux tables.

IV. Création et modification de la structure de base de données relationnelle :

Introduction :

Il existe deux modes pour créer une base de données :

➔ Mode assisté

➔ Mode commande

1. Mode assisté :

- Le **mode assisté** permet de créer les éléments de la base de données à l'aide des **assistants graphiques**.
- L'assistant graphique est une **interface** composée d'une suite de fenêtres où chacune représente une étape du processus de création.

Le mode assisté est un moyen qui *facilite* la création des éléments d'une base de données.

Création de la base de données

Création d'une base de données

Nom de base de données:

Filtres

Contenant le mot :

Base de données Interclassement Action

Console de requêtes SQL

phpMyAdmin

Server: localhost.3306 » Base de données: montuto_bdd

Table	Action	Lignes	Type	Interclassement	Taille	Perte
wp_commentmeta	Afficher Structure Rechercher Insérer Vider Supprimer	0	MyISAM	utf8mb4_unicode_ci	4 Kio	-
wp_comments	Afficher Structure Rechercher Insérer Vider Supprimer	1	MyISAM	utf8mb4_unicode_ci	7,3 Kio	-
wp_links	Afficher Structure Rechercher Insérer Vider Supprimer	0	MyISAM	utf8mb4_unicode_ci	1 Kio	-
wp_options	Afficher Structure Rechercher Insérer Vider Supprimer	128	MyISAM	utf8mb4_unicode_ci	269,9 Kio	-
wp_postmeta	Afficher Structure Rechercher Insérer Vider Supprimer	1	MyISAM	utf8mb4_unicode_ci	10,1 Kio	-
wp_posts	Afficher Structure Rechercher Insérer Vider Supprimer	3	MyISAM	utf8mb4_unicode_ci	13,7 Kio	-
wp_termmeta	Afficher Structure Rechercher Insérer Vider Supprimer	0	MyISAM	utf8mb4_unicode_ci	4 Kio	-
wp_terms	Afficher Structure Rechercher Insérer Vider Supprimer	1	MyISAM	utf8mb4_unicode_ci	13 Kio	-
wp_term_relationships	Afficher Structure Rechercher Insérer Vider Supprimer	1	MyISAM	utf8mb4_unicode_ci	3 Kio	-
wp_term_taxonomy	Afficher Structure Rechercher Insérer Vider Supprimer	1	MyISAM	utf8mb4_unicode_ci	4 Kio	-
wp_usermeta	Afficher Structure Rechercher Insérer Vider Supprimer	15	MyISAM	utf8mb4_unicode_ci	11 Kio	-
wp_users	Afficher Structure Rechercher Insérer Vider Supprimer	1	MyISAM	utf8mb4_unicode_ci	6,1 Kio	-
Somme		152	InnoDB	latin1_swedish_ci	347 Kio	0

Version imprimable Dictionnaire de données

Nouvelle table

Nom: Nombre de colonnes:

2. Mode Commande :



Dans le mode commande, on utilise le langage **SQL (Structured Query Language)** est un langage pour les bases de données relationnelles permet la :

- ✓ Définition de données (**LDD**)

Ce sont des commandes qui permettent de **créer, modifier** et supprimer les différentes **structures** de la BD.

- ✓ Manipulation de données (**LMD**)

Ce sont des commandes qui permettent de manipuler **le contenu** de la BD, c'est à dire d'insérer, de modifier, de consulter ou de supprimer **des lignes** dans les tables de la BD. ...

- ✓ Contrôle de données (**LCD**)

Ce sont des commandes qui permettent de contrôler l'utilisation de la BD (un sous-ensemble de SQL pour **contrôler l'accès aux données** d'une base de données.).

A. Langage de définition de données : LDD

Il permet de créer, modifier et supprimer des bases de données relationnelle, des tables, les associations, et les contraintes.

→ Création de la base de données :

```
CREATE DATABASE "Nom_de_la_bd" ;
```

→ La création de tables :

Le langage de définition de données (**LDD**) permet de créer des tables grâce au mot clé **CREATE TABLE**.

```
CREATE TABLE nom_table  
(définition_ colonne | définition_ contrainte, ... )
```

Remarques :

1. Le nom du table doit être unique dans la BD.
2. Une table doit contenir au moins une colonne.

- **La clause « définition_colonne »** permet de préciser les caractéristiques d'une colonne. Elle a la syntaxe suivante :

Nom_colonne **TYPE** [[**NOT**] **NULL**] [**DEFAULT** valeur] [*contrainte_colonne*]

Les Principaux types sont :

INT (n) Numérique à n chiffres

DECIMAL (n, m) Numérique à n chiffres dont m décimales

VARCHAR(n) Chaîne de caractères de longueur variable dont la taille maximale est n

DATE Date et/ou heure

L'option **NULL** veut dire que la colonne n'est pas obligatoire.

L'option **NOT NULL** veut dire que la colonne est obligatoire.

L'option **DEFAULT** permet d'attribuer une valeur par défaut à cette colonne lorsque aucune valeur ne lui a été affectée.

Cette option ne peut pas être indiquée lorsque la colonne est obligatoire (**NOT NULL**).

L'option « **contrainte_colonne** » permet de préciser une contrainte d'intégrité relative à la colonne. Cette contrainte peut être une contrainte de clé primaire, de clé étrangère ou de valeurs.

La syntaxe correspondante est la suivante :

```
CONSTRAINT contrainte
{ PRIMARY KEY } (colonne1, colonne2, ...)
| FOREIGN KEY (colonne1, colonne2, ...)
  REFERENCES nom_table [(colonne1, colonne2, ...)]
  [ON DELETE CASCADE]
| CHECK (condition)}
```

CONSTRAINT est un mot optionnel et sert à attribuer un nom à la contrainte. Le paramètre *contraint* sert en tant qu'identificateur.

PRIMARY KEY spécifie que la colonne est utilisée comme clé primaire.

FOREIGN KEY définit une contrainte d'intégrité référentielle relative à plusieurs colonnes.

REFERENCES définit une contrainte d'intégrité référentielle. Le nom de la table précisé après le mot-clé **REFERENCES** est celui de la table mère. Le nom de la colonne est celui de la colonne vers laquelle on se réfère et il ne doit être précisé que lorsqu'il est différent du nom de la colonne courante.

ON DELETE CASCADE est une option qui permet de maintenir l'intégrité référentielle en supprimant automatiquement les valeurs d'une clé étrangère dépendant d'une valeur d'une clé primaire si cette dernière est supprimée.

CHECK est un mot clé associé à une condition qui doit être vérifiée pour chaque valeur insérée.

- La clause « **définition_contrainte** » de la commande **CREATE TABLE** permet de définir une contrainte d'intégrité au niveau de la table. Elle doit être utilisée lorsque la contrainte ne s'applique pas à une seule colonne. Elle a la syntaxe suivante :

[

CREATE TABLE **Nom_table** (*définition_colonne*|*définition_contraintes*,...)

✓ *Définition_colonne* :

Nomcolonne **Type_donnée** [*contrainte*][**CONSTRAINT** *contrainte*],...);

✓ *Contraintes d'intégrité* :

Les contraintes d'intégrité doivent être exprimées dès la création de la table grâce aux mots clés suivants:

<i>Clause</i>	<i>Rôle</i>
DEFAULT valeur	Valeur par défaut
PRIMARY KEY	Indiquer la clé primaire
NULL	La valeur de la colonne n'est pas obligatoire
NOT NULL	La valeur de la colonne est obligatoire
CHECK (condition)	Faire un test sur un champ
UNIQUE	Vérifier que la valeur saisie pour un champ ne se répète pas
REFERENCES table(colonne_referée)	Indique la clé étrangère.

Exemple de création de table avec contraintes :

CREATE TABLE clients(

CIN INT(8) PRIMARY KEY,

Nom varchar(30) NOT NULL,

Prenom varchar(30) CONSTRAINT nl_pre NOT NULL,

Age int(2) check (age < 100),

Email varchar(50) NOT NULL check (Email LIKE "%@%"),

Adr int(3) UNIQUE REFERENCES Adresse(code_adr)

);

Application :

❖ Exemple 1:

Représentation textuelle :

```
Equipe (code_equ, Nom_equ, date_cre) ;
Joueur (Num_j, Nom_j, Prénom_j, date_j, code_equ#)
Arbitre (Num_arb, Nom_arb, Prénom_arb)
Match (Num_match, date_match, heure_match, res_match, Num_arb#)
Participe (Num_j#, Num_match#)
```

Ecrire les commandes nécessaires pour créer ces tables.

➤ Création de la table Equipe :

```
Create Table Equipe
(code_equ Varchar(10) primary Key,
Nom_equ Varchar(20) Not null,
Date_cre Date Not null) ;
```

➤ Création de la table Joueur :

```
Create Table joueur
( Num_j Varchar (5) ,
```

```
Nom_j    Varchar(20) Not null,
Prénom_j Varchar(20) Not null,
Date_j   Date Not null,
Code_equ Varchar(10) references Equipe [on delete cascade] ;
```

On ajoute on delete cascade pour maintenir l'intégrité référentiel en supprimant automatiquement les valeurs d'une table qui en relation avec d'autre table.

- Création de la table Arbitre :

Create Table Arbitre

```
( Num_arb    Varchar (5) Primary Key,
  Nom_arb    Varchar(30) Not null,
  Prénom_arb Varchar(30) Not null ) ;
```

- Création de la table Match :

Create Table Match

```
( Num_match  Int(2) Primary Key,
  Date_match Date Not null,
  Heure_match Date Not null,
  Res_match   Varchar (5) Not null,
  Num_arb     Varchar (5) references Arbitre [on delete cascade] ) ;
```

- Création de la table Participe :

Create Table Participe

```
( Num_j    Varchar (5) references joueur [on delete cascade] ,
  Num_match Int(2) references match [on delete cascade] ,
  Primary Key (Num_j, Num_match));
```

❖ Exemple 2:

Représentation textuelle :

Article (Code_art, des_art, PU, qte_stock)

Client (Code_cli, nom_cli, adr_cli, tel_cli)

Commande (Num_comm, date_comm, code_cli#)

Detail_commande (Num_ligne, num_comm#, code_art#, qte_comm)

- Création de la table Article :

Create Table Article

```
(code-art Varchar(10) Primary Key,
Des-art Varchar(50) Not Null,
PU Decimal(8,3) ,
Qte-Stock int(5) Default 0 check (Qte-Stock >=0)) ;
```

- Création de la table Client :

Create Table client

```
(Code-cl Varchar(10) Primary Key,
Nom-cl Varchar(30) Not Null,
Adr-cl Varchar(50) Not Null,
Tel-cl int(10)) ;
```

- Création de la table Commande :

Create Table Commande

```
(Num-cmd Varchar(20) Primary Key,
Date-cmd Date Not Null,
Code-cl Varchar(10) Références Client)) ;
```


- Création de la table Détail Commande :
- (Num-cmd Varchar(20) **Références** Commande,
 - Num-ligne int (4),
 - Code-art Varchar(10) **Références** article,
 - Qte-cmd int(5) **check** (Qte-cmd >0),
 - Primary Key**(Num-cmd, Num-ligne)) ;

✓ **Nommer une contrainte :**

Il est possible de donner un nom à une contrainte grâce au mot clé **CONSTRAINT** suivi du nom que l'on donne à la contrainte.

Définition_contraintes :

Syntaxe :

[CONSTRAINT contrainte]

{PRIMARY KEY (colonne1, colonne2,...)}

/FOREIGN KEY (colonne1, colonne2,...)

REFERENCES nom_table[(colonne1,colonne2,...)]

[ON DELETE CASCADE]

/CHECK (condition)}

Exemple :

```
CREATE TABLE clients(
    CIN INT(8),
    Nom varchar(30) NOT NULL,
    Prenom varchar(30) CONSTRAINT nl_pre NOT NULL,
    Age int(2) check (age < 100),
    Email varchar(50) NOT NULL, check (Email LIKE "%@%"),
    ADR int(3) UNIQUE,
    CONSTRAINT PK_cin PRIMARY KEY (cin)
    FOREIGN KEY (ADR) REFERENCES Adresse (code_adr)
);
```

✓ Modification de la structure d'une base de données :

▪ **Modifier la structure d'une table :**

C'est grâce à la close **ALTER TABLE** qu'on peut modifier la structure d'une table.

Syntaxe :

ALTER TABLE Nom_table

[DROP CONSTRAINT Nom_contrainte]

[ADD CONSTRAINT définition_contrainte]

[DROP COLUMN Nom_colonne]

[ADD COLUMN (définition_colonne)]

[MODIFY (définition_colonne)]

[ENABLE/DISABLE nom_contrainte]

L'option **ADD COLUMN** permet de rajouter des nouvelles colonnes à la table. La clause « **définition_colonne** » a la même syntaxe que celle utilisée dans la commande CREATE TABLE.

Exemple:

ALTER TABLE clients MODIFY Age DATE ;

Question n°1 :

Ecrire la commande qui permet d'ajouter la colonne « E_mail » (de 70 caractères) à la table **Joueur**.

ALTER TABLE Joueur

ADD COLUMN (e_mail VARCHAR(70)) ;

L'option **ADD CONSTRAINT** permet de rajouter une nouvelle contrainte à la table. La clause « *définition_contrainte* » a la même syntaxe que celle utilisée dans la commande CREATE TABLE.

Question n°2 :

Supposons que la table Joueur a été créée sans clé primaire. Ecrire la commande qui permet de préciser cette clé primaire.

ALTER TABLE Joueur

ADD CONSTRAINT Primary Key (Num_j);

L'option **MODIFY** permet la modification de certaines caractéristiques d'une **colonne existante**. La clause « *définition_colonne* » a la même syntaxe que celle utilisée dans la commande CREATE TABLE.

Question n°3 :

Supposons qu'on souhaite élargir la taille de la colonne « E_mail »(taille 100) de la table Joueur.

ALTER TABLE Joueur

MODIFY e_mail VARCHAR(100);

L'option **DROP COLUMN** permet de supprimer une colonne de la table.

Supposons qu'on souhaite supprimer la colonne « e_mail » de la table Joueur.

ALTER TABLE Joueur

DROP COLUMN e_mail ;

L'option **DROP CONSTRAINT** permet de supprimer une contrainte d'intégrité de la table.

Question n°5 :

Supposons qu'on ne souhaite plus assurer l'identification des Joueur par le Num_j. Ecrire la commande qui permet de supprimer la clé primaire de la table Joueur.

ALTER TABLE Joueur

DROP CONSTRAINT PRIMARY KEY ;

L'option **DISABLE** permet de désactiver une contrainte d'intégrité. Lorsqu'une contrainte est désactivée, le SGBD ne va plus effectuer le contrôle imposé par cette contrainte.

Question n°6 :

Désactiver la clé primaire de la table Joueur.

ALTER TABLE Joueur

DISABLE CONSTRAINT PRIMARY KEY ;

Remarque : Lorsque cette clé primaire est référencée dans une ou plusieurs autres tables, il faut supprimer les clés étrangères dans les tables qui s'y réfèrent avant de procéder à la suppression de la clé primaire.

L'option **ENABLE** permet de réactiver une contrainte d'intégrité. Lorsqu'une contrainte est réactivée, le SGBD va de nouveau effectuer le contrôle imposé par cette contrainte.

Question n°7 :

Réactiver la clé primaire de la table Joueur.

**ALTER TABLE Joueur
ENABLE CONSTRAINT PRIMARY KEY ;**

▪ **La suppression d'une table :**

Elle se fait en suivant la syntaxe suivante : **DROP Nom_table;**

Exemple:

DROP clients ;

Question n°8 :

Ecrire la commande qui permet de SUPPRIMER la table Joueur.

DROP TABLE Joueur;

B. Langage de manipulation de données : LMD

Il permet de sélectionner, insérer, modifier ou supprimer des données dans une table d'une base de données relationnelle.

1- Mise à jour des données

a. Insertion de lignes

Permet d'ajouter de nouvelles lignes dans une table dont la structure est déjà créée dans la base

INSERT INTO nom_table [liste_nom_colonnes]
VALUES (liste des valeurs)

Remarques :

Pour que l'opération d'insertion puisse être exécutée, les conditions suivantes doivent être respectées :

- Les types des données de liste_valeur doivent être compatibles avec ceux des colonnes de la table,
- L'ordre des valeurs est celui des colonnes
- La liste des noms des colonnes est facultative dans le cas où toutes les colonnes de la table sont concernées par l'opération d'ajout
- Unicité des lignes (contrainte de clé primaire),

- Caractère obligatoire associé à une colonne (clause NOT NULL),
- Existence de la valeur dans une autre table quand il s'agit d'une clé étrangère (contrainte d'intégrité référentielle) : nécessité de respecter un certain ordre lors de l'insertion des lignes
- Vérification d'une condition de validité (contrainte valeur : clause CHECK).

b. La modification de lignes

Permet de modifier des valeurs de colonnes d'une table

```
UPDATE nom_table
SET nomcolonne1= Expression1[, nomcolonne2=Expression2....]
[WHERE condition]
```

Remarque :

- Toutes les lignes vérifiant la condition seront mises à jour
- Les nouvelles expressions peuvent utiliser les anciennes valeurs de la ligne à mettre à jour, le résultat de la nouvelle expression remplace l'ancienne valeur de la colonne
- Si la clause WHERE est absente, toutes les lignes de la table seront mises à jour

c. Suppression de lignes

Permet de supprimer une ou plusieurs lignes existantes à partir d'une table

```
DELETE FROM nom_table
[WHERE condition]
```

Remarque :

- Toutes les lignes vérifiant la condition sont supprimées
- Si la clause WHERE **est absente**, toutes les lignes de la table **seront effacées**
- La suppression d'une ligne peut entraîner la suppression d'autres lignes dans d'autres tables **si la contrainte ON DELETE CASCADE est définie au moment de la création**

2- Recherche de données : Requêtes

Une requête est une opération de recherche de données à partir d'une ou plusieurs tables.

Cette recherche peut concerner :

- Certaines colonnes d'une table : **Projection**
- Certaines lignes d'une table : **Sélection**
- Deux tables en relation : **jointure**

Une requête peut être réalisée en combinant ces trois actions.

a. Requêtes de projection

Cette requête ne concerne qu'une seule table de la BD, elle doit comporter au moins une colonne de la table, et toutes les lignes de cette colonne seront affichées au résultat.

```
SELECT [DISTINCT] * / liste_nom_colonnes
FROM nom_table
```

Remarque :

- La **liste_nom_colonnes** précise les colonnes à afficher au résultat, elle peut être remplacée par ***** pour tout afficher.
- **DISTINCT** permet d'éliminer les lignes en double dans le résultat.
- Le résultat de la commande **SELECT** est une nouvelle table résultat.
- Par défaut, les colonnes de la table résultat portent les mêmes noms que ceux de la table de départ
- On peut utiliser des **ALIAS**, pour modifier les noms des colonnes de la table résultat

b. Requêtes de Sélection

Cette requête permet d'afficher les lignes qui vérifient une condition.

```
SELECT [DISTINCT] * / liste_nom_colonnes
FROM nom_table
WHERE condition
```

Remarque :

- Le paramètre **condition** précise le critère qui doit être vérifié par les lignes à afficher
- La condition est une expression logique, qui peut utiliser :
 - Les opérateurs de comparaison et logiques
 - L'opérateur BETWEEN pour les intervalles de valeurs
 - IN pour la liste de valeurs
 - IS NULL / IS NOT NULL
 - LIKE pour filtrer une chaîne de caractères

c. Requêtes Jointure

C'est une recherche à partir de plusieurs tables

```
SELECT [DISTINCT] * / liste_nom_colonnes
FROM nom_table1 [Alias1], nom_table2 [Alias2]...
WHERE condition
```

Remarque:

- La **condition** de jointure doit porter sur les **colonnes en communs** aux tables (clé primaire-clé étrangère) qui ne doit pas être confondue avec la **condition critère de sélection**.
- Pour alléger l'écriture de la commande SELECT, un **Alias** peut être utilisé. Il permet de donner un nom abrégé à une table.

d. Recherche de données avec Tri

```
SELECT [DISTINCT] * / liste_nom_colonnes
FROM nom_table1 [Alias1], nom_table2 [Alias2]...
[WHERE condition]
ORDER BY nom_colonne1 [ASC / DESC] [, nom_colonne2 [ASC/ DESC]....]
```

Remarque:

- **ORDER BY** réalise le tri
- **ASC / DESC** ordre croissant / décroissant
- Le tri peut être associé à n'importe quelle opération de recherche (projection, sélection et jointure)

e. Utilisation des fonctions de calcul (agrégat)

Le langage SQL offre certaines fonctions standards de calcul appelées **fonctions agrégat**.

Ces fonctions permettent de faire un certain calcul sur les lignes recherchées.

Ces fonctions sont :

- **COUNT** : permet de compter le nombre de lignes du résultat obtenu par la commande
SELECT
- **SUM** : permet de faire la somme des valeurs d'une colonne dont le type est numérique
- **MIN** : détermine la valeur minimale d'une colonne
- **Max** : détermine la valeur maximale d'une colonne
- **AVG** : détermine la moyenne des valeurs numériques d'une colonne



Application :

Créer une nouvelle base de données dans votre dossier de travail et lui donner le nom « **Gestion des notes + votre nom** ».

Sql

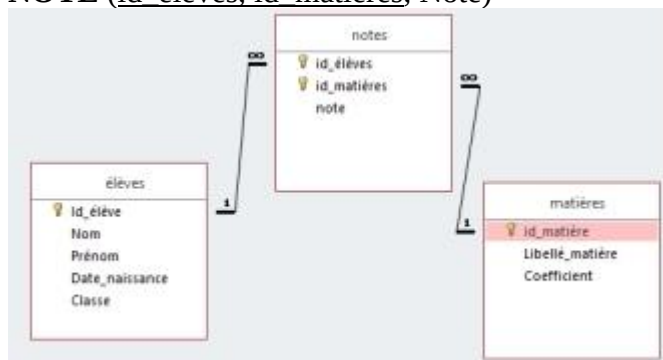
.....

On veut Créer les tables suivantes en respectant le schéma de la base de données présenté ci- dessous.

ELEVE (id_élèves, Nom, Prénom, Date_naissance, Classe)

MATIERE (id_matières, libellé_matière,coefficient)

NOTE (id_élèves, id_matières, Note)



La table **Elève** (La clé primaire est ID_ELEVE)

Champ	Type de données	Propriétés
<u>ID_ELEVE</u>	Varchar	Taille = 8
Nom	Varchar	Taille = 30
Prénom	Varchar	Taille = 30
Date_naissance	Date/Heure	
Classe	Varchar	Taille = 8

La table **Matière** (La clé primaire est ID_MATIERE)

Champ	Type de données	Propriétés
<u>ID_MATIERE</u>	Varchar	Taille = 3
Libellé_matière	Varchar	Taille = 30
Coefficient	Numérique	réel simple

3) La table **Note** (La clé primaire est ID_ELEVE, ID_MATIERE)

Champ	Type de données	Propriétés
ID_ELEVE	Varchar	Taille = 8
ID_MATIERE	Varchar	Taille = 3
Note	Numérique	Taille = réel simple

A)Créer les tables élèves,matières et notes puis saisir les données

La table ELEVE

1)Création de la table élèves (structure)

Sql:

2Création de la table matière

sql:.....

3)Création la table note

Sql

Création des relations :

b)Insertion de données:

Id_élève	Nom	Prénom	Date de naissance	Classe
E01	Tounsi	Ahmed	04/01/2003	3SI
E02	Mrad	Amira	23/10/2001	4ECO2
E03	Dridi	Wided	22/01/2002	4ECO1
E04	Jileni	Amira	23/10/2004	3ECO2
E05	Mtir	Amine	22/03/2006	1s1
E06	Nouira	Rami	05/04/2007	1s1
E07	Masmoudi	Ahmed	06/12/2004	3SI

Sql: (se limiter à deux enregistrements)

La table MATIERE

ID_MATIERE	Libellé_Matière	Coefficient
M01	Algorithmme	3
M02	STI	3
M03	Math	1,5

d)insertion de données dans la table matière :

.....

.....

.....

.....

.....

La table NOTE

ID_ELEVE	ID_MATIERE	Note
E01	M01	14
E01	M02	13
E01	M03	10
E02	M03	11
E03	M03	17
E07	M02	14,5

Requête de sélection :

1)Afficher tous les élèves

.....

.....

2)Afficher les nom et prénom des élèves

.....

.....

3)Afficher les élèves dont la classe est 3SI

.....

.....

.....

4)compter le nom des élèves (count)

.....

.....

.....

5) Afficher Les notes qui sont comprises entre 10 et 15 (opérateur BETWEEN)

(opérateur BETWEEN)	Opérateur AND
.....	
.....	
.....	
.....	
.....	

6)Modifier La classe de l'élèves dont l'identifiant est E01 de 3SI à 4SI

.....

.....

.....

7)Afficher le nombre des élèves du classe 4ECO

.....

.....

.....

8)Afficher les ID des élèves qui ont des notes supérieures à 12 et dont l'ID matière est « M02 »

.....

.....
.....

9) Afficher l’ID et la note de l’élève qui a eu la note maximale dont l’id matière est «M03»

.....
.....

.....

10) Afficher l’ID et la note de l’élève qui a eu la note minimale dont l’id matière est «M03»

.....
.....
.....

Correction serie 1

Création des tables

Table élèves

CREATE TABLE élèves (id_élèves varchar(8), Nom varchar(20), Prénom varchar(20), Date_naissance date, classe varchar(8), PRIMARY KEY (id_élèves));	OU	CREATE TABLE élèves (id_élèves varchar(8) PRIMARY KEY, Nom varchar(20), Prénom varchar(20), Date_naissance date, classe varchar(8));
--	----	---

Table matière

CREATE TABLE matière (id_matière varchar(8) , libellé_mat varchar(20), Coefficient float , PRIMARY KEY (id_matière));		CREATE TABLE matière (id_matière varchar(8) PRIMARY KEY , libellé_mat varchar(20), Coefficient float);
---	--	---

Table note

CREATE TABLE note (id_élèves varchar(8) REFERENCES élèves(id_élèves), id_matière varchar(8) REFERENCES matière(id_matière), note float, PRIMARY KEY (id_élèves, id_matière));
--

Création de relation :

1) changement du type de la tables élèves ,matière et note à INNODB au lieu de MyISAM

 Opérations	Type de la table: ?	Exécuter	requête SQL: ALTER TABLE 'élèves1' TYPE = INNODB
	INNO DB		requête SQL: ALTER TABLE 'matière1' TYPE = INNODB
			requête SQL: ALTER TABLE 'note1' TYPE = INNODB

b)Création des relations

Relié à				
	InnoDB			
id_élèves	élèves->id_élèves	ON DELETE	CASCADE	ON UPDATE CASCADE
id_matière	note->id_matière	ON DELETE	CASCADE	ON UPDATE CASCADE
note				
Exécuter				

Résultat :

ALTER TABLE `note`

ADD CONSTRAINT `note\_ibfk\_1` FOREIGN KEY (`id\_élèves`) REFERENCES `élèves` (`id\_élèves`)
ON DELETE CASCADE ON UPDATE CASCADE,ADD CONSTRAINT `note\_ibfk\_2` FOREIGN KEY (`id\_matière`) REFERENCES `matière`
(`id\_matière`) ON DELETE CASCADE ON UPDATE CASCADE;

Insertion de données

INSERT INTO élèves VALUES ('E01', 'Tounsi', 'Ahmed', '2003-01-04', '4SI');

INSERT INTO matière VALUES ('M01', 'Algorithmme', 3);

Requête de sélection :

1) Afficher tous les élèves

R1 : SELECT * FROM élèves;	requête SQL: SELECT * FROM 'élèves' LIMIT 0 , 30	Trier sur l'index: aucune Exécuter <table border="1"> <thead> <tr> <th></th> <th>id_élèves</th> <th>Nom</th> <th>Prénom</th> <th>Date_naissance</th> <th>classe</th> </tr> </thead> <tbody> <tr><td>☐</td><td>E01</td><td>Tounsi</td><td>Ahmed</td><td>2003-01-04</td><td>3SI</td></tr> <tr><td>☐</td><td>E02</td><td>Mrad</td><td>Amira</td><td>2001-10-23</td><td>4ECO2</td></tr> <tr><td>☐</td><td>E03</td><td>Dridi</td><td>wided</td><td>2002-01-22</td><td>4ECO1</td></tr> <tr><td>☐</td><td>E04</td><td>Jilani</td><td>Amira</td><td>0000-00-00</td><td>3ECO2</td></tr> <tr><td>☐</td><td>E05</td><td>Mtir</td><td>Amine</td><td>2006-03-22</td><td>1S1</td></tr> <tr><td>☐</td><td>E06</td><td>Nouira</td><td>Rami</td><td>2007-04-05</td><td>1S1</td></tr> <tr><td>☐</td><td>E07</td><td>Masmoudi</td><td>Ahmed</td><td>2004-12-06</td><td>4SI</td></tr> </tbody> </table>		id_élèves	Nom	Prénom	Date_naissance	classe	☐	E01	Tounsi	Ahmed	2003-01-04	3SI	☐	E02	Mrad	Amira	2001-10-23	4ECO2	☐	E03	Dridi	wided	2002-01-22	4ECO1	☐	E04	Jilani	Amira	0000-00-00	3ECO2	☐	E05	Mtir	Amine	2006-03-22	1S1	☐	E06	Nouira	Rami	2007-04-05	1S1	☐	E07	Masmoudi	Ahmed	2004-12-06	4SI
	id_élèves	Nom	Prénom	Date_naissance	classe																																													
☐	E01	Tounsi	Ahmed	2003-01-04	3SI																																													
☐	E02	Mrad	Amira	2001-10-23	4ECO2																																													
☐	E03	Dridi	wided	2002-01-22	4ECO1																																													
☐	E04	Jilani	Amira	0000-00-00	3ECO2																																													
☐	E05	Mtir	Amine	2006-03-22	1S1																																													
☐	E06	Nouira	Rami	2007-04-05	1S1																																													
☐	E07	Masmoudi	Ahmed	2004-12-06	4SI																																													

2) Afficher les nom et prénom des élèves

R2 : SELECT Nom,Prénom FROM élèves ;	requête SQL: SELECT 'Nom' , 'Prénom' FROM 'élèves' WHERE 1 LIMIT 0 , 30	Trier sur l'index: aucune Exécuter <table border="1"> <thead> <tr> <th></th> <th>Nom</th> <th>Prénom</th> </tr> </thead> <tbody> <tr><td>☐</td><td>Tounsi</td><td>Ahmed</td></tr> <tr><td>☐</td><td>Mrad</td><td>Amira</td></tr> <tr><td>☐</td><td>Dridi</td><td>wided</td></tr> <tr><td>☐</td><td>Jilani</td><td>Amira</td></tr> <tr><td>☐</td><td>Mtir</td><td>Amine</td></tr> <tr><td>☐</td><td>Nouira</td><td>Rami</td></tr> <tr><td>☐</td><td>Masmoudi</td><td>Ahmed</td></tr> </tbody> </table>		Nom	Prénom	☐	Tounsi	Ahmed	☐	Mrad	Amira	☐	Dridi	wided	☐	Jilani	Amira	☐	Mtir	Amine	☐	Nouira	Rami	☐	Masmoudi	Ahmed
	Nom	Prénom																								
☐	Tounsi	Ahmed																								
☐	Mrad	Amira																								
☐	Dridi	wided																								
☐	Jilani	Amira																								
☐	Mtir	Amine																								
☐	Nouira	Rami																								
☐	Masmoudi	Ahmed																								

3) Afficher id_élèves, id_matière dont la note est supérieur à 12

SELECT id_élèves, id_matière, note FROM note WHERE note >12;	Trier sur l'index: aucune Exécuter <table border="1"> <thead> <tr> <th></th> <th>id_élèves</th> <th>id_matière</th> <th>note</th> </tr> </thead> <tbody> <tr><td>☐</td><td>E01</td><td>M01</td><td>14</td></tr> <tr><td>☐</td><td>E01</td><td>M02</td><td>13</td></tr> <tr><td>☐</td><td>E03</td><td>M03</td><td>17</td></tr> <tr><td>☐</td><td>E07</td><td>M02</td><td>14</td></tr> </tbody> </table>		id_élèves	id_matière	note	☐	E01	M01	14	☐	E01	M02	13	☐	E03	M03	17	☐	E07	M02	14
	id_élèves	id_matière	note																		
☐	E01	M01	14																		
☐	E01	M02	13																		
☐	E03	M03	17																		
☐	E07	M02	14																		

4) Afficher le nombre des élèves en utilisant l'opérateur count

SELECT count(*) FROM élèves ;	requête SQL: SELECT COUNT(*) FROM 'élèves'	count(*) 7
--	--	-----------------------------

5) Afficher id_élèves, id_matière dont la note est entre 12 et 17 en utilisant l'opérateur BETWEEN

(a) SELECT note FROM note WHERE (note BETWEEN 12 AND 17);	(b) SELECT note FROM note WHERE (note >12 AND note <17)	note ▲ 13 14 14 17
--	--	--------------------------------

Requête de mise à jour

6) Modifier La classe de l'élèves dont l'identifiant est E01 de 3SI à 4SI

UPDATE élèves SET classe="4SI" WHERE id_élèves="E01";	Nombre d'enregistrements affectés : 1 (traitement) requête SQL: UPDATE élèves SET classe = "4SI" WHERE id_élèves = "E01"	Trier sur l'index: aucune Exécuter <table border="1"> <thead> <tr> <th></th> <th>id_élèves</th> <th>Nom</th> <th>Prénom</th> <th>Date_naissance</th> <th>classe</th> </tr> </thead> <tbody> <tr><td>☐</td><td>E01</td><td>Tounsi</td><td>Ahmed</td><td>2003-01-04</td><td>4SI</td></tr> </tbody> </table>		id_élèves	Nom	Prénom	Date_naissance	classe	☐	E01	Tounsi	Ahmed	2003-01-04	4SI
	id_élèves	Nom	Prénom	Date_naissance	classe									
☐	E01	Tounsi	Ahmed	2003-01-04	4SI									

Requête de sélection :**7)Afficher les élèves dont la classe est 3SI**

(a) SELECT élèves.* FROM élèves WHERE élèves.classe= '3SI' ; Ou (b) SELECT * FROM élèves WHERE classe= '3SI';	requête SQL: SELECT 'élèves' . * FROM élèves WHERE ('élèves'.classe = "3SI") LIMIT 0 , 30	
--	--	--

Remarque : on peut écrire dans la clause select le **Nom_du_table.nom_du_colonne** → (a) ou le **nom du du colonne directement**.-->(b) s'il n ya deux colle qui porte le même nom.

8)Afficher les ID des élèves qui ont des notes supérieures à 12 et dont l'ID matière est « M02 »

SELECT id_élèves FROM note WHERE note>12 AND id_matière="M02";	SELECT id_élèves FROM note WHERE note >12 AND id_matière = "M02" LIMIT 0 , 30	
--	---	--

9) Afficher la note maximale

SELECT MAX(note) FROM note;	SELECT MAX(note) FROM note	
--	---	--

(b) avec un titre le champ calculé MAX.

SELECT MAX(note) AS "Le maximum des note " FROM note	
---	--

10) Afficher la note de l'élève minimale

requête SQL: SELECT MIN(note) FROM note	
--	--

11)Afficher la moyenne des notes

requête SQL: SELECT AVG(note) FROM note	
--	--

–Application 2 base de données

Soit le schéma relationnel de la base de données « gestion de projets »

Employé (NumEmp, nom, Prénom, Adresse, Tél, Grade, NumService #)

Service (NumService, NomService, Responsable, Tél)

Projet (NumProjet, DatDeb, Datfin, Numservice#).

- 1) Créer cette base de données avec le nom « gestion de projets »

Sql :

- 2) Créer les tables de la base de données

Employé

<i>champ</i>	<i>Type de données</i>	<i>propriétés</i>
NumEmp	Numérique	Entier taille 8
Nom	texte	Taille= 15
Prénom	Texte	Taille = 20
Adresse	texte	Taille = 30
Tél	numérique	Entier taille=8
Grade	texte	Taille = 15
NumService	texte	Taille = 3

Service

<i>champ</i>	<i>Type de données</i>	<i>propriétés</i>
NumService	texte	Taille = 3
NomService	texte	Taille= 20
Responsable	texte	Taille = 25
Tél	numérique	Entier taille=8

Projet

<i>champ</i>	<i>Type de données</i>	<i>propriétés</i>
NumProjet	Numérique	Entier taille 11
NomProjet	texte	Taille= 10
DatDeb	Date	
Datfin	Date	
Numservice	texte	Taille = 3

- 3) Créer les relations entre les tables.
 4) Donner la représentation graphique des tables

5) Insérer les données des différentes tables en mode sql.

Service

<i>NumService,</i>	<i>NomService</i>	<i>Responsable</i>	<i>Tél</i>
S01	financier	001	73787987
S02	commercial	003	73123543
S03	Administratif	001	73126543
S04	commercial	003	73254376
S05	travaux	001	73238765

Employé

<i>NumEmp</i>	<i>Nom</i>	<i>Prénom</i>	<i>Adresse</i>	<i>Tél</i>	<i>Grade</i>	<i>NumService</i>
001	Tounsi	Safa	Tunis	98765676	Ingénieur	S01
002	Kefi	Ali	Tunis	97867897	ouvrier	S02
003	Beji	Mohamed	Béja	99876786	employé	S02
004	Soussi	Lamia	Sousse	96545674	ouvrier	S03
005	Touati	leila	Sousse	95765423	cadre	S04

Projet

<i>NumProjet</i>	<i>NomProjet</i>	<i>DatDeb</i>	<i>Datfin</i>	<i>Numservice</i>
01	Projet1	02/01/2004	15/07/2005	S01
03	Projet2	28/10/2005	15/05/2006	S03
04	Projet3	12/01/2004	18/06/2006	S04
05	Projet4	20/01/2006	01/07/2006	S01

5) Créer les requêtes suivantes

✓ **R1:** Afficher la liste des employés

.....

✓ **R2 :** donner la liste des employés (nom, prénom) qui habitent à tunis

.....

- ✓ **R 3:**Afficher la liste des employés qui habitent à Sousse par ordre croissant des noms.

.....

- ✓ **R4 :** donner la liste des employés (nom, prénom) dont le nom commencent par T.

.....

- ✓ **R 5 :**Afficher les projets dont la datedeb est **supérieur** au 02/01/2004

.....

- ✓ **R6:**Afficher les noms de services dont la datedeb est **inférieur** au 20/08/2005.

.....

- ✓ **R7 :**donner la liste des projets dont la datedeb est entre 02/01/2004 et 20/01/2006.

.....

- ✓ **R8a)** Afficher le nombre de projets (count)

.....

- ✓ **R8a)** Afficher le nombre de projets du service S01

.....

- ✓ **R8c)** Afficher le nombre de projets pour chaque service (group by)

.....

R8d) Afficher le nombre de projets du service S01

.....

- ✓ **R9** Insérer le champ “email” dans la table Employé (texte(20)).

.....

- ✓ *ALTER TABLE Nom_table*
- ✓ *[DROP CONSTRAINT Nom_contrainte]*
- ✓ *[ADD CONSTRAINT définition_contrainte]*
- ✓ *[DROP COLUMN Nom_colonne]*
- ✓ *[ADD COLUMN (définition_colonne)]*
- ✓ *[MODIFY (définition_colonne)]*
- ✓ *[ENABLE/DISABLE nom_contrainte]*

Rappel

- ✓
- ✓ R10 Ajouter la contrainte suivante (le champ email doit contenir le caractère"@").

.....

.....

.....

- ✓ R11 : Afficher les nom des villes des employés sans répétition (DISTINCT)
- ✓ Quels sont les noms des services correspondants à l'employé « Tounsi Safa »

.....

.....

.....

- ✓ R12 : compter le nombre de villes des employés

.....

.....

.....

- ✓ R13 Ajouter un email pour Tounsi Safa (UPDATE) Email :tounsiSafa@gmail.com

.....

.....

Permet de modifier des valeurs de colonnes d'une table

```
UPDATE nom_table
SET nomcolonne1= Expression1[, nomcolonne2=Expression2....]
[WHERE condition]
```

- ✓ R14 Afficher les employées qui n'ont pas d'email

.....

.....

- ✓ R15 compter le nombre d'employées qui n'ont pas un email

.....

.....

.....

- ✓ R16 Supprimer les enregistrement de la table employé dont le champ email est vide
 - ✓ Permet de supprimer une ou plusieurs lignes existantes à partir d'une table

- ✓ DELETE FROM nom_table
- ✓ [WHERE condition]

.....

.....

.....

- ✓ R17-Supprimer le champ email de la table employé

.....

.....

.....

- ✓ R18 Quels sont les noms des services correspondants au employés dont la deuxième lettre du nom est « o »

.....

.....

.....

- ✓ R19 Afficher les nom des services qui ont des projet

.....

.....

Serie 2 base de données(correction)

Soit le schéma relationnel de la base de données « **gestion de projets** »

Employé (NumEmp, nom, Prénom, Adresse, Tél, Grade, NumService #)

Service (NumService, NomService, Responsable, Tél)

Projet (NumProjet, DatDeb, Datfin, Numservice#).

- 1) Créer cette base de données avec le nom « gestion de projets »

Sql : **CREATE DATABASE** `gestionprojet` ;

- 2) Créer les tables de la base de données

Employé

champ	Type de données	propriétés	sql
NumEmp	Numérique	Entier taille 10	CREATE TABLE employée (numemp int(10) , Nom varchar(15), Prénom varchar(20), Adresse varchar(30), Tél int(8) , Grade varchar(15), numservice varchar(3) REFERENCES service(numservice), PRIMARY KEY (numemp));
Nom	texte	Taille= 15	
Prénom	Texte	Taille = 20	
Adresse	texte	Taille = 30	
Tél	numérique	Entier taille 8	
Grade	texte	Taille = 15	
NumService	texte	Taille = 3	

Service

champ	Type de données	propriétés	sql
NumService	texte	Taille = 3	CREATE TABLE service (numservice varchar(3), nomservice varchar(20), Responsable varchar(25), tel bigint, PRIMARY KEY (numservice))
NomService	texte	Taille= 20	
Responsable	texte	Taille = 25	
Tél	numérique	Entier long	

Projet

champ	Type de données	propriétés	sql
NumProjet	Numérique	entier	CREATE TABLE projet (numprojet int nomprojet varchar(10), datdeb date, datfin date, numservice varchar(3) REFERENCES service(numservice), PRIMARY KEY (numprojet),)
NomProjet	texte	Taille= 10	
DatDeb	Date		
Datfin	Date		
Numservice	texte	Taille = 3	

3) Créer les relations entre les tables.

1) changement du type de la tables Employé ,service et projet à INNODB au lieu de MyISAM

 Opérations	Type de la table: ?	Exécuter
	INNO DB	

2-Création d'index pour numservice du table projet et du table employée.

Table projet :

requête SQL:	Nom de la clé	Type	Cardinalité	Action	Champ
ALTER TABLE `projet` ADD INDEX (`numservice`)	PRIMARY	PRIMARY	0		numprojet
	numservice	INDEX	0		numservice

3-création des relations

Table projet

 Gestion des relations	numservice	service->numservice	ON DELETE	CASCADE	ON UPDATE	CASCADE
--	------------	---------------------	-----------	---------	-----------	---------

Sql :

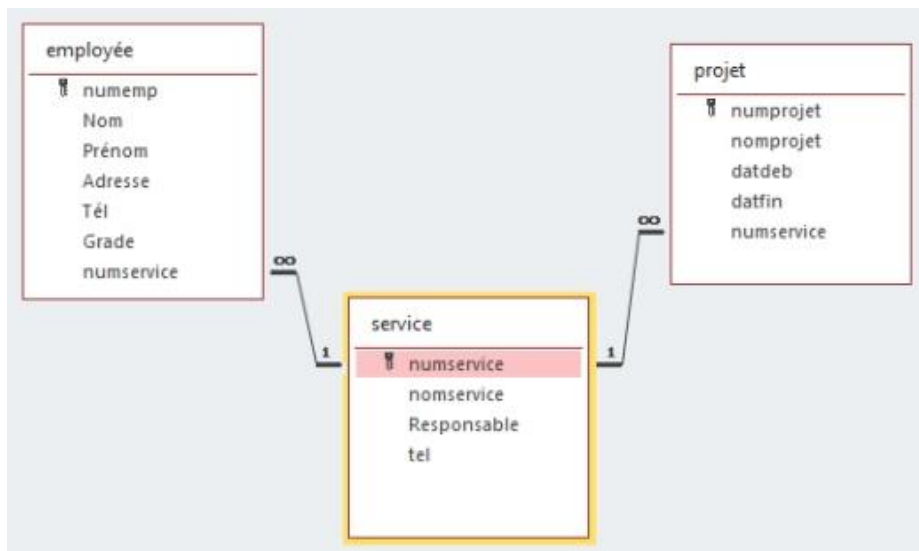
ALTER TABLE `projet`

ADD CONSTRAINT `projet\_ibfk\_1` FOREIGN KEY (`numservice`) REFERENCES `service` (`numservice`)
ON DELETE CASCADE ON UPDATE CASCADE;

ALTER TABLE `employée`

ADD CONSTRAINT `employée\_ibfk\_1` FOREIGN KEY (`numservice`) REFERENCES `service` (`numservice`) ON DELETE CASCADE ON UPDATE CASCADE;

Donner la représentation graphique des tables



5) Insérer les données des différentes tables en mode sql.

Service

NumService,	NomService	Responsable	Tél
S01	financier	001	73787987

S02	commercial	003	73123543
S03	Administratif	001	73126543
S04	commercial	003	73254376
S05	travaux	001	73238765

Employé

NumEmp	Nom	Prénom	Adresse	Tél	Grade	NumService
001	Tounsi	Safa	Tunis	98765676	Ingénieur	S01
002	Kefi	Ali	Tunis	97867897	ouvrier	S02
003	Beji	Mohamed	Béja	99876786	employé	S02
004	Soussi	Lamia	Sousse	96545674	ouvrier	S03
005	Touati	leila	Sousse	95765423	cadre	S04

Projet

NumProjet	NomProjet	DatDeb	Datfin	Numservice
01	Projet1	02/01/2004	15/07/2005	S01
03	Projet2	28/10/2005	15/05/2006	S03
04	Projet3	12/01/2004	18/06/2006	S04
05	Projet4	20/01/2006	01/07/2006	S01

Table service (se limiter à un enregistrement)

Sql: INSERT INTO service VALUES ('s01', 'financier', '001', 73787987);

Table employée (se limiter à un enregistrement)

Sql: INSERT INTO employee VALUES (1, 'Tounsi', 'Safa', 'Tunis', 98765676, 'ingenieur', 's01');

Table Projet : (se limiter à un enregistrement)

Sql: INSERT INTO projet VALUES (1, 'projet1', '2004-02-01', '2005-07-15', 's01');

Créer les requêtes suivantes (se limiter à un enregistrement)

- ✓ **R1:** Afficher la liste des employés

SELECT *

FROM employée ;

- ✓ **R2 :** donner la liste des employés (nom, prénom) qui habitent à tunis

SELECT nom, prénom

FROM employée

WHERE adresse="Tunis" ;

- ✓ **R3:** Afficher la liste des employés qui habitent à Sousse par ordre croissant des noms.

SELECT *
FROM employée
WHERE adresse="Sousse"
ORDER BY (nom) ASC;

←T→	numemp	nom	prénom	adresse	tel	grade	numservice
☐	4	Soussi	Lamia	Sousse	96545674	ouvrier	s03
☐	5	Touati	Leila	Sousse	95765423	cadre	s04

- R4 :** donner la liste des employés (nom, prénom) dont le nom commence par T.(LIKE)

SELECT nom, prénom
FROM employée
WHERE nom LIKE "T%" ;

←T→	nom	prénom
☐	Tounsi	Safa
☐	Touati	Leila

- ✓ **R5 :** Afficher les projets dont la datedeb est **inférieure** au 02/01/2004

SELECT *

FROM projet
WHERE datdep<"2004-02-01"

- ✓ **R6** :donner la liste des projets dont la datedep est entre 02/01/2004 et 20/01/2006. (BETWEEN)

SELECT *
FROM projet
WHERE datdeb BETWEEN "2004-02-01" AND "2006-01-20";

R8a) : Afficher le nombre de projets des services

SELECT COUNT(numprojet)
FROM projet

R8b) : Afficher le nombre de projets du service S01

SELECT COUNT(numprojet)
FROM projet
Where numservice='S01' ;

R8c) : Afficher le nombre de projets pour chaque service (GROUP BY)

SELECT numservice,COUNT(numprojet)
FROM projet
GROUP BY(numservice)

← T →	
numservice	COUNT(numprojet)
s01	2
s03	1
s04	1

- ✓ **R8d** : Afficher le nombre de projets du service S01 (HAVING)

SELECT numservice,COUNT(numprojet)
FROM projet
GROUP BY(numservice)
HAVING numservice="S01";

← T →	
numservice	COUNT(numprojet)
s01	2

- ✓ **R9** Insérer le champ “email” dans la table Employé (texte(20)).(ALTER TABLE)
ALTER TABLE employée
ADD email varchar(20);

R10 Ajouter la contrainte suivante (le champ email doit contenir le caractère”@”).

ALTER TABLE employée
ADD CONSTRAINT c1 CHECK(email LIKE ‘ %@%’);

- ✓ **R11** Afficher les nom des villes des employés sans répétition (DISTINCT)

SELECT DISTINCT(adresse)
FROM employée

← T →		adresse
☐		Tunis
☐		Beja
☐		Sousse

- ✓ **R12** compter le nombre de villes des employés

SELECT COUNT(DISTINCT(adresse))
FROM employée

COUNT(DISTINCT(adresse))
3

- ✓ **R13** Ajouter un email pour Tounsi Safa (UPDATE)
Email :tounsiSafa@gmail.com

```
UPDATE employée
SET email="tounsisafa@gmail.com"
Where nom="Tounsi" AND prénom="Safa";
```

- ✓ R14 Afficher les employées qui n'ont pas d'email

SELECT * FROM `employée` WHERE email IS NULL ;	← T → <table><thead><tr><th></th><th>numemp</th><th>nom</th><th>prénom</th><th>adresse</th><th>tel</th><th>grade</th><th>numservice</th><th>email</th></tr></thead><tbody><tr><td> ☐ </td><td>2</td><td>Kefi</td><td>Ali</td><td>Tunis</td><td>978676897</td><td>ouvrier</td><td>s02</td><td>NULL</td></tr><tr><td> ☐ </td><td>3</td><td>Beji</td><td>Mouhamed</td><td>Beja</td><td>99876786</td><td>employé</td><td>s02</td><td>NULL</td></tr><tr><td> ☐ </td><td>4</td><td>Soussi</td><td>Lamia</td><td>Sousse</td><td>96545674</td><td>ouvrier</td><td>s03</td><td>NULL</td></tr><tr><td> ☐ </td><td>5</td><td>Touati</td><td>Lella</td><td>Sousse</td><td>95765423</td><td>cadre</td><td>s04</td><td>NULL</td></tr></tbody></table>		numemp	nom	prénom	adresse	tel	grade	numservice	email	☐	2	Kefi	Ali	Tunis	978676897	ouvrier	s02	NULL	☐	3	Beji	Mouhamed	Beja	99876786	employé	s02	NULL	☐	4	Soussi	Lamia	Sousse	96545674	ouvrier	s03	NULL	☐	5	Touati	Lella	Sousse	95765423	cadre	s04	NULL
	numemp	nom	prénom	adresse	tel	grade	numservice	email																																						
☐	2	Kefi	Ali	Tunis	978676897	ouvrier	s02	NULL																																						
☐	3	Beji	Mouhamed	Beja	99876786	employé	s02	NULL																																						
☐	4	Soussi	Lamia	Sousse	96545674	ouvrier	s03	NULL																																						
☐	5	Touati	Lella	Sousse	95765423	cadre	s04	NULL																																						

- ✓ R15 compter le nombre d'employées qui n'ont pas un email

SELECT count(*) FROM `employée` WHERE email IS NULL	SELECT count(numemp) FROM `employée` WHERE email IS NULL	count(*) 4
---	--	----------------------------------

- ✓ R16 Supprimer les enregistrement de la table employé dont le champ email est vide

DELETE FROM employée WHERE email IS NULL;	127.0.0.1 indique Voulez-vous vraiment effectuer : DELETE FROM employée WHERE email IS NULL; OK Annuler Nombre d'enregistrements effacés : 4 (traitement: 0.0024 sec.) requête SQL: DELETE FROM employée WHERE email IS NULL
--	---

- ✓ R17-Supprimer le champ email de la table employé

ALTER TABLE employée DROP COLUMN email;	? Voulez-vous vraiment effectuer ALTER TABLE employée DROP COLUMN email ? Oui Non
--	--

Votre requête SQL a été exécutée avec succès (traitement: 0.0143 sec.)

requête SQL:
ALTER TABLE employée DROP COLUMN email

Résultat :

Serie 3

Exercice N°1 :

Pour chacune des propositions suivantes, cocher la (ou les) bonne(s) réponse(s).

❶ Dans une base de données relationnelle, on peut dire que :

- ☐ Les tables servent à stocker des données
☐ Les tables servent à stocker des règles de validation
☐ Les tables doivent comporter chacune au moins deux clés étrangères

❷ Une clé primaire :

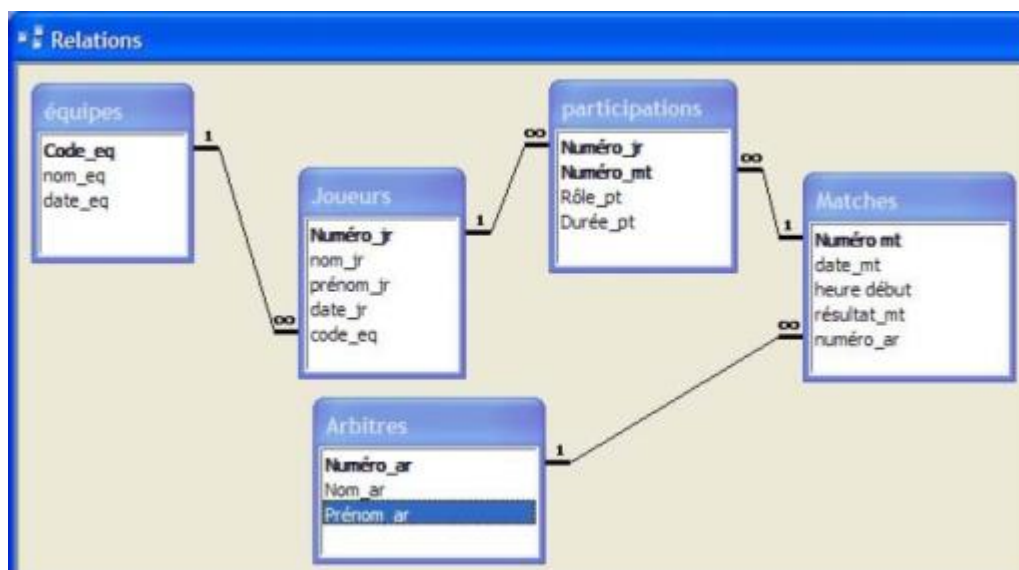
- ☐ Doit être définie par une seule colonne
☐ Peut être une clé étrangère dans une autre table
☐ Ne peut être que de type numérique

❸ Dans une base de données relationnelle :

- ☐ Il existe des liens entre les différentes tables de la base
☐ Les liens entre les tables s'organisent d'une manière hiérarchique
☐ Les liens peuvent être de différents types (1,1) ou (1, n)

Exercice N°2 :

Traduire cette représentation graphique en représentation textuelle :

**Exercice 3 :**

Corriger les commandes de création de tables suivantes :

CREATE TABLEAU Personne
 (Nom VARCHAR (40) PRIMARY KEY,
 Prénom VARCHAR (40) PRIMARY KEY,
 Adresse VARCHAR (50),
 E-mail VARCHAR (50),

Téléphone DECIMALE (10,3) DEFAULT 1);

CREATE TABLE Voiture
 (immat int(3),
 Marque VARCHAR (15) NOT NULL,
 Modèle VARCHAR (20),
 Puis_CH VARCHAR (2) DEFAULT 4 CHECK
 (Pui_CH >=4));

Exercice 4:

Soient les requêtes en SQL suivantes permettant la création des tables **ESPECE** et **CATEGORIE** d'une base de données relative à la gestion des espèces d'animaux en voie de disparition.

```
CREATE TABLE ESPECE
( IdEspece VARCHAR (8) ,
  NomEspece VARCHAR (25) NOT NULL ,
  NombreRestant INT CHECK (NombreRestant >= 0 ) ,
  AgeMoyen INT ,
  PRIMARY KEY (IdEspece) );
```

```
CREATE TABLE CATEGORIE
( IdCategorie VARCHAR (4) ,
  NomCategorie VARCHAR (30) ,
  Description VARCHAR (10) ,
  PRIMARY KEY (IdCategorie) );
```

1) donner la représentation textuelle de la table ESPECE

.....

.....

2) Utiliser la liste des types de contraintes ci-dessous pour compléter le tableau par le type de la contrainte correspondante à chaque proposition.

Types de contraintes : de table / de domaine / d'intégrité référentielle

Proposition	Type de la contrainte
PRIMARY KEY (IdEspece)	Contrainte
NomEspece VARCHAR(25) NOT NULL	Contrainte
NombreRestant INT CHECK (NombreRestant >= 0)	Contrainte

Correction serie3

Exercice N°1 : (3 Points)

Pour chacune des propositions suivantes, cocher la (ou les) bonne(s) réponse(s).

❶ Dans une base de données relationnelle, on peut dire que :

- ☒ Les tables servent à stocker des données
☐ Les tables servent à stocker des règles de validation
☐ Les tables doivent comporter chacune au moins deux clés étrangères

❷ Une clé primaire :

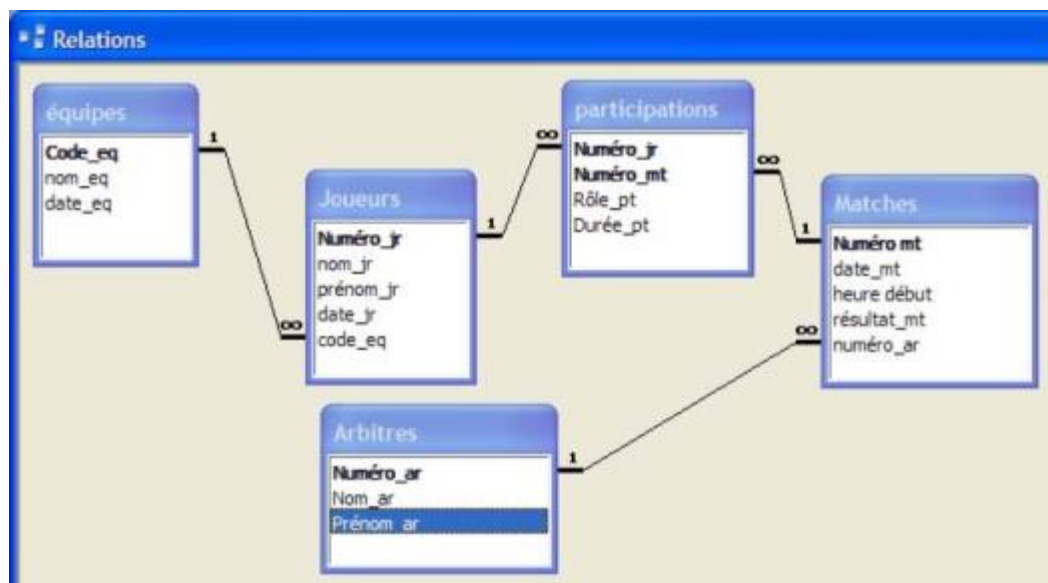
- ☐ Doit être définie par une seule colonne
☒ Peut être une clé étrangère dans une autre table
☐ Ne peut être que de type numérique

❸ Dans une base de données relationnelle :

- ☒ Il existe des liens entre les différentes tables de la base
☐ Les liens entre les tables s'organisent d'une manière hiérarchique
☒ Les liens peuvent être de différents types (1,1) ou (1, n)

Exercice N°2 : (5 Points)

Traduire cette représentation graphique en représentation textuelle :



Equipe (code-eq, nom-eq,date-eq)

Joueur (Numéro-jr, nom-jr, prénom-jr, date-jr, **code-eq#**)

Participation (Numéro-jr#,Numéro-mt#, Rôle-pt, Durée-pt)

Matches (Numéro-mt, date-mt, heure-deb, résultat-mt, **Numéro-ar#**)

Arbitre (Numéro-ar, Nom-ar, Prénom-ar)

Exercice3 :

Corriger les commandes de création de tables suivantes :

CREATE TABLEAU Personne (Nom VARCHAR (40) PRIMARY KEY, Prénom VARCHAR (40) PRIMARY KEY, Adresse VARCHAR (50), E-mail VARCHAR (50), Téléphone DECIMALE (10,3) DEFAULTT 1) ;	CREATE TABLE Personne (Nom VARCHAR (40), Prénom VARCHAR (40), Adresse VARCHAR (50) NOT NULL, E-mail VARCHAR (50), Téléphone INT (8) DEFAULTT 11111111, PRIMARY KEY(Nom, Prénom);
--	---

CREATE TABLE Voiture (immat int(3) , Marque VARCHAR (15) NOT NULL, Modèle VARCHAR (20), Puis_CH VARCHAR (2) DEFAULT 4 CHECK (Pui_CH >=4)) ;	CREATE TABLE Voiture (immat Varchar(10) PRIMARY KEY , Marque VARCHAR (15) NOT NULL, Modèle VARCHAR (20) NOT NULL, Puis_CH int (2) DEFAULT 4 CHECK (Pui_CH >=4)) ;
--	--

Exercice4 : **Exercice d'application :**

On considère la création des tables suivantes :

Table client :

Nom colonne	Type	Taille	Contrainte
Codeclt	Numérique	4	Clé primaire
Nom	Chaine	20	Non nulle
Prenom	Chaine	20	Non nulle
Adresse	Chaine	50	Nulle
Chiffre_affaire	Numérique	10,3	>=0

Table commande :

Nom colonne	Type	Taille	Contrainte
Code_com	Numérique	2	Clé primaire
Date_com	Date	-	Non nulle
Codeclt	Numérique	4	Clé étrangère

Question :

- Créer les tables suivantes en langage SQL.
- Ecrire les commandes SQL suivantes :
 - Ajouter la colonne Tel_clt (numérique 13) à la table client.
 - Supprimer la colonne Adresse de la table client.
 - Supprimer la clé primaire de la table client.
 - Désactiver la clé primaire de la table commande.
 - Supprimer la table client.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Serie 4 base de données

(Mode Commande)

Objectifs : Maîtriser les commandes SQL pour la création des bases de données et la manipulation des données.

Rappel :

Un langage de définition de données (**LDD**, ou en anglais DDL, Data definition language) qui permet de **modifier la structure** de la base de données.

• Un langage de manipulation de données (**LMD**, ou en anglais DML, Data manipulation language) qui constitue la partie la plus courante et la plus visible de SQL, permettant de **consulter et modifier** le contenu de la base de données.

Le mode commande a comme principal avantage l'indépendance par rapport au SGBD. La forme générale des trois commandes permettant de créer et de modifier la structure d'une base de données (CREATE, ALTER et DROP) est la même dans tous les SGBD qui utilisent SQL.

sql	
LDD	LMD
-Create :créer un objet (base,table,...)	-Select
-Alter :modifier la structure	-Insert :ajouter
-Drop :supprimer un objet (base,une table..)	-Update :mettre à jour
	-delete :supprimer

Enoncé :

1. Créer la base de données « BDgestion_deptg.. », qui est décrite par le schéma textuel simplifié suivant:

Employes (Num_Emp, Nom_emp, Fonction, Salaire, Prime, DateEmb, Num_dept#)

Département (Num_dept, Nom_dept, Ville,)

Le tableau suivant indique les noms et les types des champs de chaque table.

Champ	Description	Type
--- Table Employes ---		
1 Num_Emp	Numéro de l'employé	Entier (auto-incrémenté)
2 Nom_emp	Nom et prénom ou raison social	Chaine de 35 caractères
3 Fonction	Type du compte	Chaine de 25 caractères (Fonction :Ingénieur, Vendeur, Analyste,Président,Directeur)
4 Salaire	Salaire de l'employé	Décimal (10,3) supérieur à 0
5 Prime		Décimal (10,3)
6 DateEmb	Date d'embauche de l'employé	Date (inférieur au date 12-02-2023)
7 Num_dept	Numéro de département	Chaine de 10 caractères
--- Table Département ---		
1 Num_dept	Numéro de département	Chaine de 10 caractères
2 Nom_dept	Nom du département	Chaine de 40 caractères
3 Ville	Intitulé de la ville	Chaine de 40 caractères

2.b) Insérer dans la table « Employes » les lignes suivantes :

Num_Emp	Nom_emp	Fonction	Salaire	Prime	DateEmb	Num_dept
1	Mohammed	Ingénieur	2500	100	2010-05-20	20
2	Saleh	Vendeur	1700	300	2020-11-04	30
3	Sami	Directeur	3000	500	2005-06-13	30
4	Taher	Analyste	4000	100	2001-05-12	20
5	Malek	Président	5000	300	1999-01-01	10
6	Walid	vendeur	1500	80	2002-10-12	30
7	Wael	Directeur	2300	200	2001-12-12	20

Travail demandé :1-Création de table en respectant les contraintes d'intégrités(de domaine, de table et référentiels)

Q0-Créer les tables Employés et départements.

II-La projection avec sql :SELECT liste des attributs FROM nom de relation ;

Q1 :Donner les noms de tous les employés et le salaire de chacun d'eux ?

.....

Q2 :La liste de tous les départements dans lesquels travaille au moins un employé ?

.....

Q3 :Liste de tous les enregistrement de la relation Département ?

.....

III-La projection avec sql :SELECT liste des attributs FROM nom de relation WHERE condition ;

Q4 :Quels sont les employés travaillant dans le département 30 ?

.....

Q5 :Quels sont les numéros et les noms des employés travaillant dans le département 30 ?

.....

Q6 :Quels sont les numéros et les noms des employés du département 30 et qui ont un salaire >2000 ?

.....

Q7 :Quels sont les fonctions exercés par les employés des départements 20 et 30 en éliminant les lignes en double ?

.....

Q8 : Quels sont les fonctions dont Le salaire est compris entre 2000 et 4000 ?

.....

.....

.....

.....

Q9 : Quels sont les employés du département numéro 30 dont le nom commence par 'S' ?

.....

.....

.....

.....

Q10 :Quels sont les employés du département numéro 20 dont la fonction n'est ni 'Ingénieur' ni 'Analyste' ?

.....

.....

.....

.....

IV-La jointure avec sql :

Q11 :Quels sont les numéros et les noms des employés qui travaillent à 'Tunis' ?

.....

.....

.....

.....

Q12 :Quels sont les noms,les fonctions et les salaires des employés du département 'Recherche' dont le salaire est supérieur à 2400 ?

.....

.....

.....

.....

V-Les fonctions

Q13 :Quel est le salaire maximum,le salaire minimum et la différence entre ces deux valeurs ?

.....

.....

.....

Q14 :Quel est le nombre d'employés percevant une prime ?

.....

.....

.....

.....

Q15 :Quel est le nombre des différentes fonctions dans le département 20

.....

.....

.....

.....

Q16 :Quel est la moyenne des salaires du département 20 ?

.....

.....

V-Groupement

Q17 : Quel est le nombre des employés de chaque département

.....

.....

.....

.....

Q18 : Quel est la somme des salaires de chaque département

.....

.....

.....

.....

Q19-afficher la somme des salaires de chaque département >5000

.....

.....

.....

.....

VI-Sous requête : requête non corrélée

Q20 Afficher les nompEmployé dont le salaire est supérieur au **moyenne** des salaire

.....

.....

.....

.....

VII-Tri du résultat d'une requête :

Q21 :liste des employés du département 20 par ordre décroissant des salaires ?

.....

.....

.....

Q22 :La liste par ordre alphabétique des employés du département 30 ?

.....

.....

.....

.....

Q23 :La liste des employés triés selon un **ordre alphabétique** de leurs **fonctions** et à l'intérieur de chaque fonction les trier selon un **salaire décroissant** ?

.....

.....

.....

.....

VIII-Requête de mise à jour

Q24 : Augmenter les salaires des ingénieurs de 15%

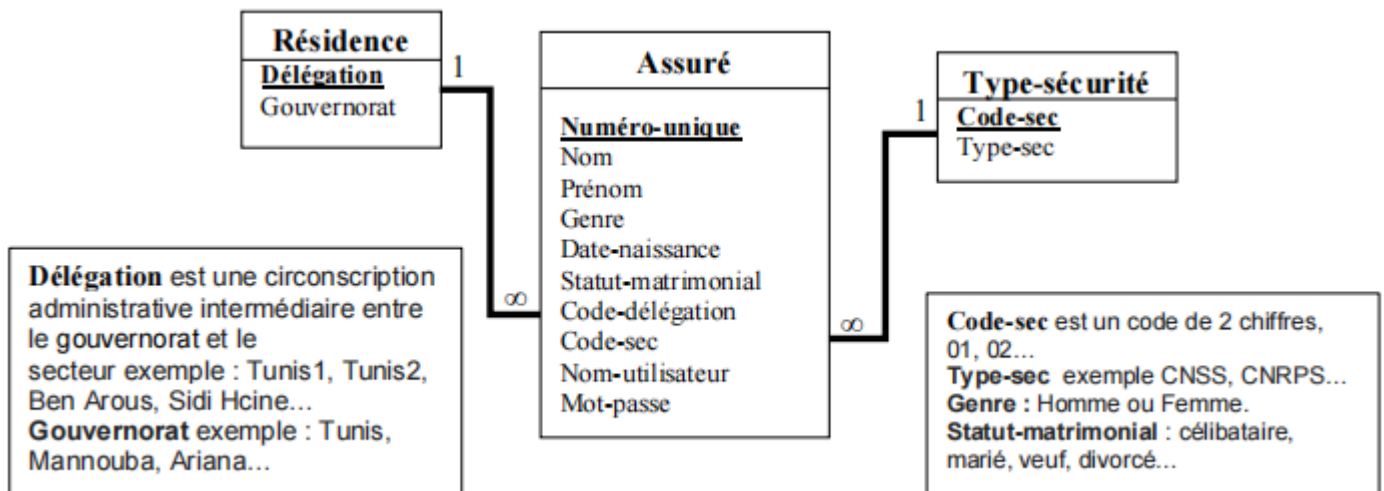
.....

.....

.....

Serie5 LDD-LMD Fonctions chaines et date

Soit la base de données réduite suivante relative à la gestion de la sécurité sociale d'un assuré



Questions :

En utilisant le langage SQL répondre aux questions suivantes :

1. Afin d'optimiser la Base de données on veut changer les valeurs de la colonne Genre par 'H' au lieu de 'Homme' et 'F' au lieu de 'Femme', on veut aussi changer le type de la même colonne vers un seul caractère, écrire les requêtes SQL qui permettent de répondre à cette demande :

.....

.....

.....

2. Faites une projection de la table Résidence :

.....

3. Liste des assurés (Numéro, nom et prénom) qui sont divorcés et le genre est Homme :

.....

.....

.....

4. Nom et prénom des assurés (affiché en une seule colonne avec l'alias 'Nom et prénom') dont le nombre de caractères du mot de passe est inférieur ou égal à 4 :

.....

.....

5. Liste des assurés (numéro, nom et prénom) qui sont de Sidi Hcine et dont le type de sécurité sociale est CNPRS trié par ordre alphabétique des noms puis alphabétique décroissant des prénoms :

.....

.....

6. Le nombre d'assurés qui sont nés l'année 2000 :

.....

.....

7. Le nombre d'assurés par délégation, afficher seulement ceux qui dépassent 100 :

.....

.....

8. La liste des assurés (Numéro, nom et prénom, afficher que les 3 premiers caractères du nom) dont le nom commence par 'A', qui sont du gouvernorat de 'Mannouba' et ont le type de sécurité social 'CNSS'

.....
.....
.....

9. La liste des assurés (numéro, nom et prénom) qui ont le même mot de passe que 'Bennour Ahmed' :

.....
.....
.....

10. La liste des assurés (Nom-utilisateur, nom et prénom) qui ont les mêmes 3 premiers caractères du nom d'utilisateur que celui de 'Bennour Ahmed' :

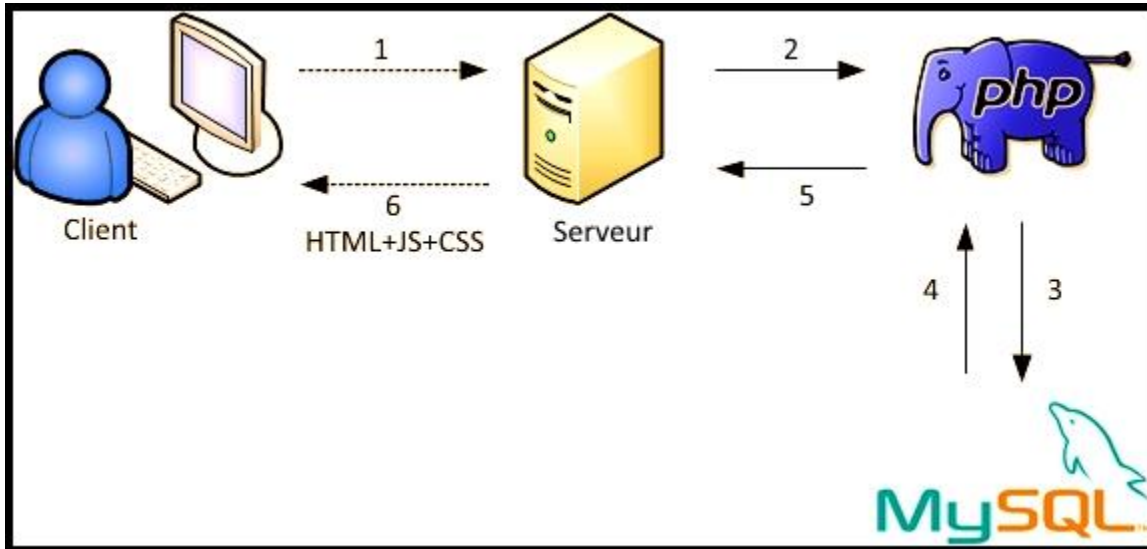
.....
.....
.....

PHP

I. INTRODUCTION :

1) Définition et principe de fonctionnement :

Un site Web dynamique est un site dont les pages peuvent être générées à 'la volée' (dynamiquement) en fonction d'une demande d'un utilisateur.



- 1) Le client demande une page web avec du code PHP.
- 2) Le serveur Web (Apache, IIS...) demande à l'interpréteur PHP d'exécuter le code PHP.
- 3) L'interpréteur PHP demande les données à partir du SGBD (MySQL, PostgreSQL...).
- 4) Le SGBD retourne les données
- 5) L'interpréteur PHP génère le code HTML et le renvoi au serveur.
- 6) Le serveur envoie la page demandée au client (**Sans aucun code PHP**).

2) Environnement de Travail :

Pour pouvoir développer en PHP, on a besoin :

- ☞ d'un serveur Web (**Apache**, IIS, Nginx...),
- ☞ de l'interpréteur **PHP** et
- ☞ si notre site web contient une base de données, on aura besoin aussi d'un SGBD (**MySQL**, Oracle, PostgreSQL...).

Dans la suite du cours, on utilisera le paquetage « **EasyPHP** » qui installera **Apache**, **Php**, **MySQL** et **phpMyAdmin** qui est une interface Web pour la gestion des bases de données MySQL.

II. LE LANGAGE PHP :

1) Introduction : Règles à respecter

Une page **PHP** est en fait une simple page **HTML** qui contient des instructions en langage **PHP**.

⚠ Une page web qui comporte le moindre petit bout de code **PHP**, doit avoir l'extension **".php"**.

En clair : si vous avez une page nommée **index.htm** et que vous y insérez du code **PHP**, il vous faudra la renommer en **index.php**

⚠ Un code **PHP** doit être délimité par les deux marqueurs **<?php** et **?>**

⚠ Les variables en **PHP** commencent obligatoirement par le signe **\$**.

⚠ Chaque instruction en **PHP** se termine obligatoirement par un point-virgule **;**

⚠ Pour les variables, **PHP** est sensible à la casse (**\$somme**, **\$SOMME** et **\$Somme** sont 3 variables différentes)

⚠ Pour pouvoir tester les fichiers **PHP**, on doit les enregistrer dans un dossier à créer dans le répertoire d'hébergement du serveur (Dossier **www** de **EasyPHP**)

2) Les commentaires en PHP :

Pour commenter le code on utilise `// cmt mono-ligne` ou `/* cmt multi-lignes */`

3) Les variables :

a. Les différents types de variables :

- Les chaînes de caractères (**string**) (*entre guillemets doubles ou simples*)
- Les nombres entiers (**integer**, **int**)
- Les nombres décimaux (**double**, **float**)
- Les booléens (**boolean**, **bool**)
- Les tableaux (**array**)

b. Conversion de types :

- La fonction **settype** permet de définir le type d'une variable.
 - Exemple : **settype(\$somme, "integer");**
- La fonction **settype** renvoie **TRUE** en cas succès et **FALSE** dans le cas contraire.
- En précédant les variables à convertir par des clauses (**type**).
 - Exemple : **\$somme=(integer) \$somme;** //Renvoie 0 s'il n'est pas possible de faire la conversion

c. Fonctions de manipulation de variables :



- **isset()** : Permet de savoir si une variable existe ou pas. Elle retourne **TRUE** si la variable existe, sinon **FALSE**.
- **var_dump()** : Affiche les informations d'une variable (Type / Valeur)

4) Les opérateurs :

Arithmétiques : `+, -, /, *, %`

Relationnels (de comparaison) : `<, <=, >, >=, ==, !=`

Logiques : `AND (ou &&), OR (ou ||), !, XOR`

5) L'instruction « **echo** » :

L'instruction **echo** permet d'afficher du texte dans la page.

L'opérateur de concaténation est le point.

Exemple 1 : `<?php echo 'Ceci est du texte'; ?>`

On peut même inclure des balise HTML :

Exemple 2 : `<?php echo "Ceci est du <b>texte</b>"; ?>`

→ le mot **texte** s'affichera en gras

Affichage du contenu d'une variable :

Exemple 3 : `$somme=50 ;`

`<?php echo $somme; ?>` //Affichera : 50

Exemple 4 : `$somme=50 ;`

`<?php echo "Somme= " . $somme; ?>` //Affichera : Somme= 50

`<?php echo "Somme= ", $somme; ?>` //Affichera : Somme= 50

`<?php echo "Somme=$somme"; ?>` //Affichera : Somme= 50

`<?php echo 'Somme=$somme'; ?>` //Affichera : Somme=\$somme

6) Les Structures de contrôle :

<pre><?php \$a=5; \$b=3; if (\$a>=\$b) { echo'a est plus grand que b'; } else { echo'a est plus petit que b'; } ?></pre>	<pre><?php \$i=0; \$x=0; while (\$i<10) { \$i++; \$x+=\$i; } echo'\$x ' ; ?></pre>
<pre>\$i=0; \$x=0; do { \$i++; \$x+=\$i; } while (\$i<10);</pre>	<pre>var val=8; switch (val){ case 1: case 3: case 4: echo'1 ou 3 ou 4'; break; default : echo'Val par défaut'; break; }</pre>
<pre>for (\$i=1 ; \$i<10 ; \$i++){ echo '4SI'.\$i.
; }</pre>	

7) Récupérer les données en PHP :

Pour récupérer des données envoyées par un formulaire, on utilise l'une des deux syntaxes suivantes :

`$_GET['nomobjet']` ou `$_POST['nomobjet']`

Récupérer les données en PHP :

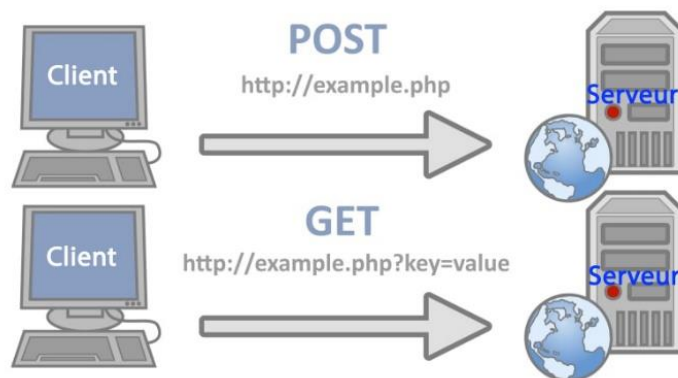
Pour récupérer des données envoyées par un formulaire, on utilise l'une des deux syntaxes suivantes :

`$_GET['nomobjet']` ou `$_POST['nomobjet']`

Selon la méthode utilisée dans la balise `<form>` :

`<form method="POST">` ou `<form method="GET">`

Les deux méthodes **GET** et **POST** sont utilisées pour transférer des données du **client** au **serveur** avec le protocole **HTTP**. La différence clé entre les méthodes POST et GET est que GET transporte le paramètre dans la chaîne d'URL, **tandis que** POST transporte le paramètre dans le corps du message, ce qui le rend plus sûr le transfert des données du **client** au **serveur** avec le protocole **http**.



Exemple de récupération des données de formulaire avec la méthode GET :

Fichier **4_a.html**

```
<!DOCTYPE html>
<head>
  <meta charset="UTF-8"><title>Document</title>
</head>
<body><form action="4_a.php" method="GET">
  <fieldset><table>
    <tr><td>Votre nom</td><td><input type="text" name="nom"></td> </tr>
    <tr><td>Votre age</td><td><input type="text" name="age"></td></tr>
  </table>
  <input type="submit" value="ENVOYER">
  </fieldset>
</form></body></html>
```

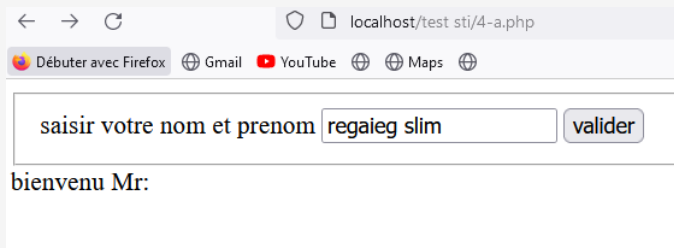
Fichier **4_a.php**

```
<!DOCTYPE html>
<html Lang="en">
<head>
  <meta charset="UTF-8">
  <title>Document</title>
</head>
<body>
  <h1>bonjour, <?php echo $_GET["nom"] ?></h1>
```

```
<h2>votre age est: <?php echo $_GET["age"] ?></h2>
</body>
</html>
```

Exemple2: Exemple de récupération des données de formulaire avec la méthode POST :
Fichier 4-a.php

```
<!DOCTYPE html>
<html Lang="en">
<head>
  <meta charset="UTF-8">
  <meta http-equiv="X-UA-Compatible" content="IE=edge">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
<form action="4-a.php" method="POST">
  <fieldset>
    <table>
      <tr>
        <td>saisir votre nom et prenom</td>
        <td><input type="text" name="np"></td>
        <td><input type="submit" value="valider"></td>
      </tr>
    </table>
  </fieldset>
</form>
<?php
$np=$_POST["np"];
echo("bienvenu Mr: $np");
?>
</body></html>
```



Les fonctions en PHP :

```
<?php
for ($i=1 ; $i<10 ; $i++){
echo 'Le cube de '.$i.' est égal à :'.Cube($i).'  
>';
}
function Cube($x){
return $x*$x*$x;
} ?>
```

8) Les tableaux :

a. Introduction :

Les tableaux en PHP peuvent contenir des indices de type **integer** et/ou **string**.

Il est possible de stocker des éléments de types différents dans un même tableau.

b. Déclaration et initialisation d'un tableau :

Exemple.a : `$T[0]="P"; $T[]=2; $T[55]=true; $T["Classe"]="4SI"; $T[]="TIC";`

Exemple.b : `$T=array("P",2);`

Exemple.c : `$T=array("Janvier"=>31, 2=>28, "Mars"=>31, 30);`

c. Parcourir un tableau :

La structure de contrôle **foreach** existe tout spécialement pour les tableaux. Elle fournit une manière pratique de parcourir un tableau.

Exemple d'utilisation de foreach :

soit : `$T[0]="P"; $T[]=2; $T["Classe"]="4SI"; $T[]="TIC";`

	Code PHP	Résultat de l'exécution
Valeur sans indice	<code>foreach (\$T as \$Valeur){ echo \$Valeur . "
"; }</code>	P 2 4SI TIC
Indice & Valeur	<code>foreach (\$T as \$i => \$Valeur){ echo \$i . " --> " . \$Valeur . "
"; }</code>	0 --> P 1 --> 2 Classe --> 4SI 2 --> TIC

d. Fonctions pour les tableaux :**Fonctions pour parcourir un tableau****Fonction Description**

sizeof() Retourne le nombre d'éléments dans un tableau

sort() Trie un tableau en ordre croissant

rsort() Trie un tableau en ordre décroissant

N.B. : sort() et rsort() assignent de nouveaux indices pour les éléments du tableau \$T. (*et effacent les anciens indices*)

9) Les fonctions sur les chaînes de caractères :

☞ **substr()** : **\$ch2=substr(\$ch1,début, taille) ;**

//Retourne une portion de \$ch1, spécifiée avec le début et la longueur taille.

Lorsqu'un **début** négatif est spécifié, la chaîne retournée commencera au caractère numéro **début** à partir de la fin de la chaîne.

Si **taille** est omise, la sous chaîne commençant à partir de **début** jusqu'à la fin sera retournée.

☞ **trim()** : **\$ch=trim (\$ch) ;**

// Supprime les espaces en début et fin de chaîne et retourne la chaîne nettoyée.

☞ **strlen()** : **\$x=strlen (\$ch) ;**

// Retourne la longueur de la chaîne string.

☞ **implode()** : **\$ch= implode ("separateur", \$T3) ;**

//Retourne une chaîne constituée de tous les éléments du tableau, pris dans l'ordre, transformés en chaîne, et séparés par "separateur".

☞ **explode()** : **\$T3=explode ("separateur", \$ch) ;**

//Retourne un tableau qui contient les éléments de la chaîne \$ch extraite en utilisant le "separateur".

☞ **str_replace()** : **\$ch=str_replace("modèle", "remplacement", \$ch) ;**

//Remplace toutes les occurrences de "modèle" dans chaîne \$ch par "remplacement".

PHP & MYSQL

L'utilisation de MySql avec Php s'effectue en cinq étapes :

Étape 1. Connexion au serveur MySql :

mysql_connect("localhost", "root");

Avec gestion des erreurs de la connexion au SGBD MySql:

mysql_connect("localhost", "root") or

exit("Problème de connexion à MySql :".mysql_error());

Étape 2. Sélection de la base de données :

mysql_select_db("BD123456");

Avec gestion des erreurs de la connexion à la base de données :

mysql_select_db("BD123456") or

die("Problème de connexion à la BD :". mysql_error());

Envoyer une requête à la base de données :

```
$R=mysql_query("SELECT * FROM Revue");
```

	mysql_query(\$Q);	
	SELECT (Requêtes de sélection)	UPDATE, INSERT, DELETE... (Requêtes de mise à jour)
Exemple:	\$Q="SELECT * FROM Eleves"; \$R=mysql_query(\$Q);	\$Q= "DELETE FROM Eleves WHERE MoyGen>=10"; \$R=mysql_query(\$Q);
Résultat :	Échec	FALSE
	Succès	Une ressource (≡ Un fichier, dont chaque enregistrement/ligne est un tableau)
Déterminer le nombre de lignes	Retournées dans la Ressource	\$Nb=mysql_num_rows(\$R);
		Affectées par la requête de MAJ
		\$Nb=mysql_affected_rows(); <i>// Sans paramètres</i>

Étape 3. Exploitation du résultat de la requête SELECT :

```
$Tab=mysql_fetch_array($R)
```

mysql_fetch_array(\$R) renvoie un tableau qui contient une ligne de la ressource **\$R** et déplace le pointeur vers la ligne suivante.

Le résultat retourné par **mysql_fetch_array(\$R)** est un tableau avec des indices associatifs et numériques en même temps.

S'il n'y a plus de lignes dans la ressource **\$R**, **mysql_fetch_array(\$R)** retourne **FALSE**.

//Exemple de parcours complet d'une ressource

```
While ($Tab=mysql_fetch_array($R)){  
echo $Tab["Champ1"]." | ".$Tab["Champ2"];}
```

Étape 4. Fermeture de la connexion :

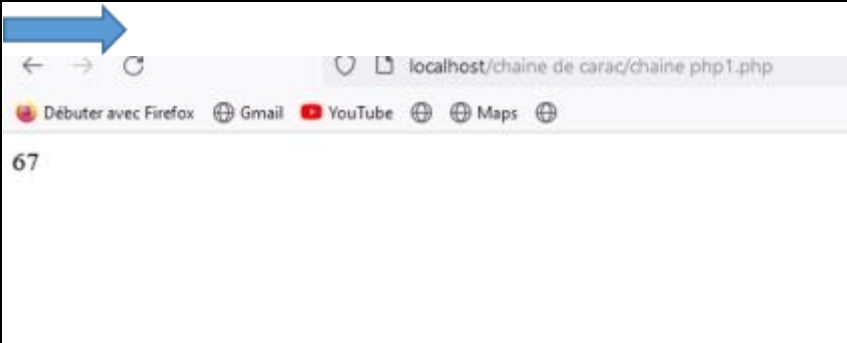
```
mysql_close();
```

La fermeture de la connexion est optionnelle parce qu'elle sera détruite automatiquement lorsque le script PHP termine son exécution.

Les chaînes en PHP

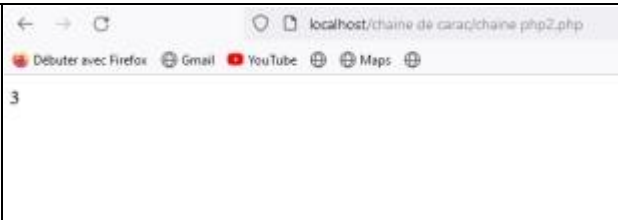
Activité1 : Taille d'une chaîne

Pour connaître le nombre de caractères contenu dans une chaîne, vous pouvez utiliser la fonction `strlen()`

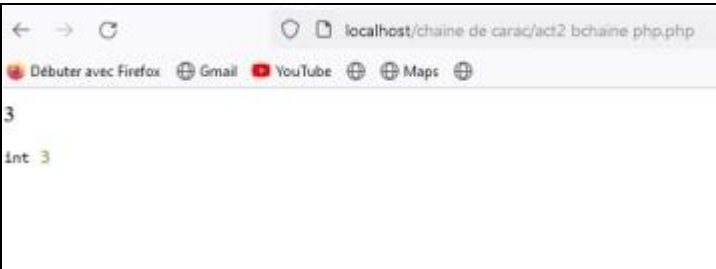
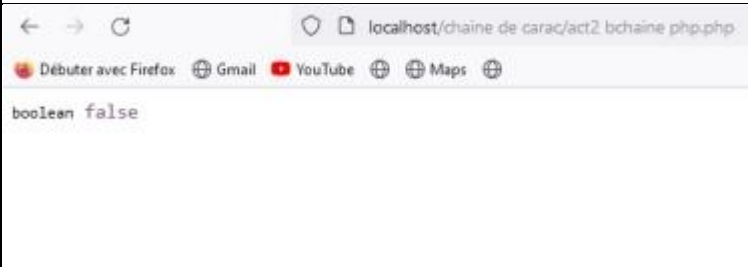
<pre><?php \$str='tounsi'; echo strlen(\$str);// 6 \$str=' ef gh '; echo strlen(\$str);// 7 ?></pre>	
--	--

Activité 2 : Trouver la première position

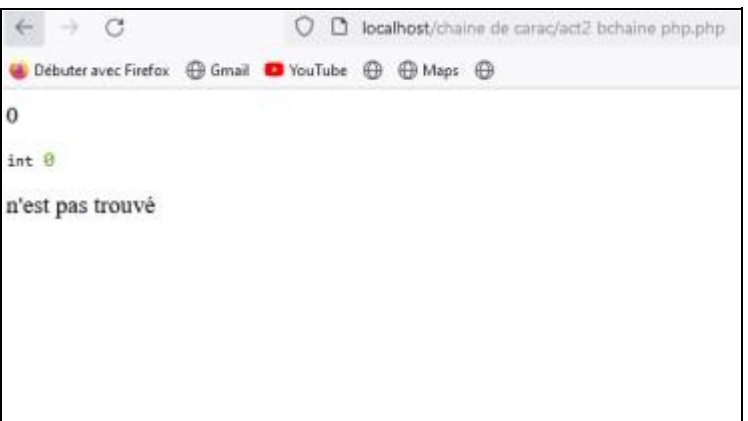
La fonction `strpos()` trouve la position de la première occurrence d'un caractère dans une chaîne.

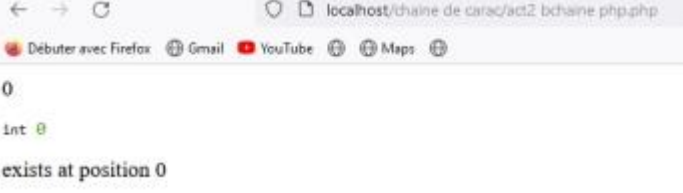
<pre><?php \$machaine = 'nom-composé'; echo(strpos(\$machaine, '-'));//3 ?></pre>	
---	--

b)

<pre><?php \$machaine = 'nom-composé'; \$pos=strpos(\$machaine, '-');// echo(\$pos); var_dump(\$pos); ?</pre>	
<pre><?php \$machaine = 'nomcomposé'; \$pos=strpos(\$machaine, '-');// echo(\$pos); var_dump(\$pos); ?></pre>	

c)

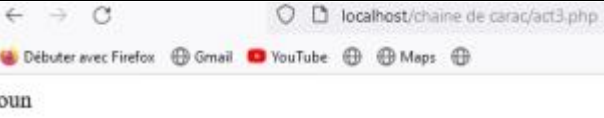
<pre><?php \$machaine = 'nomcomposé'; \$pos=strpos(\$machaine, 'n');// echo(\$pos); var_dump(\$pos); if (\$pos == false) { echo "n'est pas trouvé "; } else { echo " exists at position \$pos"; }</pre>	
--	--

<pre>} ?></pre>	
<pre><?php \$machaine = 'nomcomposé'; \$pos=strpos(\$machaine,'n');// echo(\$pos); var_dump(\$pos); if (\$pos === false) { echo "n'est pas trouvé "; } else { echo " exists at position \$pos"; } ?></pre>	

La fonction strpos() retourne false si elle ne trouve pas. Remarquez l'utilisation de l'opérateur === qui signifie que la comparaison se fera sur la valeur et le type. En PHP le zéro peut prendre la valeur false. Si une valeur est trouvée en position zéro elle ne doit pas être interprétée comme pas trouvée.

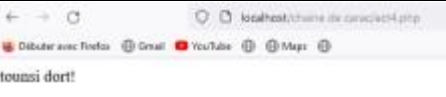
Activité 3 : Extraire une sous-chaîne

la fonction substr() permet d'extraire une chaîne (à partir d'une position précise, en partant de la fin, sur une quantité bien définie de caractères...). Il accepte trois paramètres : la chaîne d'entrée, la position de départ, la position de fin ou la quantité. Si aucune chaîne est trouvée, la fonction retournera false.

<pre><?php echo substr('tounsi', 1, 3)."
"; // affiche 'oun' ?></pre>	
--	--

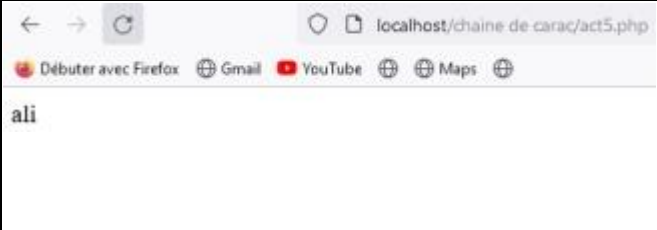
Activité 4 : Remplacer un motif dans une chaîne

La fonction str_replace() permet de remplacer un motif dans une chaîne. Il faut indiquer la sous-chaîne à rechercher, puis la sous-chaîne de remplacement et enfin la chaîne de référence où faire le remplacement.

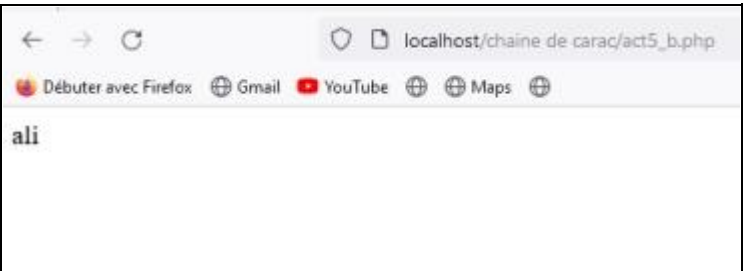
<pre><?php \$recherche = "ali"; // La sous-chaîne recherchée \$remplacement = "tounsi"; // La sous-chaîne de remplacement \$chaîne = "ali dort!"; // chaîne de référence echo str_replace(\$recherche,\$remplacement,\$chaîne); ?></pre>	
--	--

Activité5 : Élagage d'une chaîne en PHP

L'élagage se fait avec la fonction `trim()` et consiste à retirer les caractères blancs avant et après un texte.

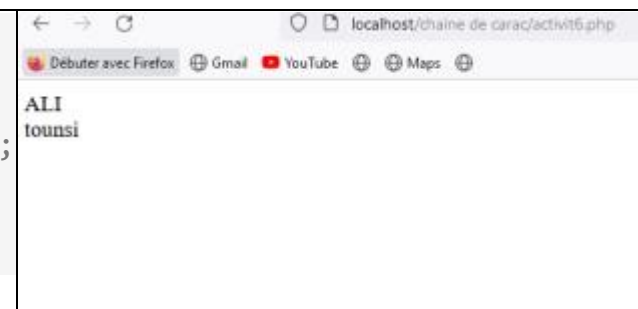
<pre><?php \$chaine = " ali "; echo trim(\$chaine); // Les espaces avant et après seront supprimés ?></pre>	
---	--

b) on peut spécifier une liste de caractères à supprimer en deuxième paramètre de la fonction `trim()`.

<pre><?php \$chaine = "*****ali!!!!!!"; echo trim(\$chaine, "*!"); // Les caractères spécifiés en deuxième paramètre seront supprimés ?></pre>	
--	--

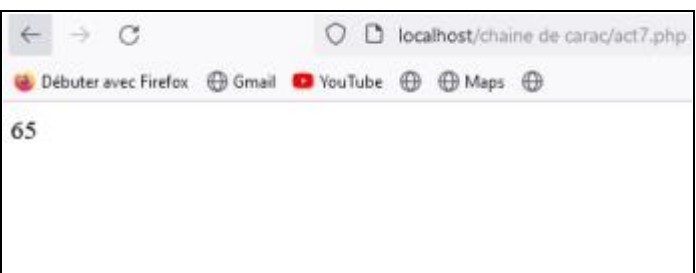
Activité6 : strtoupper() et strtolower()

La conversion minuscule en majuscule se fait en PHP avec la fonction `strtoupper()` et l'inverse avec la fonction `strtolower()`.

<pre><?php \$chaineMinuscule = "ali"; echo strtoupper(\$chaineMinuscule). "
"; \$chaineMajuscule = "TOUNSI"; echo strtolower(\$chaineMajuscule); ?></pre>	
--	--

Activité 7 : `ord($car)`: retourne le code ASCII du caractère `$car`.

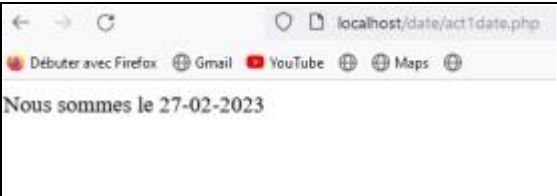
`chr($int)`: retourne le caractère correspondant au code ASCII `$int`.

<pre><?php \$car="A"; echo(ord(\$car)); // retourne le code ASCII du caractère \$car. ?></pre>	
--	--

Fonction date en PHP

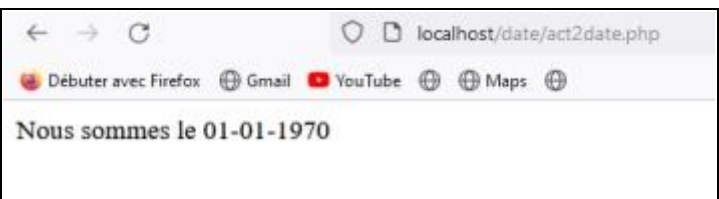
La fonction date() vous renvoie l'heure locale de votre serveur.

Activité 1

<pre><?php \$date = date("d-m-Y"); echo ("Nous sommes le \$date"); ?></pre>	
---	--

Activité 2

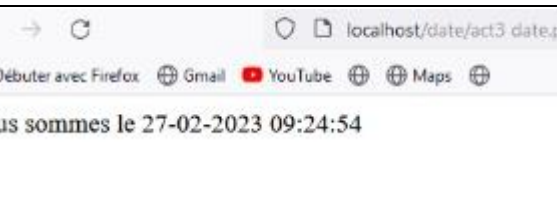
Le premier paramètre concerne le format de la date (voir les différents formats en dessous).
La fonction date() accepte un deuxième argument qui correspond à un nombre de secondes écoulées depuis le début de l'époque UNIX, (1er janvier 1970 00:00:00 GMT). Dans l'exemple ci-dessous la fonction affiche la date qu'il était après 0 seconde écoulé depuis le 01/01/1970.

<pre><?php \$date = date("d-m-Y", 0); echo ("Nous sommes le \$date"); ?></pre>	
---	--

Activité 3 : Format de date

Les options permettent une bonne représentation d'une date et heure.

Exemple :

<pre><?php \$date = date("d-m-Y H:i:s") ; echo ("Nous sommes le \$date"); ?></pre>	
--	--

Les différentes options sont :

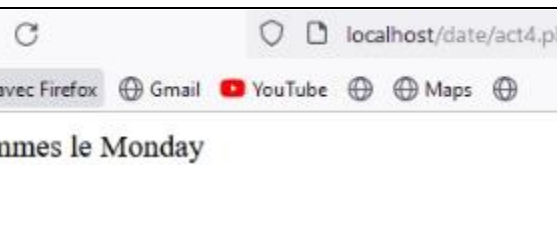
Jour

J : Jour du mois sur deux chiffres sans les zéros initiaux 1 à 31

d : Jour du mois sur deux chiffres avec un zéro initial en fonction du jour 01 à 31

l : (L minuscule) Jour de la semaine en anglais Sunday à Saturday

w : Jour de la semaine au format numérique 0 (dimanche) à 6 (samedi) z : Jour de l'année 0 à 366

<pre><?php \$date = date("l"); echo ("Nous sommes le \$date"); ?></pre>	
---	--

Semaine

W : Numéro de semaine dans l'année (les semaines commencent le lundi) Exemple : 42 (la 42ème semaine de l'année)

Mois

F : Mois, textuel, version longue; en anglais, comme January ou December January à December

m : Mois au format numérique, avec zéros initiaux 01 à 12

n : Mois sans les zéros initiaux 1 à 12 t Nombre de jours dans le mois 28 à 31.

Année

L : Est ce que l'année est bissextile 1 si bissextile, 0 sinon.

Y : Année sur 4 chiffres Exemples : 1999 et 2003

y : Année sur 2 chiffres Exemples : 99 et 03.

Heure

a : Ante méridien et Post méridien (minuscules) am ou pm

A : Ante méridien et Post méridien (majuscules) AM ou PM

g : Heure (format 12h) sans les zéros initiaux 1 à 12

G : Heure (format 24h) sans les zéros initiaux 0 à 23

h : Heure (format 12h) avec les zéros initiaux 01 à 12

H : Heure (format 24h) avec les zéros initiaux 00 à 23

s : Secondes avec zéros initiaux 00 à 59

i : Minutes avec zéros initiaux 00 à 59

checkdate()

La fonction checkdate() permet de vérifier si une date est valide. La liste des paramètres doit respecter cet ordre : mois, jour, année

```
<?php
$result = checkdate(13, 10, 2002);
// 10-13-2002
if( $result == true ){
echo 'la date est valide';
}else{
echo 'la date n\'est pas valide';}
?>
```

Les tableaux en PHP

Les tableaux permettent de rassembler plusieurs valeurs. Pour créer un tableau vide on peut utiliser la fonction `array()`

Activité1: créer un tableau vide

<pre><?php \$tab = array(); echo \$tab; ?></pre>	
--	--

Pour son remplissage, vous pouvez utiliser deux types de tableau, les tableaux indexés et les tableaux associatifs.

Le tableau indexé

C'est un tableau qui contient une liste d'éléments. Chaque élément est séparé par une virgule. Il n'y a pas à déclarer la taille du tableau, elle sera gérée automatiquement par PHP. Dans un tableau indexé, chaque valeur est repérée par une clé numérique.

La clé zéro représente la première valeur.

Pour lire une valeur d'un tableau indexé, on appelle la variable avec, entre les crochets `[]`, le numéro de la clé correspondant à la valeur.

Activité2 :

<pre><?php \$tab = array(1,2,"ali"); echo \$tab[0]; // affichera 1 ?></pre>	
---	--

Activité 3 : Le tableau associatif

C'est un tableau plus intuitif qui permet de retrouver plus facilement les valeurs. En effet, dans un tableau associatif, chaque **valeur** est repérée par une **chaîne de caractères**.

On peut créer un tableau associatif rempli de valeurs. Les valeurs sont associées aux clés par la syntaxe `=>`

<pre><?php \$tab = array("taille" => 170, "age" => 30, "prenom" => "ali"); echo \$tab; ?></pre>	
--	--

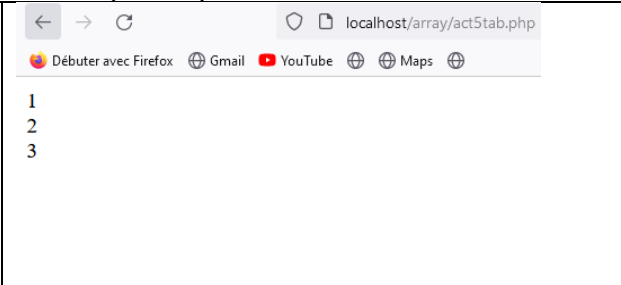
Pour lire une valeur d'un tableau associatif, on appelle la variable avec, entre les crochets `[]`, la clé sous forme de chaîne correspondant à la valeur.

<pre><?php \$tab = array("taille" => 170, "age" => 30, "prenom" => "ali"); echo \$tab["taille"]; ?></pre>	
--	--

Activité 4:Parcourir un tableau

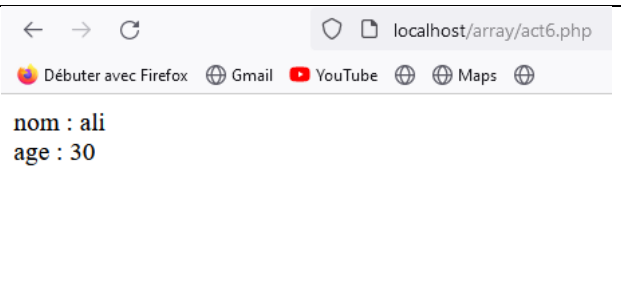
Php propose un moyen simple pour parcourir un à un tous les éléments d'un tableau, il s'agit de l'instruction `foreach()`. Il existe deux syntaxes possibles :

– Voici la syntaxe pour parcourir un tableau indexé (indiqué):

<pre><?php \$tab = [1,2,3]; foreach(\$tab as \$valeur){ echo \$valeur."
"; } ?></pre>	
--	--

Activité 5 : La fonction `foreach()` commence par lire la première valeur puis elle lit, à chaque itération, la valeur suivante du tableau. Chaque valeur du tableau que la fonction parcourt est assignée à `$valeur`.

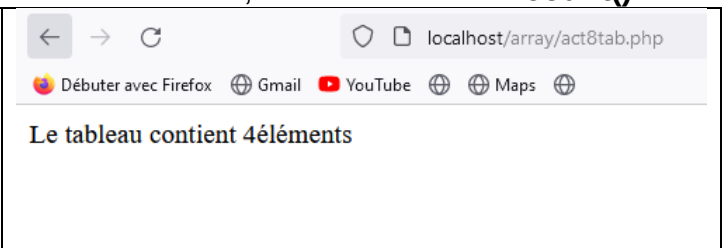
– Voici la syntaxe pour parcourir un tableau associatif :

<pre><?php \$tab = ['nom'=>'ali', 'age'=>30]; foreach(\$tab as \$cle=>\$valeur){ echo \$cle." : ".\$valeur."
"; } ?></pre>	
--	---

Le fonctionnement est identique à la précédente avec en plus, la récupération pour chaque valeur, de la clé du tableau assignée à `$cle`.

Activité 6: Taille du tableau

Pour compter le nombre d'éléments contenu dans un tableau, utilisez la fonction **`count()`**.

<pre><?php \$tab = [1,2,3,4]; echo "Le tableau contient ".count(\$tab)."éléments"; ?></pre>	
---	--

Activité7 : Parcourir un tableau avec la boucle FOR

La fonction

`count()` peut aussi être utile pour parcourir un tableau. Pour une meilleur gestion du pointeur d'un tableau, il est recommandé d'utiliser la boucle `for()`. En effet la

boucle

`foreach()` renvoie **toutes les valeurs** une à une depuis le début, tandis qu'avec la boucle

`for()`, la recherche peut commencer sur un index précis du tableau et terminer avant la fin. De plus le pointeur peut avancer de **plusieurs éléments**.

<pre><?php \$tab = [1,2,3,4,5,6,7,8,9]; for(\$i=3; \$i<count(\$tab)-2; \$i+=2){ echo \$tab[\$i]."
"; } ?></pre>	
---	--

La lecture commence à 4 (le premier élément du tableau est indexé à zéro) et se termine à 7 (count(\$tab)-2). Le pointeur avance de 2 à chaque itération

Activité 8:

<pre><?php \$vente=array(1742,1562,1920,1239,2012,720); \$s=0; for(\$i=0;\$i<count(\$vente);\$i++){ \$s+=\$vente[\$i]; } echo("le total des ventes est".\$s); ?></pre>	
---	--

```
<?php
$vente=array('lundi'=>1742,'Mardi'=>1562,'Mercredi'=>1920,'jeudi'=>1239,'vendredi'=>2012,'samedi'=>720);
$s=0;
foreach($vente as $i=>$v){
    echo("<br> vente de $i:$v unités \n");
    $s+=$v;
}
echo("le total des ventes est".$s);
?>
```

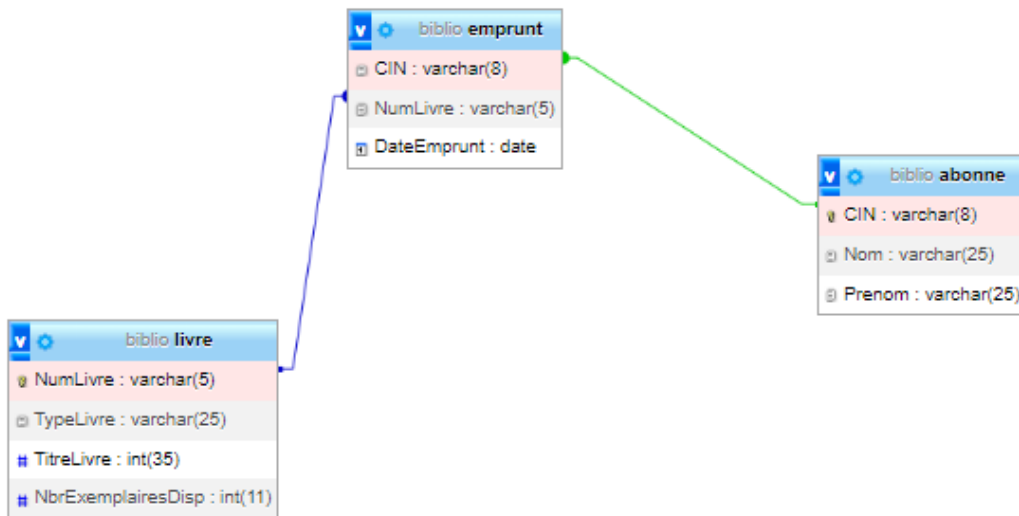
--

vente de lundi:1742 unités
 vente de Mardi:1562 unités
 vente de Mercredi:1920 unités
 vente de jeudi:1239 unités
 vente de vendredi:2012 unités
 vente de samedi:720 unités le total des ventes est9195

Application1



Soit la base des données «Biblio » suivante:



Livre (NumLivre, typeLivre, TitreLivre, NbreExemplairesDisp)

Abonné(CIN, Nom, Prenom, Classe)

Emprunt (CIN#, NumLivre#, DateEmprunt)

Travail demandé :

1-Créer la base les tables ainsi que les relations entre les tables.

2-a **Insérer** dans la table « **Livre** » les lignes suivantes :

NumLivre	TypeLivre	TitreLivre	NbrExemplairesDisp
A0001	Algorithmique	Les structures de controles	15
B0132	Base de données	SQL	22

2-b **Insérer** dans la table « **Abonne** » les lignes suivantes :

CIN	Nom	Prenom
01111111	Alaya	Ibtissem
03333333	Barhoumi	Mohamed

- c-**Insérer** dans la table « **Emprunt** » la ligne suivante :
- **Ibtissem Alaya** a emprunté le livre « **SQL** » aujourd'hui
- On souhaite créer un site web dynamique qui nous aide à la gestion des livres

Insertion d'un livre Numéro livre Type livre Titre livre Nombre d'exemplaires Ajouter	Modification code du livre A0001 Nouveau titre du livre Les str. de controle Modifier	Suppression code du livre Supprimer
--	--	--

- 3-**Insérer** un enregistrement dans la table « **Livre** » à travers ce formulaire
- A-Créer le formulaire suivant:

Insertion d'un livre

Numero livre	
Type livre	
Titre livre	
Nombre d'exemplaires	
Annuler Ajouter	

Code php:ajout.php

```
<?php
```

```
//connexion
```

```
$conx=mysqli_connect('localhost','root','','biblio') or die;
```

```
$livre=$_POST['livre'];
```

```
$type=$_POST['type'];
```

```
$titre=$_POST['titre'];
```

```
$nbex=$_POST['nbex'];
```

```
//requete
```

```
$req1="INSERT INTO `livre` (`NumLivre`, `TypeLivre`, `TitreLivre`, `NbrExemplairesDisp`) VALUES  
('$livre', '$type', '$titre', '$nbex');";
```

```
//exécution de la requête
```

```
$res1=mysqli_query($conx,$req1)or die;
```

```
//nombre de ligne affecté
```

```
if(mysqli_affected_rows($conx)!=-1)echo("Ajout de livre fait avec succès");
```

```
//fermer la connexion
```

```
mysqli_close($conx);
```

```
?>
```

A)Préparation d'environnement

Installer Xampp

- Lancer le logiciel Xampp puis démarrer les services apache et MySQL

puis cliquer sur le bouton Admin

B)Création d'une connexion à MySQL

Pour travailler avec **des enregistrements** dans MySQL, on doit d'abord créer une **connexion au serveur** à l'aide de php. Ceci est fait avec la fonction **mysqli_connect()**, qui renvoie le résultat comme une ressource de connexion. Cette ressource sera utilisée dans toutes les opérations futures de **la base de données (MySQL)**.

Syntaxe:

```
$con=mysqli_connect("hote","nom_utilisateur","mot_de_passe","nom_bd");
```

Exemple:hote:localhost,nom_utilisateur:root,mot_de_passe(pas de mot)

```
$conx=mysqli_connect('localhost','root','','biblio') or die("erreur1".mysqli_error($conx));
```

die() est un alias de la fonction `exit()` : c'est une fonction de PHP qui permet de stopper l'exécution du script, Il en résulte donc l'affichage du message d'erreur (passé en argument) et l'arrêt brutal du script.

- **mysqli_error(\$con):** renvoie une description de l'erreur MySQL.

Remarque : utilisation de require

On crée d'abord dans notre dossier de travail un fichier "connect.php" contenant le script suivant:

```
$con=mysqli_connect('localhost','root','','biblio') or die("erreur1".mysqli_error($con));
```

Et comme la connexion à la base va figurer dans tous les scripts PHP qu'on va créer, on va inclure le fichier "connect.php" dans les autres à l'aide de PHP

```
require 'connexion.php';
```

require : inclut le contenu d'un autre fichier appelé, et provoque une erreur bloquante s'il est indisponible.

C) Préparation et exécution de requête :

Préparation

```
$req="SELECT * FROM livre";
```

Variable de la connexion

Exécution

```
$res=mysqli_query($con,$req);
```

Variable de la requête

SELECT (Requêtes de sélection)

UPDATE, INSERT, DELETE... (Requêtes de mise à jour)

Exemple:

```
$Q="SELECT * FROM livre";
$res=mysqli_query($con,$q);
```

```
$Q= "DELETE FROM Eleves
WHERE MoyGen>=10";
$res=mysqli_query($con,$q);
```

Échec **FALSE**

Échec **FALSE**

Résultat :

Succès Une ressource (≡ Un fichier, dont chaque enregistrement/ligne est un tableau)

Succès **TRUE**

Déterminer le nombre de lignes dans la Ressource

Retournées dans la Ressource
`$Nb=mysqli_num_rows($con,$res);`

Affectées par la requête de MAJ

`$Nb=mysqli_affected_rows($con);`

Exploitation du résultat de la requête SELECT :

```
$Tab=mysqli_fetch_array($con,$res)
```

renvoie un tableau qui contient une ligne de la ressource **\$res** et déplace le pointeur vers la ligne suivante. Le résultat retourné par `mysqli_fetch_array($con,$res)` est

un tableau avec des **indices associatifs** et **numériques** en même temps. S'il n'y a plus de lignes dans

la ressource **res**, `mysqli_fetch_array($con,$res)` retourne **FALSE**.

```
While ($Tab=mysqli_fetch_array($R)){
    echo $Tab["Champ1"]." | ".$Tab["Champ2"];}
//Exemple de parcours complet d'une ressource
Fermeture de la connexion : mysqli_close();
```

La fermeture de la connexion est optionnelle parce qu'elle sera détruite automatiquement lorsque le script PHP termine son exécution.

4): Créer une page web pour afficher la liste des livres dans un tableau avec l'entête suivante ligne par ligne. Qu'on va la nommer **liste_livre.php**.

Identifiant	type du livre	titre du livre	Nombre d'exemplaires

Code PHP :

```
<?php
$cnx=mysqli_connect("localhost","root"
,"","biblio");
$req="SELECT* FROM livre";
$res=mysqli_query($cnx,$req);
echo"
<!DOCTYPE html>
<html>
<head>
    <title>Affichage des livres</title>
</head>
<body>
    <h1>Liste des livres</h1>
    <table>
    <tr>
    <th>Identifiant</th>
    <th>type du livre</th>
    <th>titre du livre</th>
    <th>Nombre d'exemplaires</th>
    </tr>
";
while($l=mysqli_fetch_array($res)){
    echo"<tr>
    <td>".$l['NumLivre']. "</td>
    <td>".$l['typeLivre']. "</td>
    <td>".$l['TitreLivre']. "</td>
    <td>".$l['NbreExemplairesDisp']. "</td>
    </tr>";
}
echo"</table></body></html>";
?>
```

5 : Modifier un enregistrement dans la table « Livre » à travers ce formulaire.

- A-Créer le formulaire suivant:

Modification d'un livre

Numero livre

Titre livre

- B Créer un script php pour **Modifier** un enregistrement dans la table « Livre » à travers ce formulaire.

Exemple: on va modifier le titre du livre don't le numLivre est:A0001 en modifiant le titre à **Les str de**

Modification d'un livre

Numero livre

Titre livre

controles au lieu de **Les structures de controles**.

Après exécution:

localhost/exercice1/modification.php

Modification de livre fait avec succès

☐ Tout afficher | Nombre de lignes : 25 | Filtrer les lignes: Chercher dans cette table | Trier sur l'index: Auc

+ Options				
	NumLivre	TypeLivre	TitreLivre	NbrExemplairesDisp
☐	A0001	Algorithmique	Les str de controles	15

Code PHP :

```

modification.php
1  <?php
2  //connexion
3  $conx=mysqli_connect('localhost','root','','biblio');/* or die("erreur");*/
4  if($conx)
5  | {echo('connexion au base réussi');}
6  else
7  | {echo("erreur de connexion");}
8  //récupération des données du formulaire
9  $livre=$_POST['livre'];
10 $titre=$_POST['titre'];
11 //requete
12 $req1="UPDATE livre SET TitreLivre= '$titre' where NumLivre='$livre'";
13 //exécution de la requette
14 $res1=mysqli_query($conx,$req1)/*or die*/;
15 echo("contenu de la variable res1:".$res1."<br>");
16 if ($res1)
17 | {echo("<script>alert('OK..');</script>");}
18 else
19 | {echo("<script>alert('Erreur..');</script>");}
20 }
21 if(mysqli_affected_rows($conx)!=-1)
22 | {echo("Modification de livre fait avec succès");}
23 mysqli_close($conx);
24 ?>

```

6 :

- Supprimer** un enregistrement de la table « Livre » à travers ce formulaire.

Exemple :on veut **supprimer** le livre dont le NumLivre est **B0132**

Suppression

code du livre

+ Options

	NumLivre	TypeLivre	TitreLivre
☐ Éditer Copier Supprimer	A0001	Algorithmique	Les str de c
☐ Éditer Copier Supprimer	B0132	base de données	SQL

Code PHP du fichier supprimer.php

.....

.....

[illegible]

Exemple:
on veut vérifier si un livre existe déjà dans la base de données
Exemple :Le livre dont le code est A001 existe déjà
copier le formulaire précédent un script PHP pour voir si un numéro de livre existe ou pas.

```
//requete
$req1="select * from livre where NumLivre='$livre'";
$res1=mysqli_query($conx,$req1)or die;
$nb=mysqli_num_rows($res1);
if($nb != 0)
{
    echo("deja existant!");
}
```

b) Modifier Le code de l'exercice n1 pour que avant de modifier un livre on vérifie s'il existe ou non.

Application N°2 (S.T.I)

Important :

Dans votre dossier c:\xampp_lite\www, créez votre dossier de travail intitulé **SWNom_prénom**. 'Exp : SWArbi_foulen'

Créer une base de données en la nommant **BDNom_prénom**. 'Exp : BDArbifoulen'

Une banque offre à ses clients un service de paiement à distance à travers un site web. En effet, nous allons développer des pages permettant de :

- Réaliser une transaction bancaire (paiement facture, loyer, virement, salaire, ...)
- Lister les transactions réalisées par un client.

Travail demandé :

1. Dans la base de données « BDNom_prénom », créer les tables suivantes :

- **Compte** (**NumCompte**, Titulaire, type, MotPasse, Solde, DateOuvr)
- **Transaction** (**Cdtrans**, NumCompte#, DateTrans, Libelle, Montant)

Le tableau suivant indique les noms et les types des champs de chaque table.

Champ	Description	Type
--- Table Compte ---		
1 Numcompte	Numéro du compte	Chaîne de 10 Chiffres
2 Titulaire	Nom et prénom ou raison social	Chaîne de 35 caractères
3 Type	Type du compte (Courant, Epargne, joint, ...)	Chaîne de 25 caractères
4 MotPasse	Mot de passe d'accès au compte	Chaîne de 10 caractères
5 Solde	Solde du compte	Décimal (10,3)
6 DateOuvr	Date d'ouverture du compte	Date
--- Table Transaction ---		
1 Cdtrans	Code de la transaction	Entier (auto-incrémenté)
2 NumCompte	Numéro du compte	Chaîne de 10 chiffres
3 DateTrans	Date de la transaction	DateTime
4 Libelle	Libellé de la transaction	Chaîne de 40 caractères
5 Montant	Montant de la transaction	Décimal (10,3)

2. Insérer dans la table « Compte » les lignes suivantes :

NumCompte	Titulaire	Type	MotPasse	Solde	DateOuvr
1111222212	Arbi Farid	Epargne	AF111000fa	1500,500	2010-05-20
1111222213	Ismail wafa	Courant	EW101010we	2950,820	2020-10-28
1111333314	Tounsi Arbia	Courant	TOTAaA2020	-50,000	2020-11-04
1111444415	Ste Farhan+	Courant	SF+0550+in	183000,000	2005-06-13

3. Insérer dans la table « Transaction » les lignes suivantes :

Cdtrans	NumCompte	Datetrans	Libelle	Montant
101	1111222212	2010-05-20	Loyer	400,000
102	1111222212	2020-11-04	Virement	180,500
103	1111444415	2005-06-13	Virement simple	1220,000

4. Créer une page intitulée « transaction.html » contenant le formulaire suivant :

Saisie d'une transaction

Le clic sur le bouton '**Valider**' fait appel à :

- ♣ Une *fonction Javascript* intitulée `verif()` affectant les contrôles suivants :
 - Le champ "**Numéro du compte**" doit être composé de 10 chiffres.
 - Le champ "**Montant**" doit contenir une valeur strictement positive,
- ♣ Un *script php* enregistré dans une page nommée « `action.php` » permettant, si c'est possible :
 - L'ajout des informations relatives à la table '**transaction**'.
 - La mise à jour de champ '**solde**' de la table '**Compte**'.

N.B :

La transaction est possible si le numéro du compte saisi existe dans la table 'Compte' et le montant est inférieur ou égal au solde ; sinon la transaction est impossible et la message 'transaction échouée' s'affichera.

Le clic sur le bouton '**Annuler**' initialise les champs du formulaire.

5. Créer une page nommée « `liste.html` » contient le formulaire suivant :

Le clic sur le bouton `lister` fait appel à :

- ♣ Une *fonction Js* vérifiant que :
 - Le champ "**Numéro du compte**" est formé de 10 chiffres.
 - Le champ "**mot de passe**" est composé de 10 caractères.
- ♣ Un *script Php*, dans une page nommée '`Liste.php`' permettant d'afficher :
 - Le message "*Veuillez vérifier les paramètres de votre identification*" dans le cas où **le couple** (*numéro du compte et le mot de passe*) saisi n'existe pas dans la table `Compte`.
 - En cas de validité, un tableau formé par les champs `[DateTrans]`, `[Libelle]`, et `[montant]` ainsi que le nombre de transactions effectuées par ce client.

N.B :

Le clic sur le bouton '**annuler**' initialise les champs du formulaire.

Correction ;

Q4 :

Html :

```

<!DOCTYPE html>
<html>
<head>
  <title>Hello!</title>
  <link rel="stylesheet" href="forme.css">
</head>
<body>

<form id="FR" name="F" action="ajout.php" method="POST">
Date de transaction : <input id="D" name="Dt" type="datetime-local">
<br>
Numéro du compte : <input id="N" name="Nt" type="text">
<br>
Libellé de la transaction : <select id="L" name="Lt" size="1">
<option value="Virement simple">Virement classique</option>
<option value="Loyer">Loyer</option>
<option value="payement facture">payement facture</option>
<option value="Salaire">Salaire</option>
<option value="Don">Autre...</option>
</select> <br>
Montant: <input id="M" name="Mt" type="text">
<br>
<input name="B1" type="submit" value="Valider">
<input name="B2" type="reset" value="annuler">
</form>

</body>
</html>

```

Ajout.php

```

<!DOCTYPE html>
<html> <head> <title>transactionPHP</title> </head>
<body>
<?php

if (isset($_POST['Dt']))
{
//récupérer les données introduites dans les champs du formulaire..
//-----
$vb=$_POST['Dt'];
//echo $vb;
//$d=date("Y-m-d H:i:s") ;
echo $d;
$va=$_POST['Nt'];
$vc=$_POST['Lt'];
$vd=$_POST['Mt'];
//----- Se connecter au serveur local-----
$con=mysqli_connect("localhost","root","","bdsifoulen")or die("erreur connection au
serveur");
if ($vd<0)
{echo("<script> alert('Erreur ... verifie le montant de la transaction');</script>") ;}
else
{

```

```

$rq="select numcompte from compte where (numcompte='$va') and solde>='$vd'";
$res=mysqli_query($con,$rq);
$nb=mysqli_affected_rows($con); //Retourne Le nombre de Lignes affectees Lors de la
//derniere operation MySQL.
//Il est possible d'utiliser 'mysql_num_rows()': Elle retourne Le nombre de lignes
//d'un resultat d'execution d'une requete SQL <SELECT .. From >.
// $nb=mysqli_num_rows($res);
if($nb<=0)
    //Le resultat de la requete est 0 si $va n'existe pas dans la table
    // Compte ou -1
    //suite a une erreur d'execution...
    {echo($nb." vérifier numéro du compte ou contacter votre agence ou solde
insuffisant!");}
else
{
    $rq1="insert into transaction values(NULL,'$va','$vb','$vc','$vd')" ;
    // Le premier champ sera incrémenté automatiquement..
    $rq2="update compte set solde=solde-$vd where (numcompte='$va')";
    // mise à jour du solde dans la table compte
    //Exécuter les deux requetes et récupérer le resultat...
    mysqli_query($con,$rq1);
    $n1=mysqli_affected_rows($con);
    //Retourne Le nombre de Lignes affectées Lors de
    //La dernière opération MySQL
    if($n1==1)
    {echo "r1 reussi";}
    mysqli_query($con,$rq2);
    $n2=mysqli_affected_rows($con);
    if($n2==1)
    {echo "r2 reussi";}
    if($n1==1 && $n2==1) // une ligne insérée dans la table
    // transaction et
    //une autre modifiée dans la table compte
    {echo "OK--Transaction validée";}
    else
    {echo "Erreur --problème d'insertion et de mise à jour... vérifier vos données";}
    /*
        ou bien
    $ex1=mysqli_query($con,$rq1);
    $ex2=mysqli_query($con,$rq2);
    if ($ex1 && $ex2)
    {echo "OK--Transaction validée";}
    else
    {echo "Erreur --problème d'insertion... vérifie vos données";}
    */
}
}
mysqli_close($con);
}

```

```
?>
</body></html>
```

Q5: Liste.html

```
<!DOCTYPE html>

<html>

<head>
  <title>Hello!</title>
</head>

<body>
<form id="F" name="FL" action="liste.php" method="POST">
  <h2>Liste des transactions </h2>
  Numéro du compte: <input id="Nc" name="Nc" type="text">    <br>
  Mot de passe: <input type="text" name="Mp">    <br>
  <input id="b1" name="ba" type="submit" value="Lister"> <br>
  <input id="b2" name="bn" type="reset" value="annuler">
</form>
</body>
</html>
```

Liste.php

```
<!DOCTYPE html>
<html>
<head>
  <title>Liste des trans</title>
</head>
<body>
<?php

if ((isset($_POST['Nc']))&&(isset($_POST['Mp'])))
{
  //récupérer les données introduites dans les champs du formulaire..
  //-----
  $cp=$_POST['Nc'];
  $mp=$_POST['Mp'];
  $con=mysqli_connect("localhost","root","","bdsifoulen");
  if (!$con)
  {echo("erreur connection au serveur");}
  else
  {
    // pour vérifier que le numéro du compte et le mot de passe ...
    $rq="select * from compte where (numcompte='$cp') and (MotPasse='$mp)";
    $res=mysqli_query($con,$rq);
    $nL=mysqli_num_rows($res);
```

```

if ($nL==0) //numéro de compte ou mot de passe invalide : 0 enregistrement lors de
l'exécution de la requête $rq...
{echo(" vérifier les paramètres de votre identification");}
else
{
    $rq1="select Datetrans,libelle,Montant from transaction where (numcompte='$cp')";
    $rs1=mysqli_query($con,$rq1);
    echo ("<table border='1'
    <tr>
    <th> Datetrans </th> <th>Libellé trans </th> <th> Montanttrans</th>
    </tr>");
    //plusieurs transactions seront affichées
    while($ligne=mysqli_fetch_array($rs1))
    {echo ("<tr>
    <td> $ligne[0]</td>
    <td> $ligne[1]</td>
    <td> $ligne[2]</td>
    </tr>");}
    echo"</table>";

    $rq2="select count(numcompte) from transaction where (numcompte='$cp') group
by(numcompte)";
    $rs=mysqli_query($con,$rq2)or die ("erreur exécution troisième requête");
    $lig=mysqli_fetch_array($rs);
    {echo ("<br> Le nombre des transactions  :".['.$lig[0].']);}

}
}
mysqli_close($con);
}
?>

</body>
</html>

```

Application3 PHP**(Utilisation des fonctions sur les chaînes de caractères+ fonction sur les dates)**

1. Créer la base de données « **bdclasse** » contenant la table « **eleve** » dont la structure est la suivante :

eleve	type
Numero	Entier
Classe	auto-incrément
Nom	Chaîne
Prenom	Chaîne
Note	Chaîne
Date de naissance	float
	date

1	Numero 	int(11)		Non	Aucun(e)	AUTO_INCREMENT
2	Classe	varchar(8)	utf8_unicode_ci	Non	Aucun(e)	
3	Nom	varchar(35)	utf8_unicode_ci	Non	Aucun(e)	
4	Prénom	varchar(35)	utf8_unicode_ci	Non	Aucun(e)	
5	Note	float		Non	Aucun(e)	
6	date de naissance	date		Non	Aucun(e)	

- 1-Créer le formulaire suivant

Nom et Prénom **Classe** **Choisir une classe** ▼

Evaluation 3pts (1+1+1)

La lettre "A" est ☐ consonne ☐ voyelle

Ecrire le mot "classe" en majuscule

Le mot "TIC" est composé par

un
deux
trois

 Lettres

- Ecrire un script qui permet de contrôler la saisie des champs de la façon suivante :
 - Le champ Nom et prénom non vide et contient l'espace
 - L'élève doit choisir une classe
- Si les champs sont bien rempli, un script coté serveur sera exécuter pour permettre
 - De calculer la note de l'élève
 - D'insérer le nom et prénom, la classe ainsi que la note la **date courante** dans la base de donnée « **bdclasse** »

Correction**Formulaire.html**

```

<html><head>
<title>NomClasseN°</title>
<script language=javascript src= "lib.js" _>
</script>
</head>
<body bgcolor="#C0C0C0">
<form method="POST" action="ajout.php" name="f"
onsubmit="return note(f);">
  <p><b>Nom et Prénom</b><input name="T1" size="20" style="font-weight:
700">
<b>Classe</b><select size="1" name="D1" style="font-weight: 700">
<option>Choisir une classe</option>
<option value="première">première</option>
<option value="deuxième">deuxième</option>
<option value="troisième">troisième</option>
</select></p>
<hr>
<p align="left"><u><b>Evaluation 3pts (1+1+1)</b></u></p>
<p>La lettre &quot;A&quot; est
<input type="radio" value="consonne" name="R1">consonne
<input type="radio" name="R1" value="voyelle">voyelle</p>
<p>Ecrire le mot &quot;classe&quot; en majuscule
<input type="text" name="T2" size="20"></p>
<p>Le mot &quot;TIC&quot; est composé par <select size="3" name="D2">
<option value="un">un</option>
<option value="deux">deux</option>
<option value="trois">trois</option>
</select>Lettres</p>
<input type="submit" value="ajouter" name="B1" ></form></body></html>

```

Fichier « lib.js »

```
function note(f)
```

```

{np=document.getElementById("T1").value;p= document.getElementById("D1").selectedIndex;
if((np=="")||(np.indexOf(' ')== -1)){alert("erreur np "); return false;}
if (p== -1){alert("erreur choisir une classe"); return false;}}
```


Ajout.php

```

<?php
//r cup ration des donn es
$v1=$_POST['T1'];
$v2=$_POST['D1'];
$v3=$_POST['R1'];
$v4=$_POST['T2'];
$v5=$_POST['D2'];
// calcul de La note et pr paration des champs   ins rer
$note=0;
if ($v3=='voyelle'){$note=$note+1;}
if ($v4=='CLASSE'){$note=$note+1;}
if ($v5=='trois'){$note=$note+1;}
$d=date("Y/m/d") ; // r cup rer la date courante
$P=strpos($v1," ");
$No=substr($v1,0,$P);
$Pr=substr($v1,$P+1,strlen($v1)-$P-1);
// Connexion   la base
$conx=mysqli_connect('localhost','root','','bdclasse');

$req="insert into eleve values (NULL,'$v2','$No','$Pr','$note','$d)";
$res=mysqli_query($conx,$req);
if ($res==1){echo("insertion r ussie");}else{echo
mysqli_error($conx);}
?>

```